Solutions Architecture Overview — Selected Professional Experiences

This document presents a consolidated overview of the solutions architectures delivered across three major professional experiences, spanning complex digital platforms, cloud-native systems, and data-driven solutions. It reflects an end-to-end architectural approach covering business requirements, system design, integration patterns, data platforms, security, and cloud infrastructure, across different technology stacks and maturity stages.

The document is organized around three distinct solution architectures, each addressing different business domains and technical challenges:

- A B2B Automotive Auction Platform built on Microsoft Azure
- A Multi-tenant Hotel Digital Access Platform deployed on Amazon Web Services
- A Digital Learning Platform, presented through an architecture evolution from a 2019 design to a native 2025 architecture

For each experience, the document provides multiple architecture views, aligned with industry best practices, including:

- Executive and contextual views to position the solution within its business ecosystem
- Logical and functional views detailing core components, services, and interactions
- Integration and data architecture views highlighting system interoperability and data flows
- Physical and deployment views illustrating cloud services, scalability, availability, and resiliency patterns
- Security, governance, and control plane considerations where applicable

Experience 1 — B2B Automotive Auction Platform (Azure)

This section describes the architecture of a cloud-based B2B automotive auction platform, leveraging remote access, containerized microservices, and serverless APIs to deliver scalability, high availability, and seamless web and mobile access.
It also presents the data integration architecture connecting the auction platform with an external DMS system (EAM & CRM SaaS) into a centralized solution data warehouse, including the design of a star-schema analytical model and automated ingestion pipelines supporting business intelligence and reporting.

Experience 2 — Hotel Digital Access Platform (AWS)

This section covers the design of a multi-tenant hospitality platform on AWS, architected for high availability, fault tolerance, security, and elastic scalability.
It details a container-based microservices architecture, secure integration with mobile digital key technologies, and the design of an automated customer lifecycle management platform orchestrating cloud and on-prem systems, third-party SaaS integrations, and cryptographic trust across access control components.

Experience 3 — Digital Learning Platform (Architecture Evolution 2019 → 2025)

This section documents the architectural evolution of a digital learning platform, explicitly presenting:

- The 2019 reference architecture, reflecting the initial platform design and constraints

- The 2025 native architecture, illustrating the target-state evolution with modernized data, security, governance, and ML capabilities

The comparison highlights how the platform evolved to support scalability, advanced analytics, intelligent services, and fine-grained governance, while maintaining architectural coherence and cloud portability.

Together, these architectures demonstrate a consistent architectural approach focused on scalability, resilience, security, data governance, and long-term evolvability, while adapting to different domains, cloud providers, and technology generations.

1. B2B Automotive Auction Platform on Azure

Architecture & Design Description

- Designed a **highly available, serverless B2B automotive auction platform** on Azure, supporting **web and mobile clients** for up to ~300 B2B partners/dealers over the first three years.
- Implemented **Azure Virtual Desktop (AVD)** for secure remote access by internal staff; the **initial rollout focuses exclusively on AVD**, with public mobile access added later.
- **Backend architecture** built with **microservices on Azure Container Apps**, exposing RESTful APIs via **public ingress**, allowing both AVD and mobile clients to access backend services directly. No API Gateway or Application Gateway was used initially to optimize **budgeting and simplicity**, given the small user base and limited geographic deployment for the first releases (Montréal and Toronto, with Vancouver planned over the next 3–4 years). **The architecture was designed to support future expansion to additional cities and the introduction of a B2C module, leveraging the same backend services for mobile and web application interfaces, while the web application is exclusively intended for B2C use cases.**
- Integrated external enterprise systems (**GEM-DMS, EAM, CRM SaaS modules**) using **secure API connectors** for real-time synchronization of vehicle sale data, dealer bidding, and purchases.
    - Developed a **simplified Azure analytics solution**:
    - Designed a **star-schema Data Warehouse** in Azure SQL Database.
    - Ingested auction sales via **Azure Functions** into a staging area.
    - Orchestrated data pipelines using **Azure Data Factory** to populate dimension tables from CRM, EAM, and auction sales, powering **Power BI dashboards**.
    - Enforced **identity & access management** using Azure Entra ID, securing both user and service accounts.
- Network & security pattern simplified for the first deployment:
    - Public ingress to Azure Container Apps for mobile and AVD access.
    - Private subnets for database access.
    - Use of **private endpoints** for SaaS connectivity.
- **High availability** ensured within deployed regions using Azure Container Apps scaling and database redundancy, despite limited geographic scope.

Key architectural patterns applied:

- Serverless microservices
- Event-driven integration
- Multi-tenant SaaS architecture (small scale)
- Star-schema data warehouse for analytics
- Cloud-native security & private connectivity

Architecture & Design Description

- Cost-optimized design for small user base and limited regions

High level Architecture Diagrams

I. **Cloud (Azure) High level Architecture / Diagram 1:** Access to the application

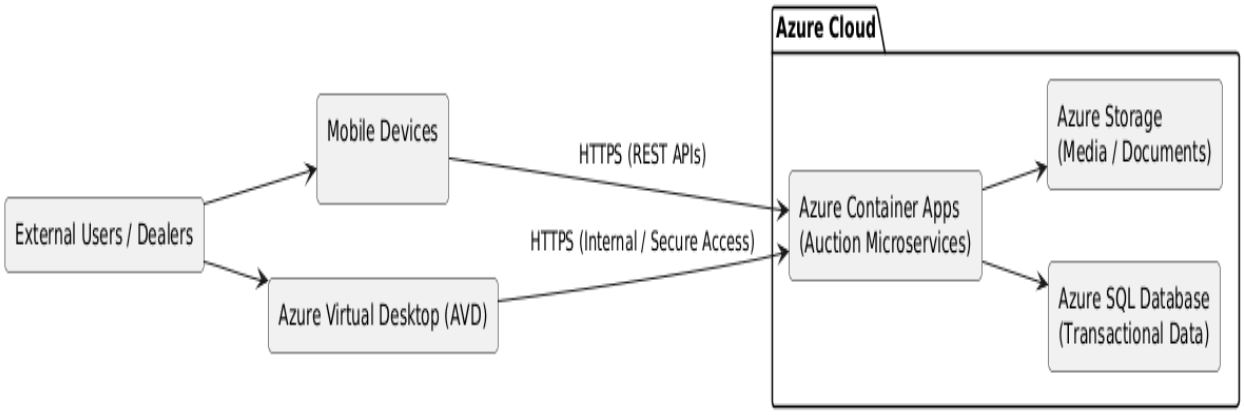


Diagram explanation:

- **Public ingress** on Azure Container Apps replaces API Gateway/Application Gateway for this budget-conscious, small-scale deployment.
- **AVD** provides internal remote access, while mobile clients access backend directly via the same public ingress.
- Backend microservices handle all auction, bidding, and reporting logic.
- Analytics pipelines ingest and transform data into the **Azure SQL Data Warehouse**, feeding **Power BI dashboards**.
- SaaS integrations remain secure via **private endpoints**.

Communication between Azure Virtual Desktop with Azure Container apps public ingress enabled

Understanding the Traffic Flow

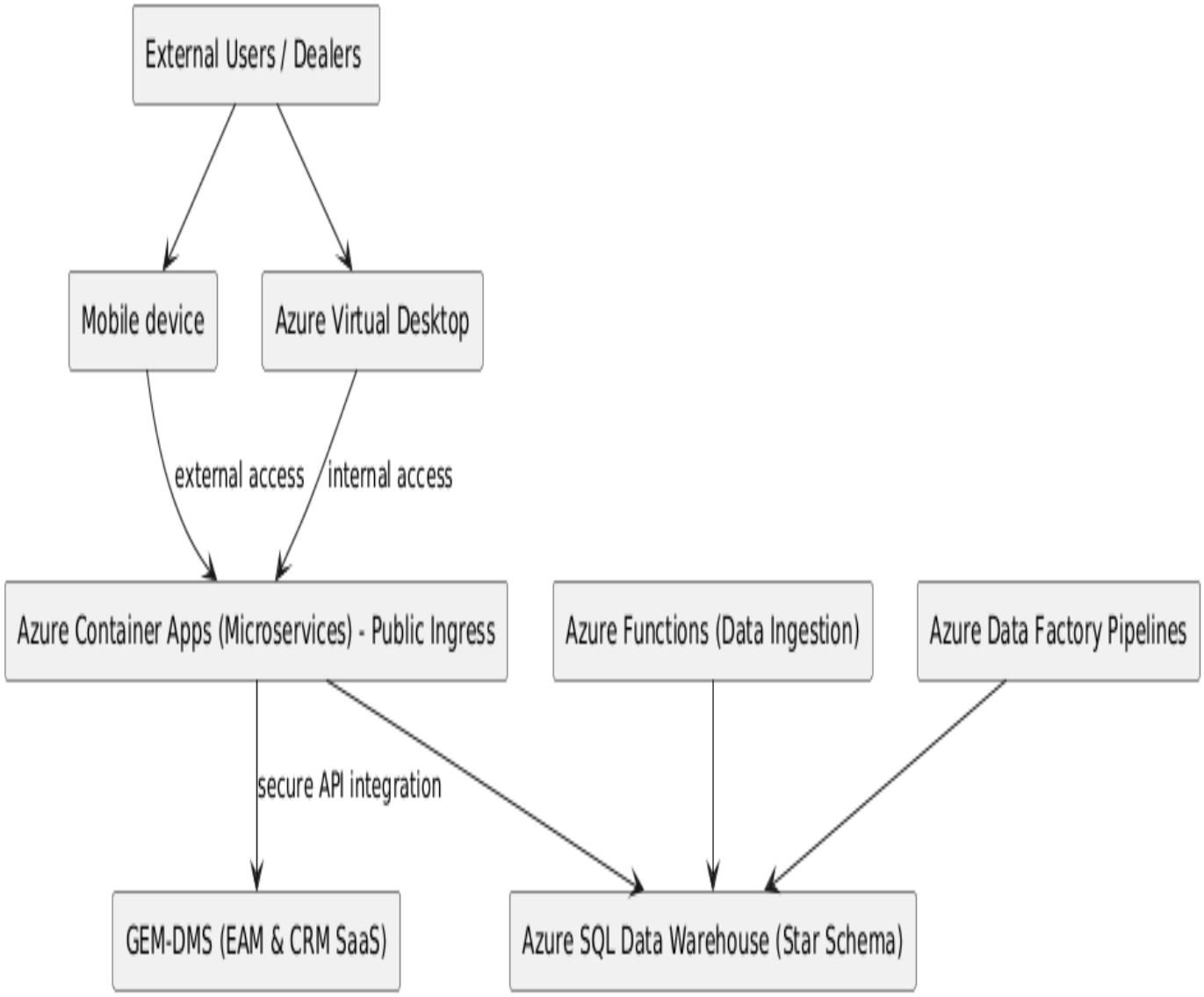
- Public Ingress: When a container app has public ingress enabled, it is accessible from the public internet via a public IP address.
- AVD Access: AVD session hosts are deployed within our Azure Virtual Network (VNet).
- Accessing a Public Endpoint from within a VNet: By default, resources within a VNet can access public endpoints on the internet. Therefore, our AVD session hosts can reach the container app's public URL through the internet (outbound from VNet, inbound to the container app's public IP). This does not keep the traffic exclusively internal to the Azure backbone network.

Achieving "Internal Remote Access" via AVD:

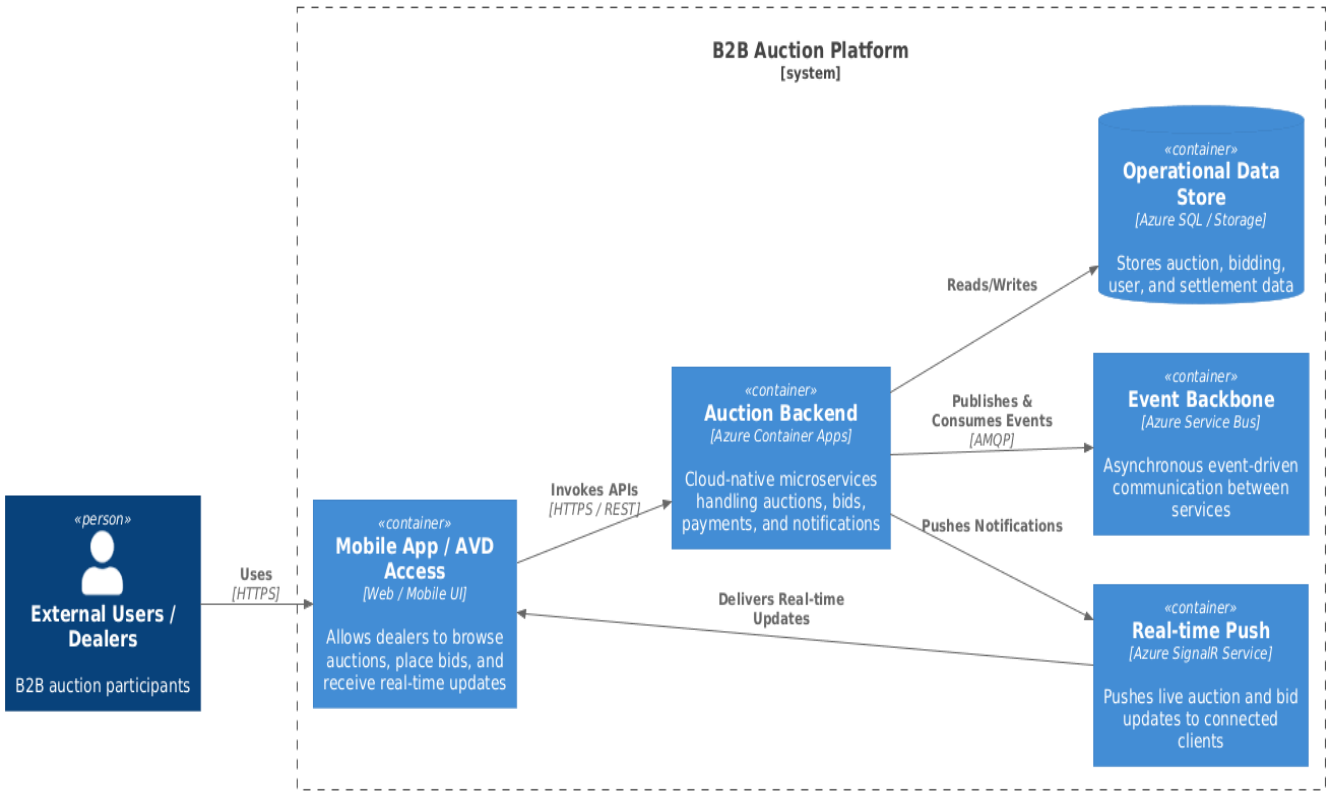
To ensure the traffic between our AVD session hosts and the container app remains internal to our private network (Microsoft's backbone), we can use one of these two main approaches:

- **Use Private Endpoints (Recommended for security):** This approach maintains public ingress for external users but forces traffic from our VNet (and thus our AVD session hosts) through a private route.

Cloud Solution High Level Architecture (Diagram 2): Integration GEM-DMS system & Data warehouse solution

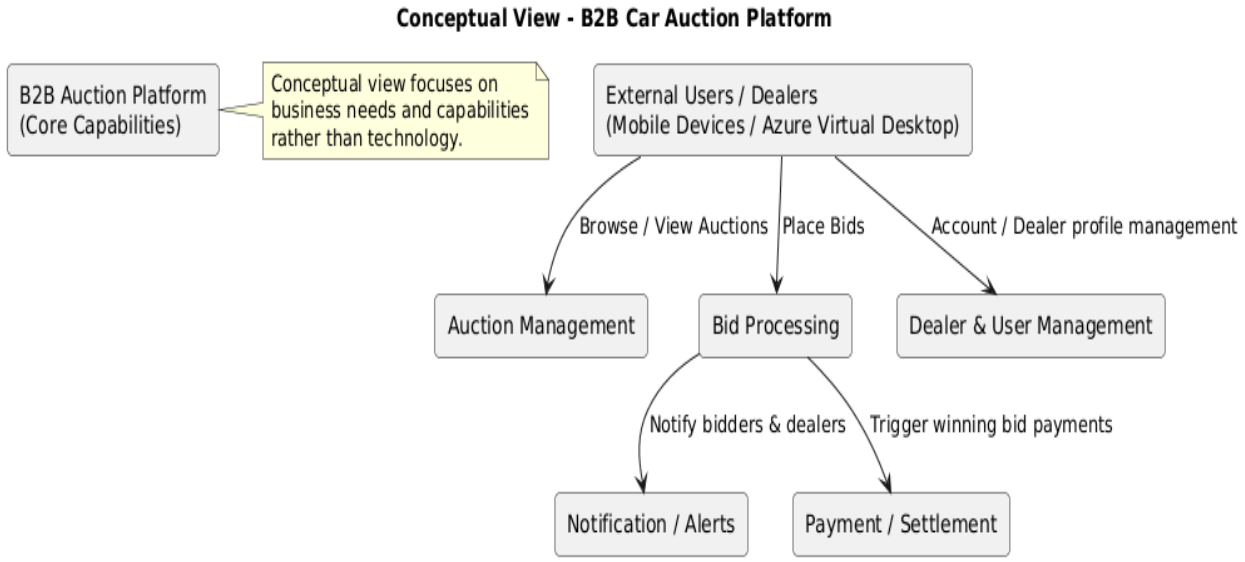


I. **Executive Architecture – B2B Automotive Auction Platform (Diagram 4):** Highlights major containers & technology choices



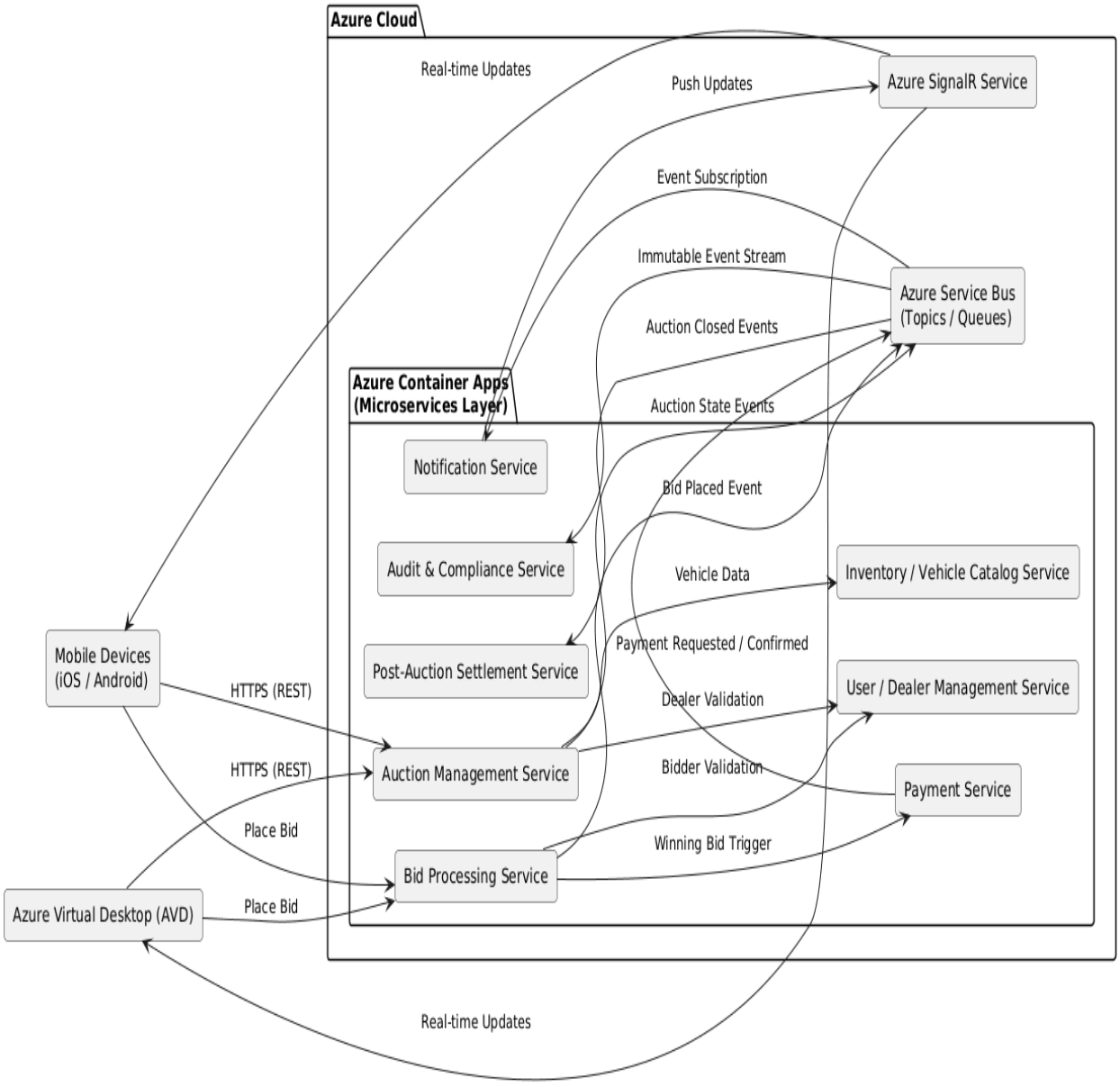
Illustrates system boundaries, major containers, and key technology choices. It gets complemented by the conceptual view which translates the business needs into a high-level logical system structure.

II. **Conceptual Architecture – B2B Automotive Auction Platform (Diagram3):** High level business specifications

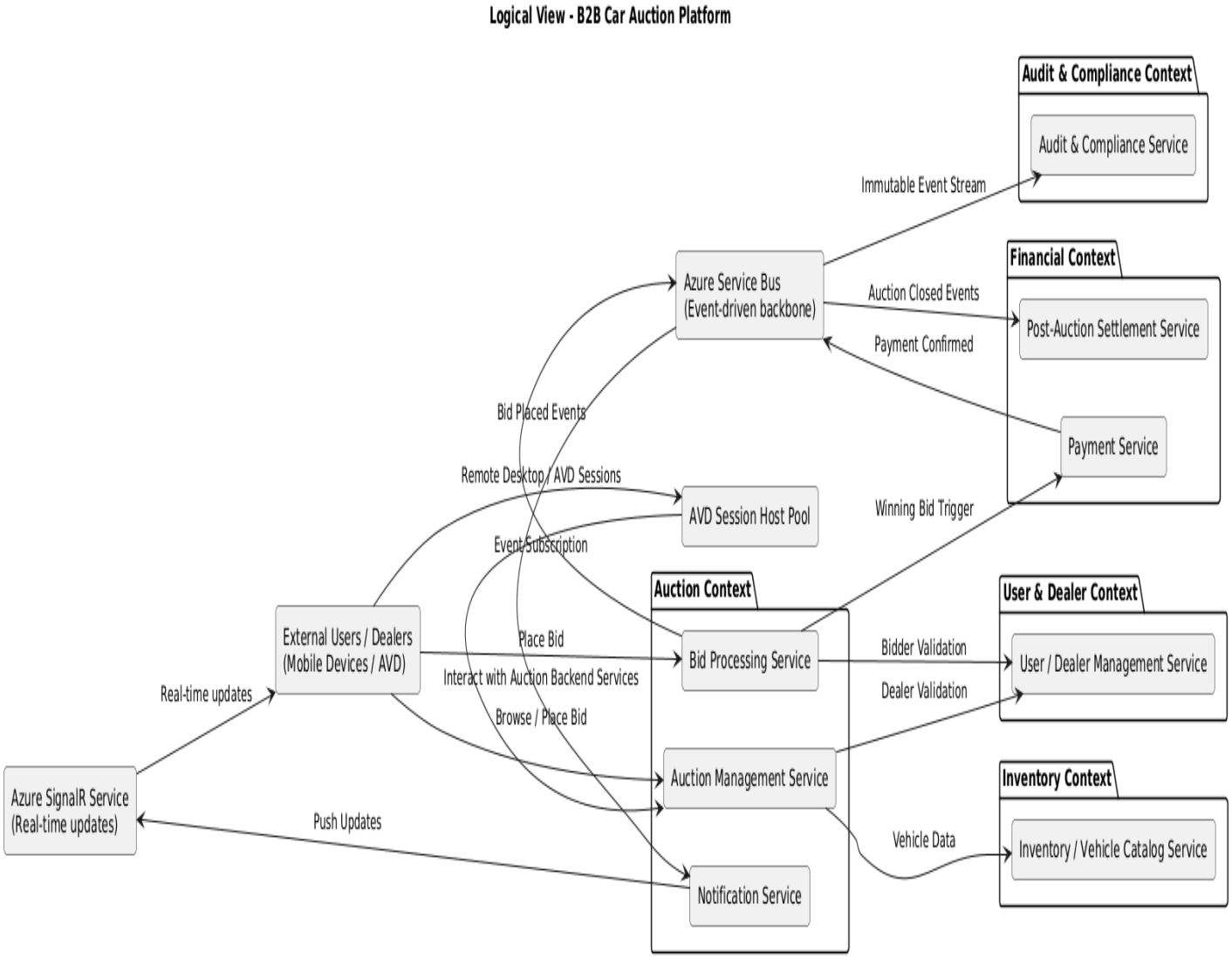


III. **Logical Architecture – B2B Automotive Auction Platform:** Microservices Design & Bounded Context diagrams

- **Diagram 4: Microservices design (Restful AP)**



- **Diagram 5 (Bounded Context)** Shows how the system is organized functionally, defining service responsibilities, domains, and interactions without specifying physical deployment or exact technology. Focuses on *how the system works*.



The **Bounded Context Diagram** is presented as a **Logical View**, illustrating how **microservices** are grouped into **business domains**, their responsibilities, and interactions. This complements the Conceptual and Executive Views by showing how the system works internally, without detailing deployment specifics. Microservices are grouped into bounded contexts reflecting core business domains: Auction, Financial, User & Dealer, Inventory, and Audit & Compliance. This design clarifies responsibilities, enables independent evolution, and aligns with event-driven architecture principles."

Bounded Contexts Overview

Purpose:

- Show domain-driven boundaries
- Explain why services are separated
- Highlight future extensibility

Contexts for our platform:

1. **Auction Context**
    - Auction Management Service
    - Bid Processing Service
    - Notification Service
2. **Financial Context**
    - Payment Service
    - Post-Auction Settlement Service
3. **User & Dealer Context**
    - User / Dealer Management Service
4. **Inventory Context**
    - Inventory / Vehicle Catalog Service
5. **Audit & Compliance Context**
    - Audit & Compliance Service

Microservice Descriptions

Auction Management Service

Manages the full auction lifecycle, including creation, scheduling, state transitions, reserve rules, and auction closure. Acts as the orchestration point for auction-related business logic.

Bid Processing Service

Handles high-frequency bid submissions with concurrency control, validation, and ordering guarantees. Publishes bid events asynchronously to ensure scalability and responsiveness under peak bidding activity.

User / Dealer Management Service

Manages dealer profiles, roles, and access rules. Integrates with enterprise identity (Azure Entra ID) for authentication while maintaining application-specific authorization and dealer metadata.

Inventory / Vehicle Catalog Service

Maintains vehicle metadata, auction listings, and associated documents. Provides a normalized view of auction inventory consumed by auction and bidding services.

Payment Service

Handles payment initiation, authorization, and confirmation for winning bids. Ensures transactional integrity and publishes payment state changes for downstream processing.

Post-Auction Settlement Service

Manages post-auction workflows including settlement finalization, ownership transfer, invoicing triggers, and downstream system notifications following successful payment.

Notification Service

Subscribes to domain events and pushes real-time updates (bid changes, auction status, payment confirmation) to connected clients using Azure SignalR.

Audit & Compliance Service

Consumes immutable event streams to maintain a complete, tamper-resistant audit trail of auctions, bids, and payments, supporting traceability, compliance, and dispute resolution.

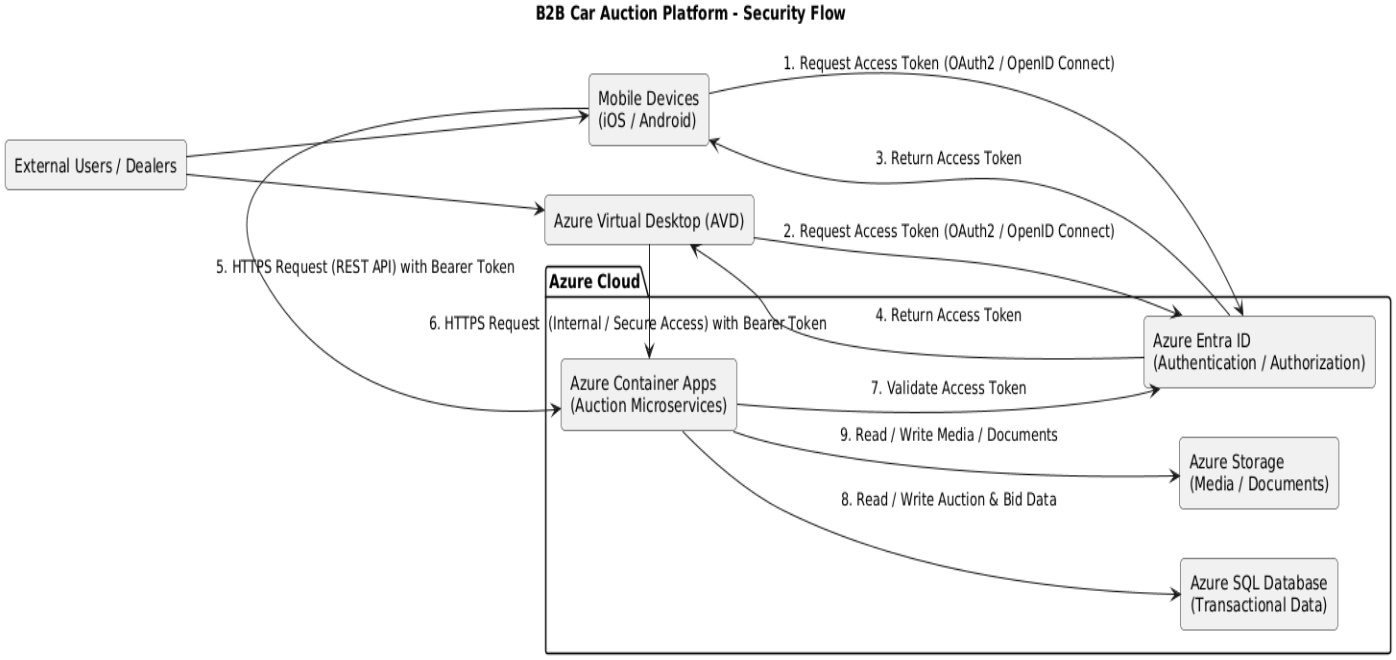
The Microservices architecture provides

- **Loose coupling** via Service Bus
- **Event-driven design** (bids, auctions, payments)
- **Scalable real-time fan-out** using Azure SignalR
- **Single backend** serving **multiple client types**

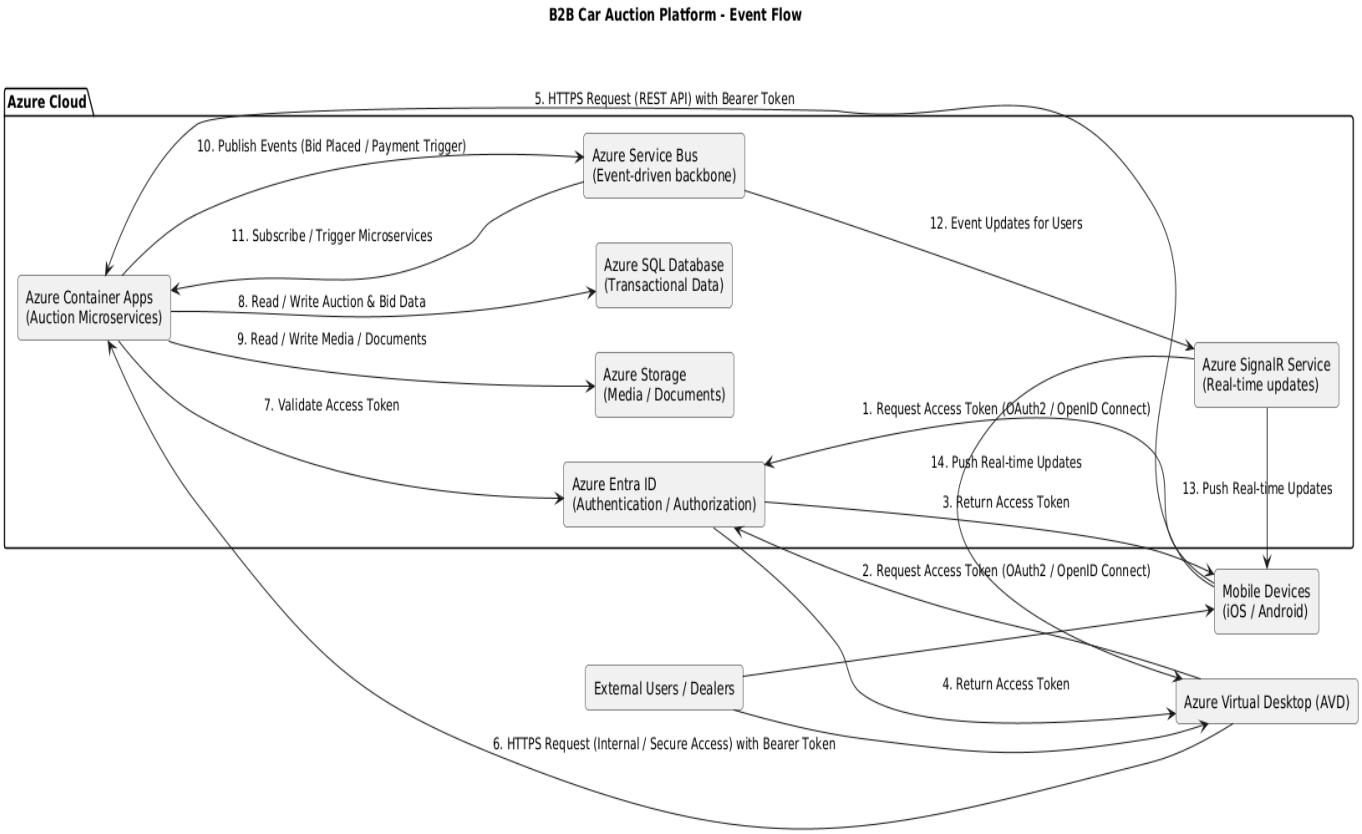
Service responsibilities

- **Auction Management** → lifecycle & rules
- **Bid Processing** → concurrency-safe bidding
- **Payment Service** → transactional boundary
- **Notification Service** → real-time & async messaging

Security Authentication & Authorization Flow (Diagram 7)

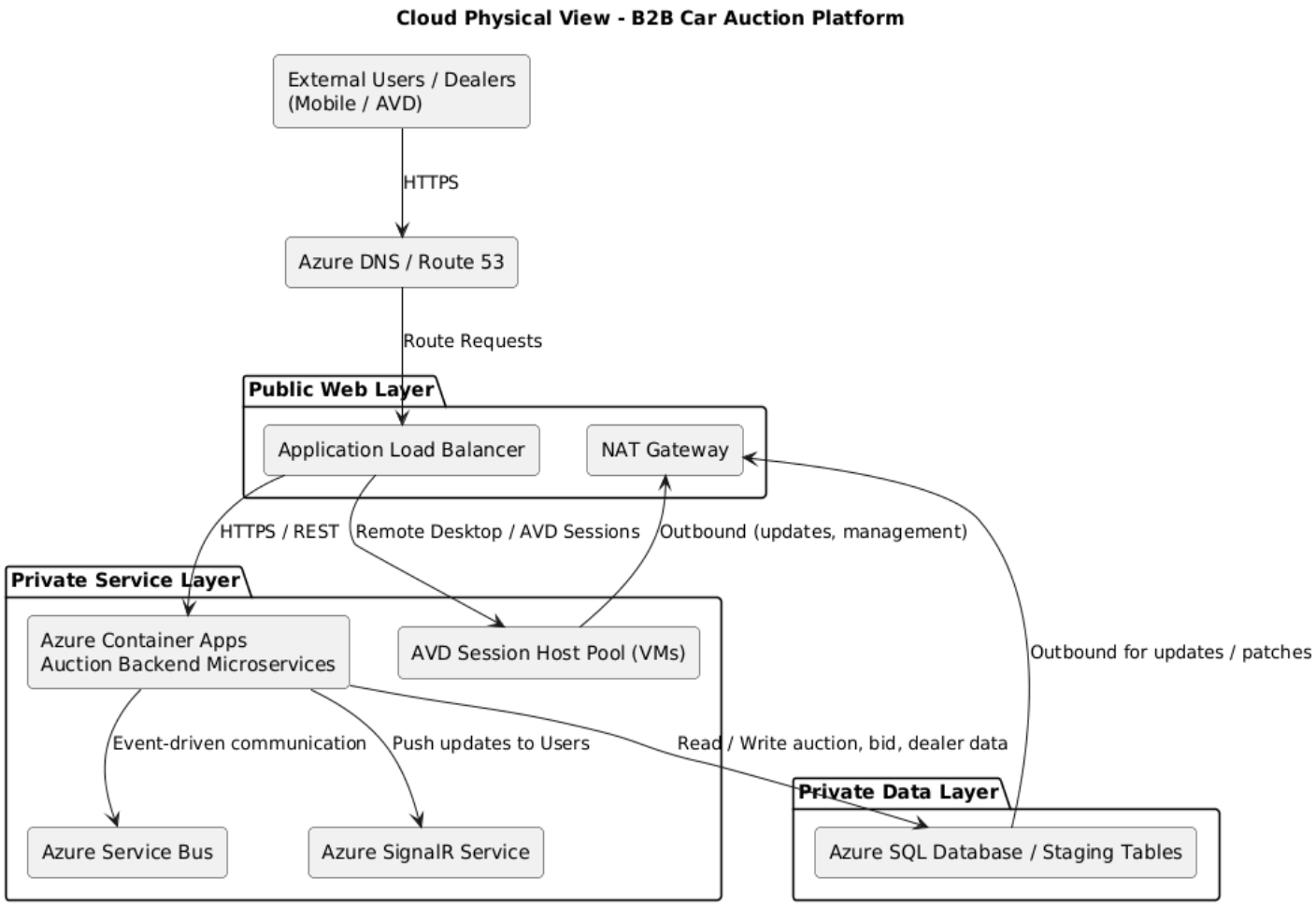


Event Flow Diagram 8: Illustrating the end-to-end user request flow and events



Architecture & Design Description

IV. Physical Architecture – B2B Automotive Auction Platform (Diagram 6)



The physical view diagram

- Shows cloud infrastructure: ALB, NAT, private/public subnets (abstracted)
- Shows backend microservices, messaging, real-time updates, database
- Physical deployment concepts: secure private service layer, data layer, outbound connections

Dedicated private IP subnet for AVD session hosts:

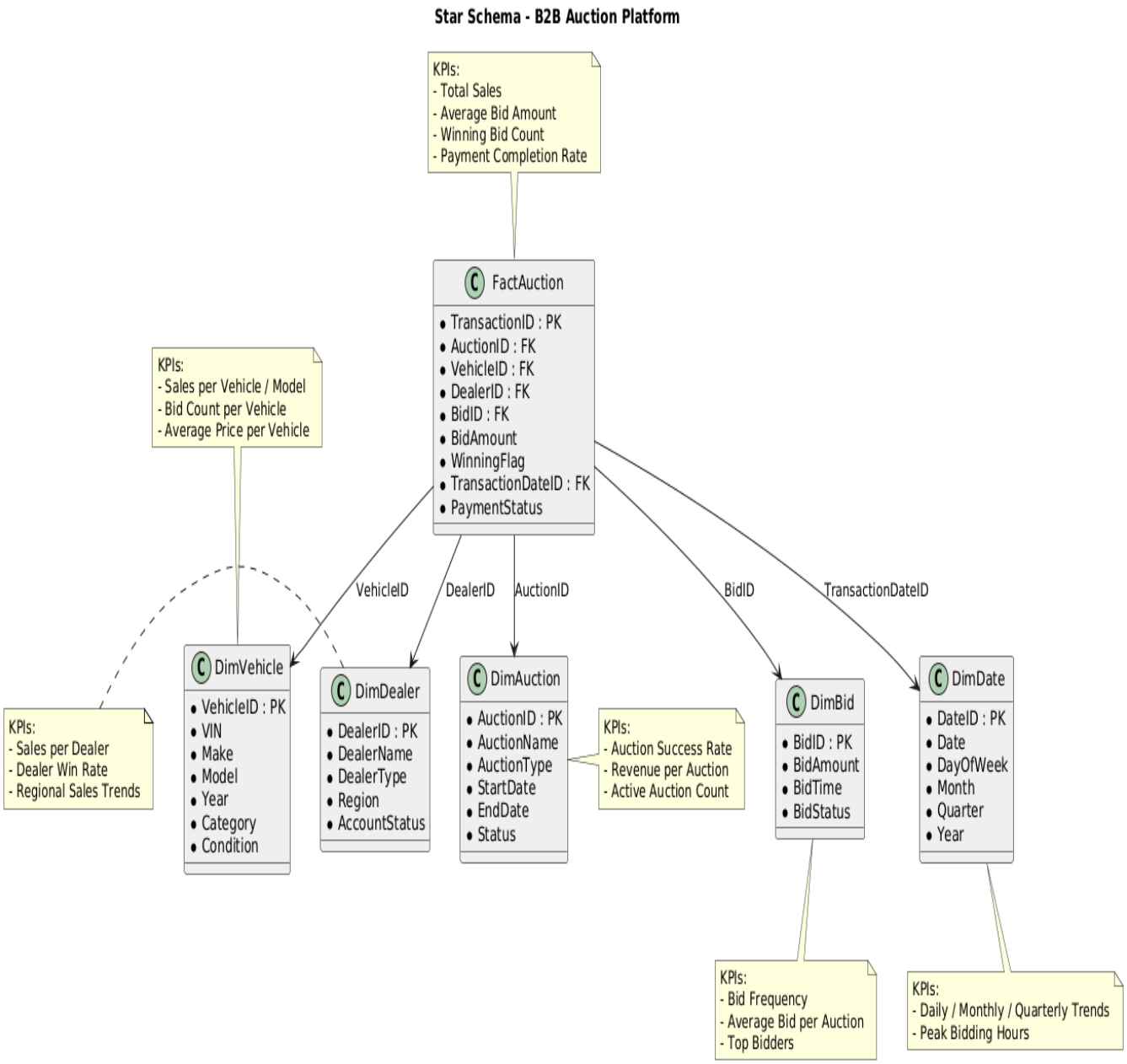
I. Isolation: Keeps VM-based session hosts (AVD) separated from containerized microservices, reducing blast radius in case of misconfigurations or security issues.

II. Network Security: Allows distinct network security groups (NSGs) and routing rules for session hosts vs. microservices.

III. Scaling / Maintenance: We can scale or patch session host VMs independently without affecting microservices.

IV. Compliance & Monitoring: Easier to monitor resource consumption, access, and logging separately.

Data warehouse star schema & KPIs Analytic view (Diagram 9): Star schema fact & dimension tables & KPIs



The above schema illustrates the analytical structures (data warehouse star schema fact and dimension tables) and their KPI consumption through Power BI

Architecture & Design Description

Conceptual Architecture View for Data Warehouse data – B2B Automotive Auction Platform (Diagram 10):
Illustrates how services provide fact and dimension tables data.

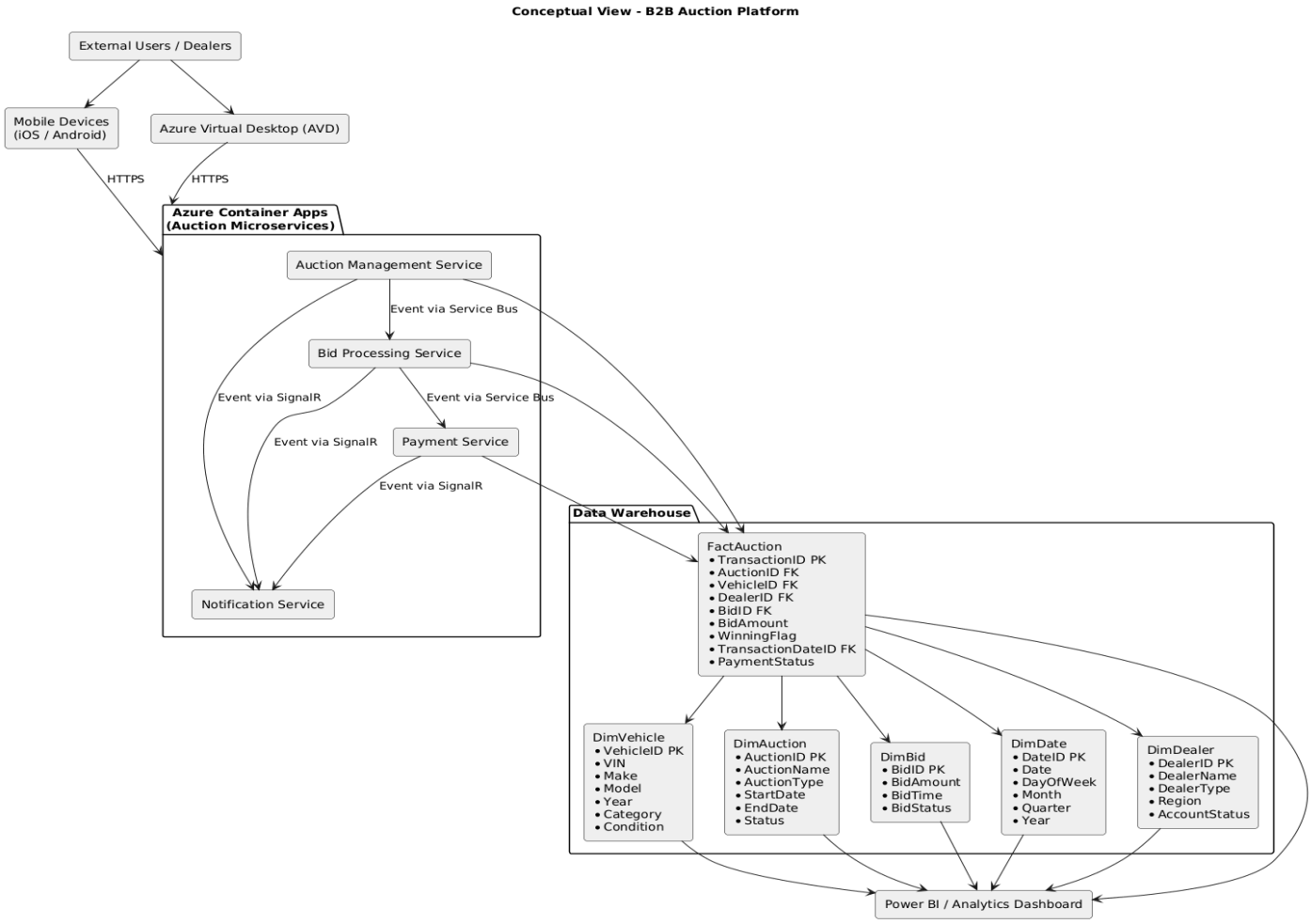
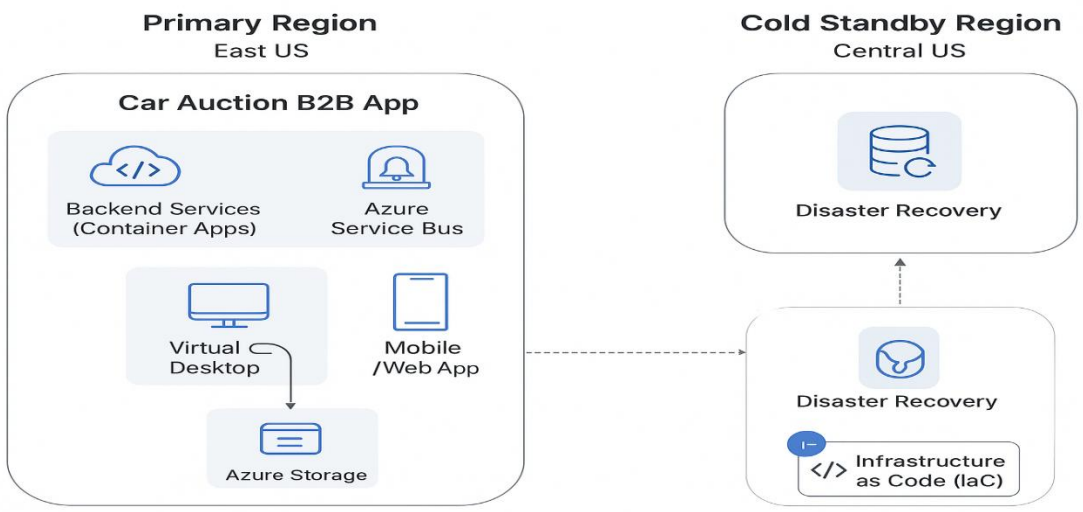


Diagram 11: Disaster Recovery solution



Architecture & Design Description

Data Integration Architecture between the B2B Auction Platform and GEM-DMS system (EAM & CRM SaaS) feeding the Solution Data Warehouse

This architecture defines the end-to-end data integration strategy used to consolidate transactional and master data originating from the B2B auction platform and the GEM-DMS SaaS ecosystem (EAM and CRM modules) into a centralized analytical Data Warehouse. The objective is to enable reliable, consistent, and scalable analytics while decoupling operational workloads from reporting and BI consumption.

The solution leverages a batch-oriented integration approach aligned with business volumes and cost constraints. Auction events (such as completed sales) are exposed through secure REST APIs and ingested into a staging area using Azure Functions. Azure Data Factory orchestrates batch pipelines to extract, transform, and synchronize data from the auction staging tables and GEM-DMS SaaS sources, ensuring conformed dimensions and referential integrity across domains. Curated fact and dimension tables are modeled using a star schema in Azure SQL Database, serving as the foundation for Power BI dashboards and KPIs.

This design ensures:

- Clear separation between operational systems and analytical workloads
- Controlled data quality, lineage, and transformation logic

This design ensures:

- Clear separation between operational systems and analytical workloads
- Controlled data quality, lineage, and transformation logic
- Cost-efficient analytics without introducing unnecessary streaming complexity

At a logical level, the data integration flow follows a hub-and-spoke analytical pattern centered on the Data Warehouse. The auction platform acts as the primary source of transactional facts (sales, bids, auction results), while GEM-DMS provides authoritative master and reference data related to dealers, assets, customers, and operational entities.

Key integration mechanisms include:

- **Azure Functions** for near–real-time ingestion of auction sales into staging tables
- **Azure Data Factory** for scheduled batch synchronization of CRM and EAM SaaS data
- **Transformation and enrichment pipelines** to populate conformed dimension tables and analytical fact tables
- **Azure SQL Data Warehouse (star schema)** as the single source of truth for BI and reporting

This architecture enables Power BI to consume curated, analytics-ready datasets with predictable refresh cycles and strong governance, while preserving the autonomy and performance of upstream operational systems.

Azure SQL Database

├ staging schema

│ ├ staging_sales

│ ├ staging_orders

│ ├ staging_customers

│ └- staging_assets

├ dim_* tables

└ fact_* tables

How data flows

Sales (from auction platform)

- **Azure Container Apps** → REST API
- **Azure Function:**
    - Validates
    - Normalizes
    - Writes to staging tables in Azure SQL DB

CRM / EAM (SaaS systems)

- **Azure Data Factory:**
    - Extracts data from SaaS connectors or REST APIs
    - Applies light transformations (mapping, cleansing)
    - Loads into staging tables (Azure SQL DB)

Then:

- **ADF (or SQL stored procedures):**
    - Populates dimension tables
    - Handles SCD (Slowly Changning Dimension) logic if needed

Does ADF load CRM / EAM data directly into dimension tables? Yes — but via staging, never directly from SaaS → dimension tables

The correct pattern is

CRM / EAM SaaS

↓

ADF (extract + transform)

↓

Azure SQL DB – staging tables

↓

ADF / SQL logic

↓

Datawarehouse Dimension tables

Covered Data Integration with the Solution Data Warehouse Architectural Views (analytic solution)

View	**Answers the question**	**Audience**
Executive View	*Why are we doing this? What value does it deliver?*	Executives, directors
Conceptual View	*What systems and data domains exist to deliver that value & achieve the business goals?*	Business + IT
Logical View	*How does data flow and get transformed to meet the functional goals?*	Architects
Physical View	*Where is this deployed technically? Where and with what services is this implemented?*	Cloud / platform architects

I. Executive Architecture View (Business-Aligned System Overview)

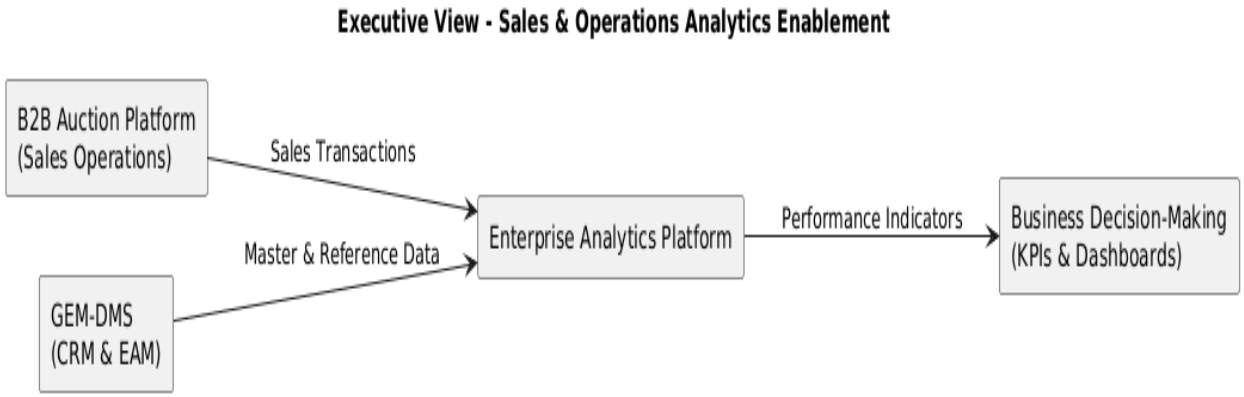
Purpose

- Show value, not technology
- No cloud provider details
- No services names (ADF, Functions, etc.)

What to show

✓ Business systems
✓ Data domains
✓ Business outcomes (KPIs, reporting)

Executive Architecture View/Diagram



Executive View illustrating how operational systems contribute to enterprise analytics and decision-making (covering business aligned system overview).

II. Conceptual Architecture View (Business Systems & Data Domains)

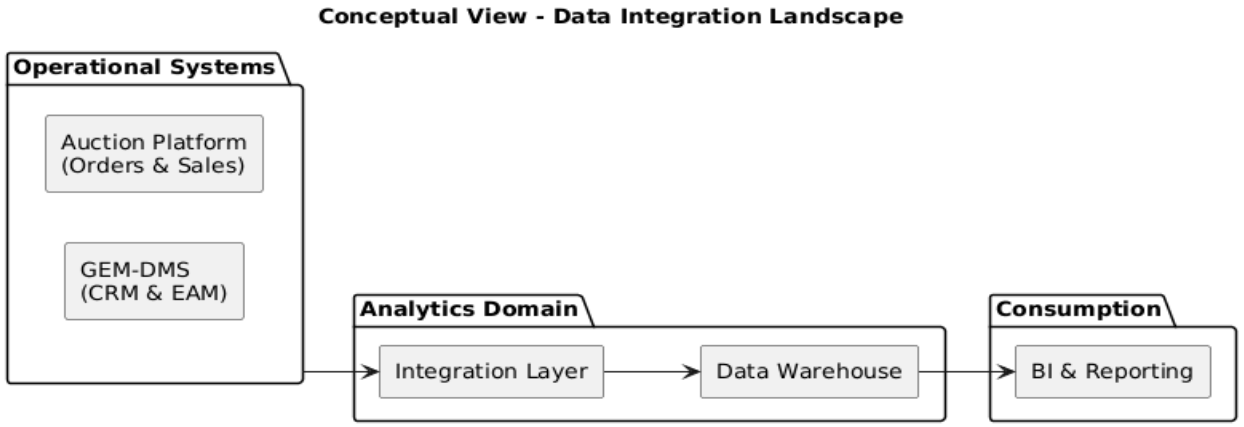
Purpose

- Define **system boundaries**
- Show **who owns what data**
- Still tech-agnostic

What to show

✓ Systems of record
✓ Integration boundaries
✓ Analytics platform as a concept

Conceptual Architecture View/Diagram



Conceptual View highlighting system responsibilities and data domains (covering business specifications).

Architecture & Design Description

III. Logical Architecture Views (Our Core Architecture Diagram)

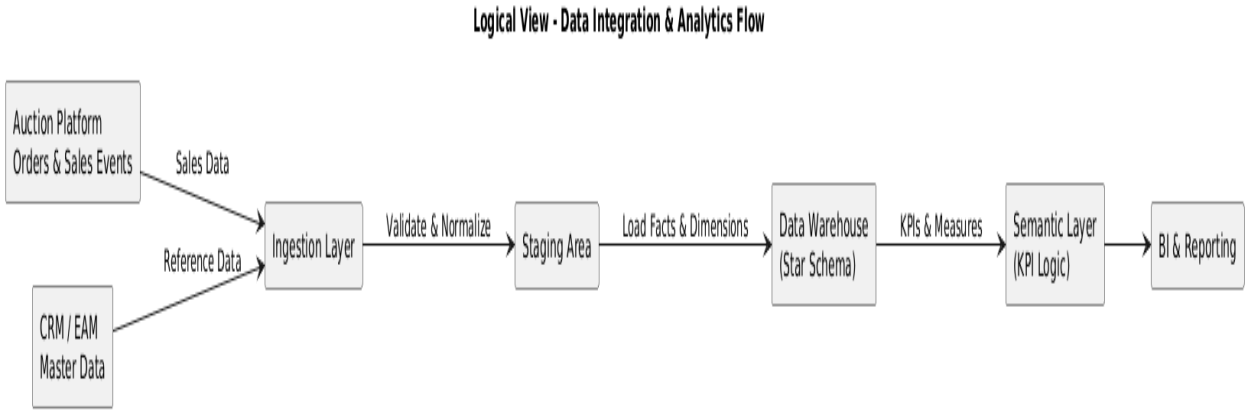
This is **the most important diagram** technically.

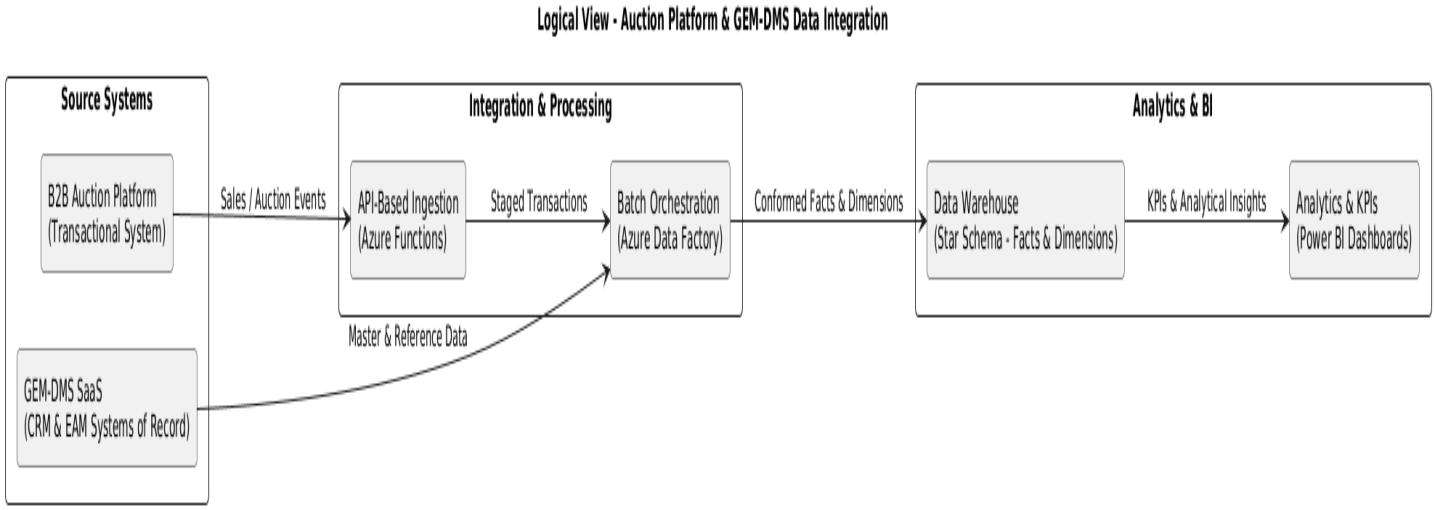
Purpose

- Explain **data flows**
- Show **ingestion patterns**
- Show **batch vs near-real-time**

What to show

✓ Azure services
✓ Integration patterns
✓ Star schema





Logical View describing data ingestion, transformation, and analytical modeling flows (covering functional specifications). Within it we find the following component: Azure Functions, ADF, Data Warehouse & Power BI

Architecture & Design Description

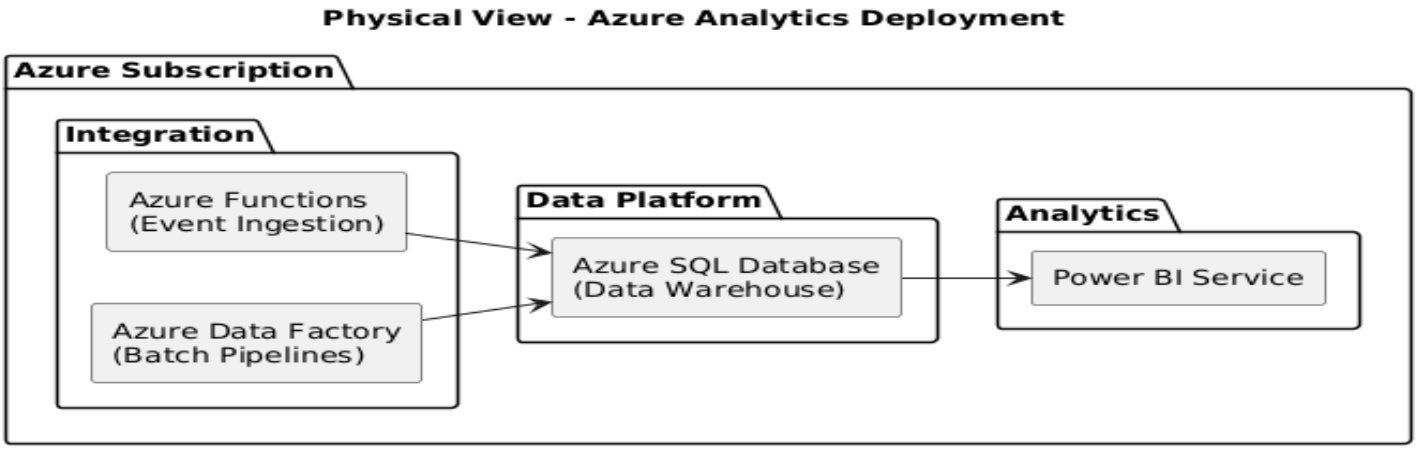
V. Physical Architecture View (Deployment & Runtime)

Purpose

- Prove **cloud competence**
- Show **where things run**
- No business logic explanation

What to show

✓ **Azure services**
✓ **Runtime boundaries**
✓ **Storage types**



Physical View showing deployment and runtime components within Azure.

Data Integration Architecture Overview: covered within the below architecture views

├ Executive View – Business Value & KPIs

├ Conceptual View – Systems & Data Domains

├ Logical View – Data Integration & Modeling

└ Physical View – Azure Deployment

High level DevOps Platform Architecture

DevOps Platform Architecture covers

- Repository strategy and governance (GitHub)
- Infrastructure as Code framework (Terraform + Ansible)
- Azure cloud infrastructure architecture
- CI/CD pipeline design and automation
- Environment provisioning (dev, test, prod)
- Deployment and release automation

High-Level Repository Architecture

Azure B2B Auction Platform
|
├── platform-infrastructure (Terraform)
├── platform-configuration (Ansible)
├── auction-microservices (Apps)
├── data-analytics (DW + ADF + BI)
└── platform-pipelines (CI/CD)

Repository Platform-Infrastructure — Terraform (Infrastructure)

platform-infrastructure/
├── environments/
| ├── dev/
| | └── main.tf
| ├── qa/
| ├── uat/
| └── prod/
|
├── modules/
| ├── networking/
| ├── avd/
| ├── container_apps/
| ├── sql_database/
| ├── service_bus/
| ├── data_factory/
| ├── monitoring/
| └── identity/
|
├── global/
| ├── naming.tf
| └── tags.tf
└── outputs.tf

Architecture & Design Description

What lives here in Platform-Infrastructure Repository (Terraform)

- VNet
- Subnets
- AVD Host Pools
- Container Apps Environment
- Azure SQL Database
- Azure Service Bus
- Data Factory
- Function Apps
- Key Vault
- RBAC
- Monitoring

Repository Platform Configuration – Ansible

platform-configuration/

```
|
├── inventories/
|   ├── dev/
|   ├── qa/
|   ├── uat/
|   └── prod/
├── playbooks/
|   ├── avd_config.yml
|   ├── gem_dms_integration.yml
|   └── security_hardening.yml
├── roles/
|   ├── avd_session_host/
|   └── cis_security_baseline/
|
└── group_vars/
```

: location where we store YAML files containing variables applicable to entire groups of hosts defined in our inventory

Repository Microservices

auction-microservices/

```
|
├── bidding-api/
├── dealer-api/
├── payment-api/
├── catalog-api/
├── audit-api/
├── notification-service/
├── docker/
└── aca_configs/
```

Contains: REST APIs, Dockerfiles & Unit tests

Architecture & Design Description

Repository Data & Analytics

data-analytics/

```
|
├── sql/
|   ├── staging/
|   ├── dimensions/
|   ├── facts/
|   └── indexes/
├── adf/
|   ├── pipelines/
|   └── datasets/
└── powerbi/
```

Repository CI/CD Pipelines

platform-pipelines/

```
|
├── terraform-deploy.yml
├── ansible-configure.yml
├── app-release.yml
└── data-release.yml
```

Cloud CI/CD Deployment Model

```
Terraform
   |
   ▼
Azure Infrastructure
   |
   ├── AVD VMs → Ansible
   ├── Container Apps → CI/CD
   ├── Functions → CI/CD
   ├── Data Factory → Git publish
   └── SQL → migration pipeline
```

Terraform handles:

- Networking
- AVD infra
- Container Apps env
- Azure Service Bus
- SQL DB
- Storage account (blob container)
- ADF
- Functions infra
- Identities
- Policies

Azure Native handles:

- API deployment
- Functions deployment
- Data pipelines release
- Container revisions
- Secrets injection

Ansible handles ONLY:

1. AVD session hosts configuration (Provisioning, configuring NICs, subnetting, NSG, storage, Automate Entra ID Joined Session Hosts, register new session hosts with existing host pools using registration tokens) & installing desktop auction client (using MSIX app attach which is a read-only virtual hard disk. vhdx or .cim at runtime)
2. GEM-DMS integrations
3. OS hardening

1. Terraform Layer — Infrastructure Provisioning

Resource organization

- Resource Groups
- Naming conventions
- Tags (cost center, environment)

Networking

- Virtual Network
- Subnets
- NSG rules
- Private endpoints
- Private DNS zones

Identity & security

- Managed identities
- Role assignments
- Key Vault
- Access policies

Platform services

- Azure Container Apps Environment
- Azure Function App
- Azure SQL Database
- Azure Service Bus
- Azure Data Factory instance
- Azure Virtual Desktop host pool

Observability

- Log Analytics workspace
- Diagnostic settings
- Alerts

Storage

- Azure Storage Account

- Blob containers (staging)

2. AVD VMs — Configuration with Ansible

Once **session hosts** are created by Terraform, ansible configures it & install the client auction application.

VM bootstrap

- install required Windows features
- install PowerShell modules
- configure WinRM (Microsoft's secure, SOAP-based protocol for remotely managing Windows servers and device)

AVD session host configuration

- Entra ID join
- register VM into host pool
- configure session limits

Auction platform client installation

- install desktop bidding software (MSIX app attach)
- configure endpoints to backend APIs
- configure authentication

System configuration

- registry tuning
- certificate installation
- firewall configuration

Monitoring agents

- Azure Monitor agent
- log collection agents

Security hardening

- CIS baseline
- patch baseline
- antivirus configuration

3. Container Apps — Deployment via CI/CD

Terraform creates:

- Container Apps Environment
- Container Apps resources
- connection to ACR

CI/CD handles **application lifecycle**.

Build pipeline

- pull source code
- run unit tests
- build Docker image
- security scan

Image management

- tag version
- push image to Azure Container Registry

Deployment pipeline

- update Container App revision
- configure environment variables
- configure secrets from Key Vault

Scaling configuration

- set autoscale rules
- configure KEDA triggers
- configure min/max replicas

Observability

- configure logging
- configure health probes

4. Azure Functions — Deployment via CI/CD

Terraform provisions:

- Function App
- Storage account
- managed identity

CI/CD pipeline handles code.

Build phase

- compile function code
- run tests
- package artifact (zip)

Deployment phase

- deploy zip package
- configure runtime settings
- configure environment variables

Integration configuration

- connect to auction API
- connect to SQL staging table
- configure Key Vault references

Performance configuration

- configure scaling limits
- configure timeout settings

5. Azure Data Factory — Git Publish

ADF has a **native Git integration model**.

Development happens in Git.

Development phase

- design pipelines
- design datasets
- define linked services
- define triggers

Git repository

ADF stores:

- pipeline JSON definitions
- dataset definitions
- data flow definitions

Publish step

Publishing performs:

- ARM template generation
- deployment to ADF runtime

CI/CD pipeline can automate:

- validation
- publishing
- environment parameter substitution

Example tasks:

- deploy staging pipelines
- deploy production pipelines
- configure triggers

6. SQL Database — Migration Pipeline

Terraform creates the **database container**:

- Azure SQL Database
- firewall rules
- identity

But **Terraform should NOT manage schema**.

Database deployment pipeline: SqlPackage / DACPAC

A migration pipeline deploys:

- schemas
- tables
- views
- stored procedures
- indexes

Used with **SSDT / Visual Studio SQL projects**.

Pipeline tasks:

- Build SQL project
- Generate DACPAC (Data-tier Application Package): self-contained, compressed file format (.dacpac extension) that contains the complete **schema** and metadata of a SQL Server database, but generally **excludes user data** by default. It is primarily used for developing, building, version-controlling, and deploying database schemas across different environments
- Deploy with SqlPackage: command-line utility used to create, deploy, and manage DACPAC files

What gets deployed:

- dimension tables
- fact tables
- staging tables
- indexes
- constraints

Azure Automation Decision Matrix

Task	**Tool**
Azure resources	Terraform
Containers deployment	CI/CD (GitHub Actions)
Functions deployment	CI/CD (GitHub Actions)
Data Factory pipelines	Native ADF CI/CD
SQL schema	DB migration tools
Secrets	Azure Key Vault & Azure App configuration service
Monitoring agents	Azure Policy automates the at-scale deployment, configuration, and compliance management of the Azure Monitor Agent (AMA) extensions on Azure Virtual Desktop (AVD) VMs.
VM software install	Ansible
AVD configuration	Ansible

2. Hotel Digital Access Platform on Amazon

Architecture & Design Description

- Responsible for **architecture plans** for the development and production teams supporting the **LGS (Lodging Global Solution) – … Server**, a multi-tenant **hotel digital access solution** issuing **physical and digital keys**.
- Designed a **cloud-native, serverless architecture on AWS**, targeting **250 Hyatt sites initially**, with scalability to support **3,000–5,000 sites over five years**.
- **Backend architecture**:
    - **Microservices** deployed as **ECS Fargate tasks**, fronted by the **custom Ambiance API Gateway**.
    - **Application Load Balancer (ALB)** routes requests from web apps and external services to the API Gateway.
    - Microservices communicate primarily via **RabbitMQ** for event-driven asynchronous messaging.
    - **Aurora Serverless (PostgreSQL)** provides the database layer with automatic scaling and multi-AZ redundancy.
- **Digital mobile access integration**:
    - **Android devices** use **Bluetooth** to unlock hotel doors, with Ambiance Server communicating securely with **External SaaS** to provision mobile credentials.
    - **Apple devices** (iPhone) use **NFC** for door access. Ambiance Server integrates **Apple APIs** and communicates with **External SaaS** for credential management while also interacting with the **hotel PMS system** to validate reservations and trigger mobile access.
    - On-site **Ambiance client app** communicates with door encoders via **WebSocket over the hotel private network** for physical key issuance.
- **Web access:**
    - Ambiance web app calls the API Gateway via HTTPS for administrative and operational functions.
- **Scalability & fault tolerance:**
    - Multi-AZ ECS Fargate deployment ensures high availability and automatic scaling for cloud microservices.
    - Aurora Serverless database scales elastically with workload.
- **Security & network:**
    - Private hotel IP network for sensitive on-site communications.
    - HTTPS for all external interactions and secure API calls to external SaaS for digital key delivery.
    - Role-based access control and least-privilege principles applied to microservices and database access.

Key architectural patterns applied:

- Cloud-native, serverless microservices
- Event-driven architecture via RabbitMQ (Amazon MQ)
- Multi-tenant SaaS solution
- Digital mobile access integration (Bluetooth for Android, NFC for Apple) via External SaaS
- WebSocket integration for real-time physical key encoding
- Scalable compute ECS/Fargate & database (Aurora) serverless solutions
- High availability & multi-AZ deployment for production-grade fault tolerance

Scalability & Cost-Efficiency Rationale – ECS/Fargate & Aurora

The platform was designed as a **multi-tenant, cloud-native solution** to support an initial onboarding of approximately **250 hotel sites (ABC_FAKE_NAME) in year one**, each averaging **~250 rooms**, while scaling to **3,000–5,000 customer sites over a 5-year horizon**, all with similar room capacities.

Given the **low annual revenue per customer site (~/year)**, cost control and operational efficiency were critical to ensure long-term profitability while maintaining high availability and performance.

To address these constraints, **Amazon ECS with AWS Fargate (serverless containers)** was selected to:

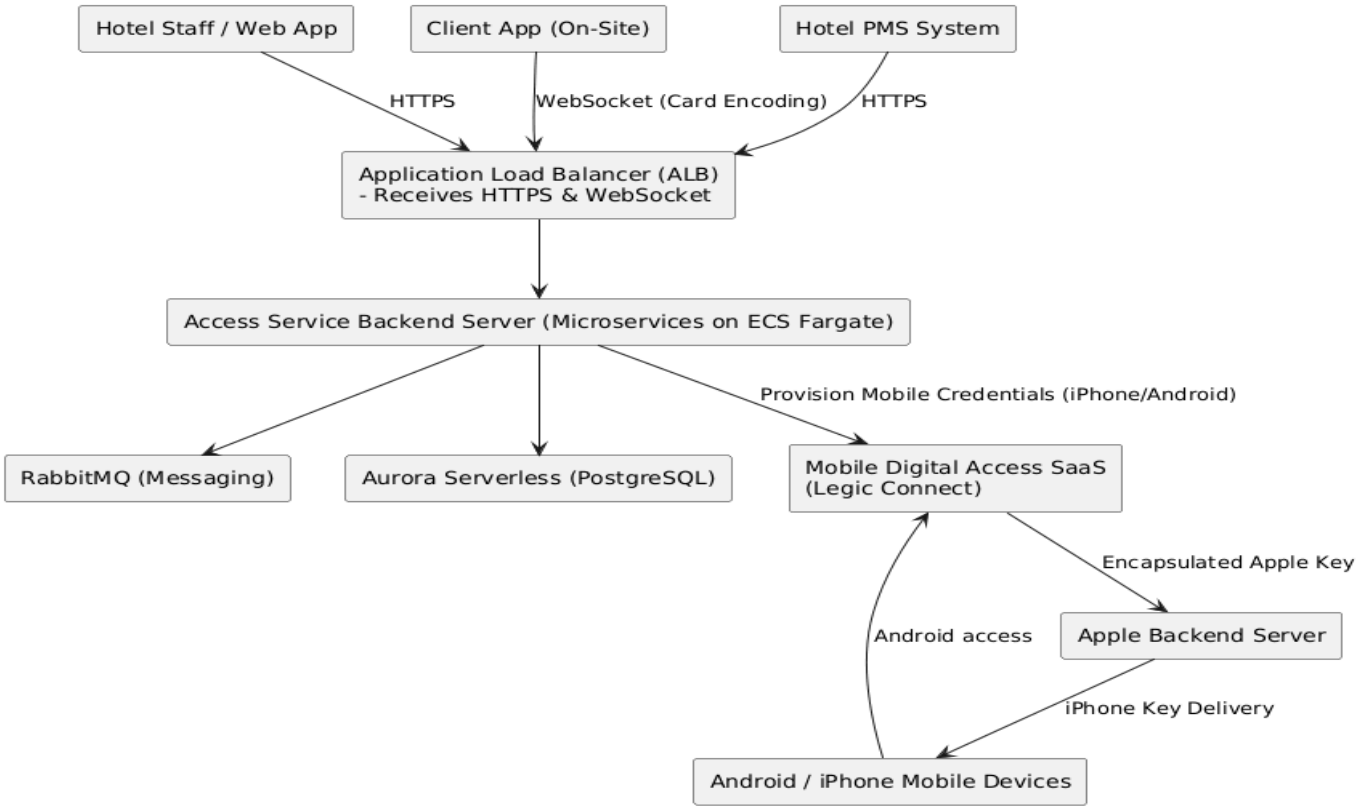
- **Eliminate fixed infrastructure costs** by avoiding always-on VM capacity.
- **Dynamically scale compute resources** based on real traffic and workload demand.
- **Support granular, per-microservice scaling**, allowing independent ECS task scaling per service (e.g. encoder service scaling higher than mobile delivery or digital key generation services).
- **Handle peak and burst workloads efficiently**, particularly during high-concurrency events such as mass check-ins, mobile key provisioning, or bulk card encoding operations.
- **Simplify operations and reduce DevOps overhead**, improving cost-to-serve per customer site.

On the data layer, **Amazon Aurora (PostgreSQL-compatible)** was chosen to complement the serverless compute model by providing:

- **Elastic database scaling** aligned with application load.
- **High availability and fault tolerance** for critical access-control data.
- **Cost-efficient performance at scale**, suitable for multi-tenant transactional workloads.

This **fully serverless architecture (ECS/Fargate + Aurora)** enabled the platform to **scale precisely where and when needed**, optimize infrastructure spend, and remain economically viable while supporting thousands of hotel sites and millions of digital access operations over time.

High level Architecture Diagram



Tree-tier network:

- Public subnet: web layer with NAT Gateway
- Private subnet: service/application layer (containers/serverless)
- Private subnet: data layer (database/PaaS)

Security layers:

- Firewalls at subnet and instance level (AWS Security Groups + Network ACLs / Azure NSG + ASG)

High availability:

- Multiple Availability Zones implied for subnets
- ALB handles load balancing

API Gateway:

- Manages API-specific tasks before routing to services

Identity & DNS:

- IAM / Azure Entra ID for auth/authz
- DNS resolves requests to appropriate backend

Architecture & Design Description

Architecture & Integration — SIMAT Automated Customer Lifecycle Platform

Client: | Hospitality Digital Access Solutions

Context & Objective

Designed and implemented an **automated customer account lifecycle management platform (SIMAT)** to orchestrate **on-prem and cloud Ambiance solutions**, integrate **ERP, billing, SaaS security platforms**, and enforce **end-to-end cryptographic trust** for hotel access control systems.

SIMAT acts as a **central cloud-hosted orchestration layer** automating customer provisioning, mobile key enablement, cryptographic key management, operational support, and usage-based billing across a multi-tenant hospitality ecosystem.

Scope of Architecture & Integration Resp

- Designed **SIMAT cloud architecture** as a centralized automation platform with three core services:
  - Customer Account Service
  - Support Service
  - Finance Service
- **Integrated SIMAT with:**
  - **Microsoft Dynamics AX (ERP)** as system of record for customer accounts
  - **LEGIC Connect SaaS** for cryptographic site key issuance and mobile key delivery
  - **Ambiance Access Platform** (on-prem & AWS cloud)
  - **Zuora SaaS** for subscription and usage-based billing
  - **Apple Wallet (iOS**) and **Android Mobile** Key ecosystems
- Designed a **secure cryptographic trust model** aligned with LEGIC hardware constraints (lock chips).
- Automated **multi-tenant cloud provisioning** for Ambiance (Aurora DB tenant creation).
- Enabled **mobile key lifecycle tracking and exception handling.**

Cryptographic & Security Design (Key Aspect)

- **Hotel Site Private Key**
  - Issued by LEGIC per hotel site (ProjectId)
  - Required to establish secure communication with LEGIC lock chips
- **Application Key**
  - Unique per hotel brand / branch
  - Used to encrypt the file containing customer door access keys
- **Customer Door Access Key**
  - Generated by Ambiance
  - Encrypted using the Application Key
  - Decrypted only after successful authentication using the Site Private Key

This design ensures hardware-backed security, controlled key distribution, and strict tenant isolation.

I. EXECUTIVE VIEW

Why are we doing this? What value does it deliver?

Purpose

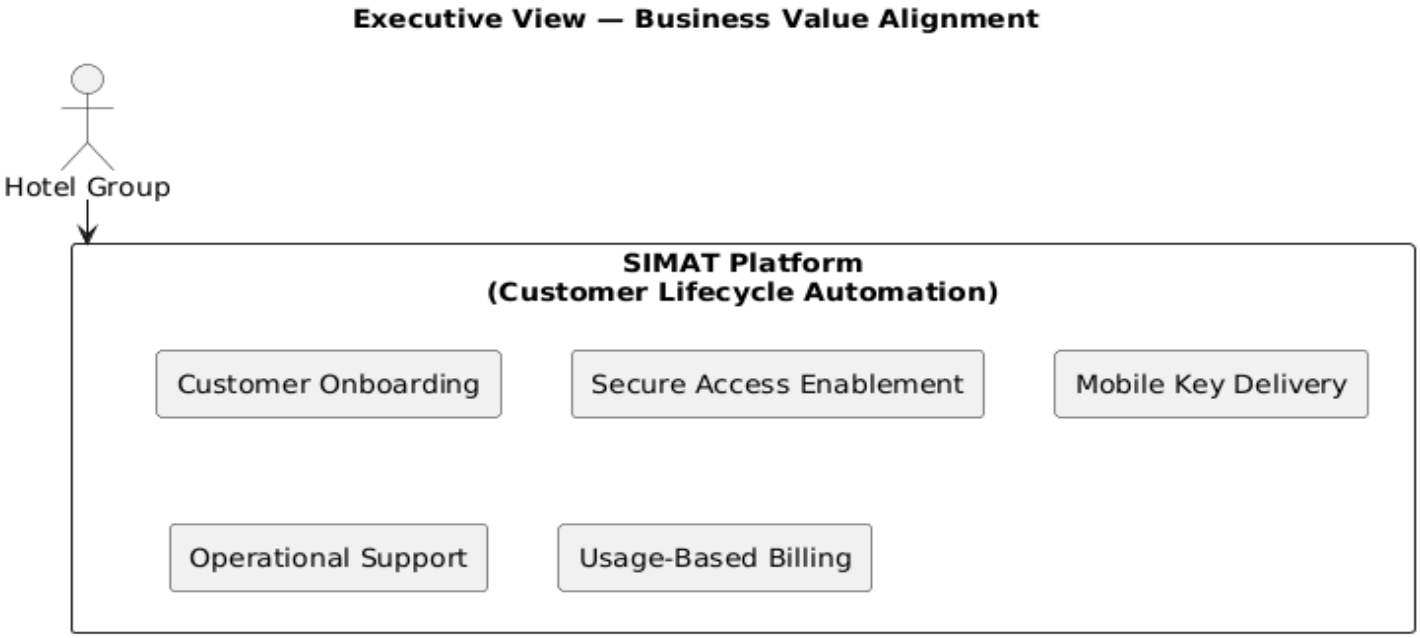
Deliver a **single automated platform** that orchestrates the **end-to-end customer lifecycle** for hospitality access solutions—covering **account provisioning, cryptographic trust, mobile key delivery, operational support, and usage-based billing**, across **on-prem** and **cloud deployments.**

Business Value

- Eliminates manual cross-system provisioning
- Enforces hardware-backed cryptographic security
- Enables scalable multi-tenant cloud onboarding
- Supports monetization through usage-based billing
- Improves customer experience through reliable mobile key delivery

Executive Architecture Views/Diagrams: Business Value Alignment plus SIMAT Automated Customer Lifecycle Platform Architecture Views

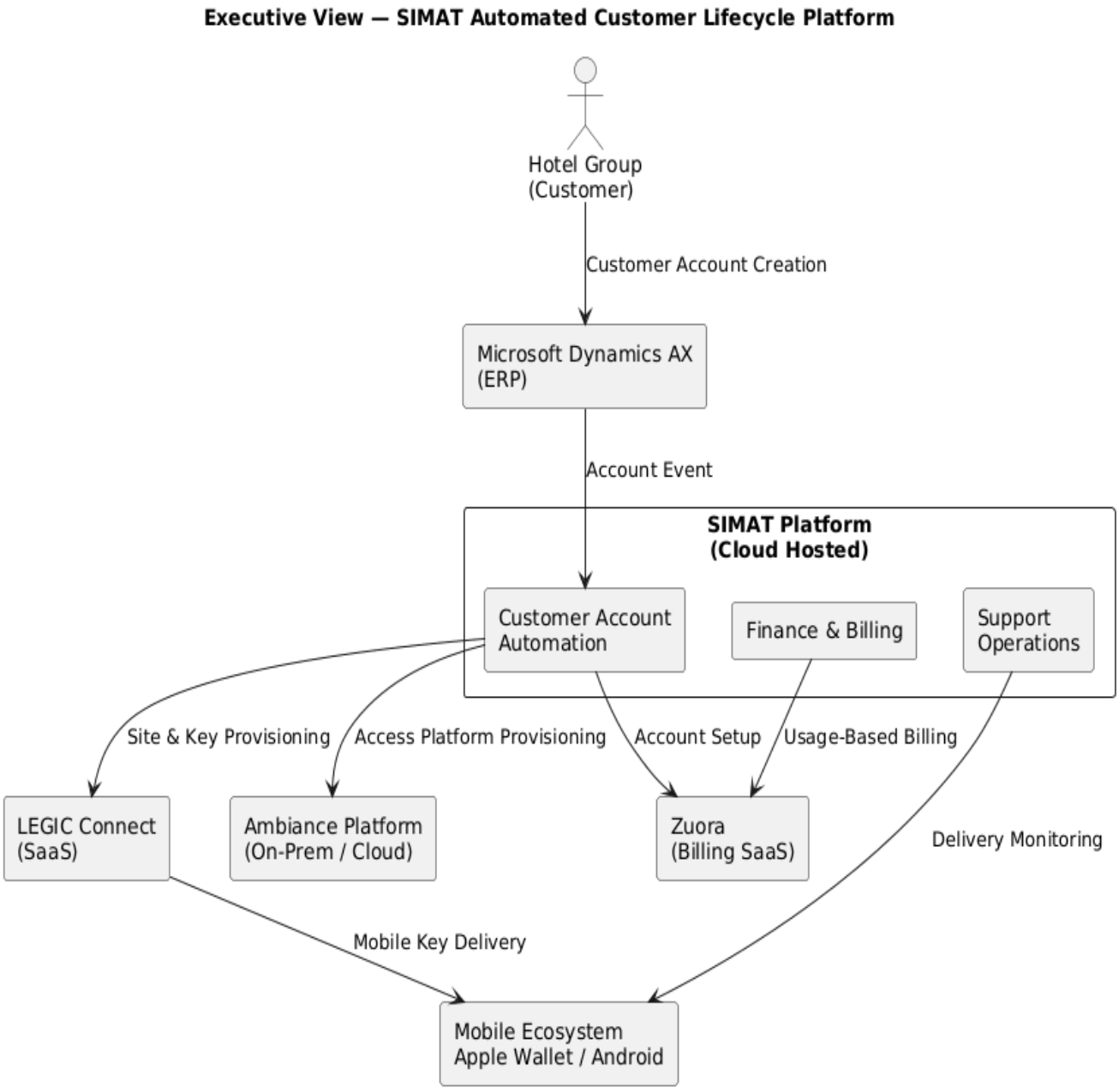
- **Diagram 1: Business Value Alignment**



How to read this view

- No systems, no technologies
- Only business capabilities and outcomes
- Answers *why SIMAT exists*, not *how it works*

- **Diagram 2: SIMAT Automated Customer Lifecycle Platform**



This view focuses on business outcomes and value, explaining why SIMAT exists and what it delivers to hotel groups. It omits technical details, systems, or data flows, ensuring clarity for stakeholders interested only in business impact.

II. CONCEPTUAL VIEW

What systems and data domains exist to deliver that value?

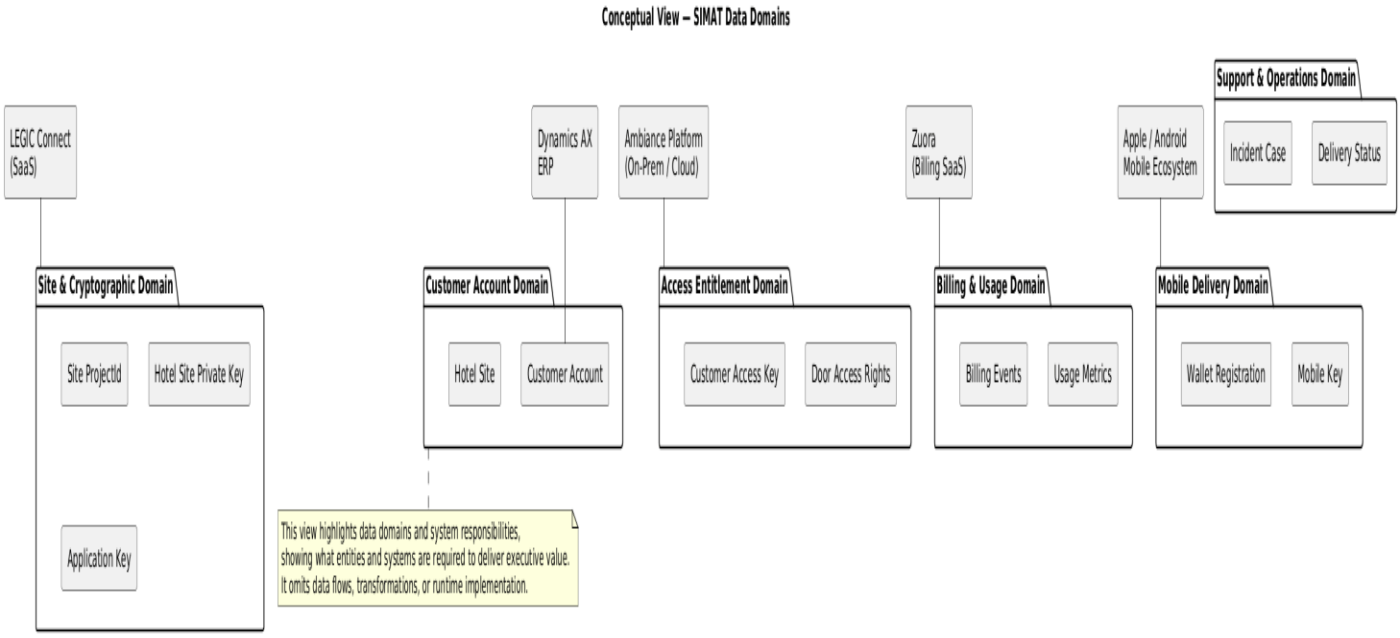
Purpose: This view introduces data domains first, with systems shown only as domain owners or participants. There is no data flow yet — only responsibility boundaries. A complementary conceptual view would be provided to illustrate the end-to-end automation flow

SIMAT Core Data Domains

Data Domain	Responsibility
Customer Account Domain	Legal customer identity, contracts, hotel sites
Site & Cryptographic Domain	Site identity, private keys, application keys
Access Entitlement Domain	Door access rights and credentials
Mobile Delivery Domain	Mobile key provisioning & delivery
Billing & Usage Domain	Usage metrics and billable events
Support & Operations Domain	Delivery tracking, incidents, retries

Conceptual Architecture Views/Diagrams: Systems & Data Domains plus SIMAT End to End Automation Architecture Views

- **Diagram1: Systems & Data Domains** shows SIMAT data domain owners or participants



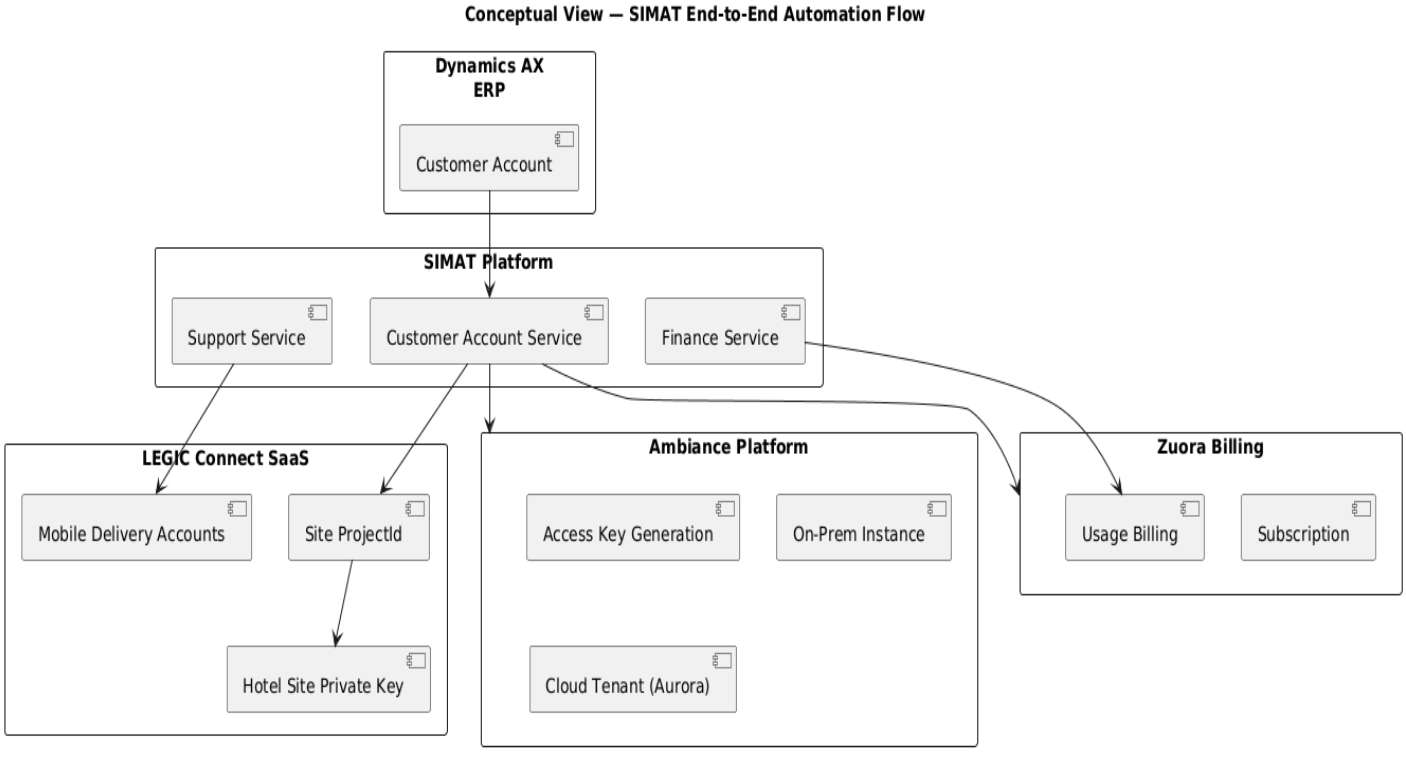
The solution is structured around **clearly defined data domains**, each owned or serviced by specialized systems, ensuring separation of concerns and long-term evolvability.

Core Data Domains

- **Customer Account Domain**
  Manages legal customer identity, contracts, and hotel site definitions.
  System of record: Microsoft Dynamics AX (ERP).
- **Site & Cryptographic Domain**
  Governs hotel site identity, cryptographic material, and trust establishment with physical locks.
  Owned by LEGIC Connect SaaS.
- **Access Entitlement Domain**
  Defines door access rights and customer credentials generated by the access control platform.
  Owned by Ambiance (on-prem and cloud).
- **Mobile Delivery Domain**
  Manages mobile key provisioning and delivery through Apple Wallet and Android ecosystems.
  Enabled via LEGIC mobile delivery services.
- **Billing & Usage Domain**
  Captures usage metrics and produces billable events for subscription and consumption-based charging.
  Implemented through Zuora SaaS.
- **Support & Operations Domain**
  Tracks mobile key delivery status, incidents, and operational follow-up.

At this level, **systems are positioned as domain participants**, not drivers of the architecture

- **Diagram2: Automation end-to-end flow**



This view highlights **data domains and system responsibilities**, showing what entities and systems are required to deliver the executive value. It **does not show data flows, transformations, or runtime implementation**, maintaining a high-level perspective suitable for architecture discussions and planning.

III. LOGICAL VIEW

How does data flow and get transformed to meet the business goals?

Purpose:

This view introduces:

- Bounded contexts (services)
- Integration flows
- Data transformations
- Security dependencies

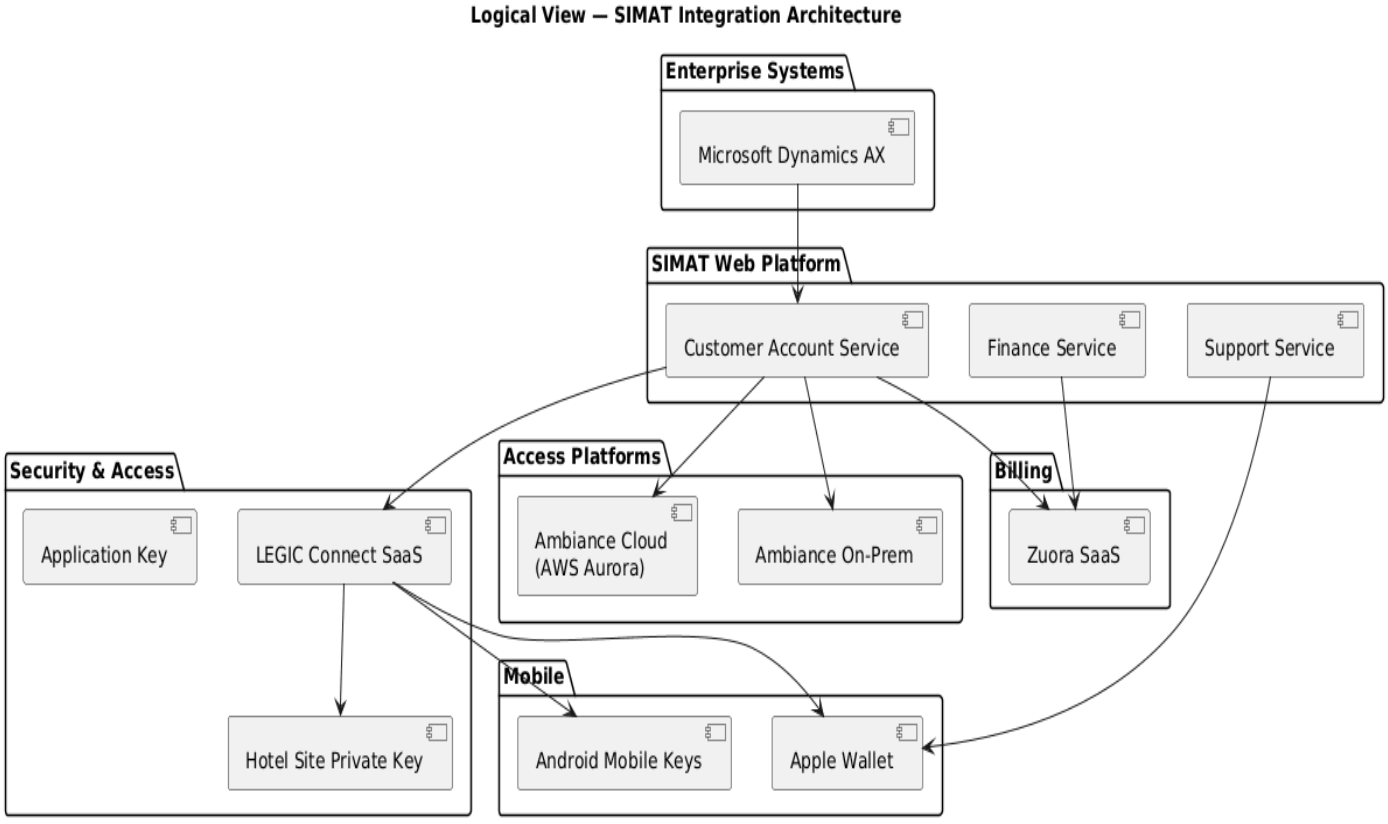
This is where automation actually becomes visible.

Logical Bounded Contexts (SIMAT)

- **Customer Account Service**
  - Orchestrates account provisioning
  - Owns customer & site lifecycle
- **Support Service**
  - Tracks mobile key delivery
  - Manages retries and incidents
- **Finance Service**
  - Aggregates usage
  - Triggers billing events

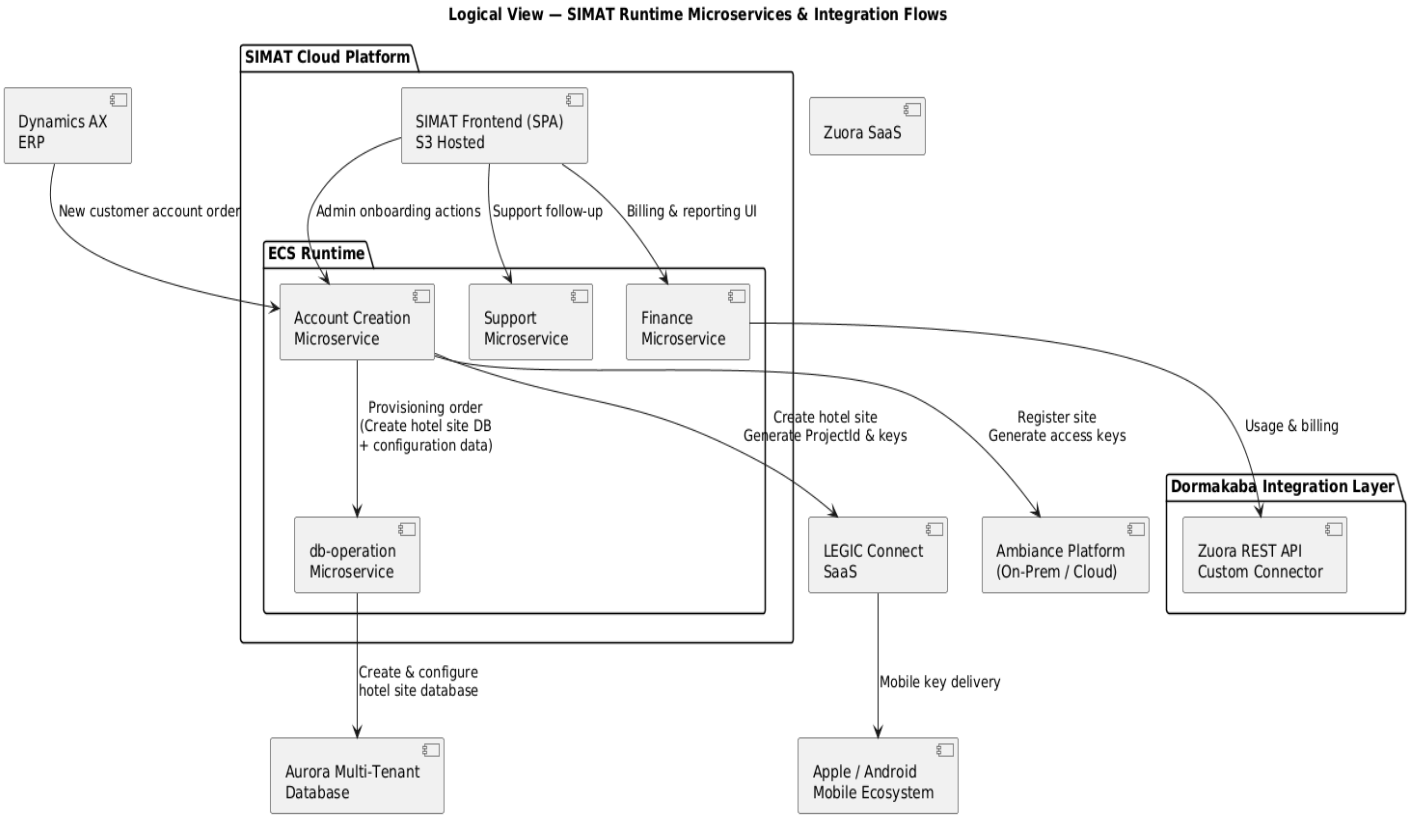
Logical High level Architecture Views/Diagrams: SIMAT Integration Architecture & SIMAT Runtime Microservices Flows Architecture Views

- **Diagram1: SIMAT Integration Architecture (Bounded Context)**



Architecture & Design Description

- **Diagram2: SIMAT Runtime Microservices Flows**



How to read this Logical View (important for senior reviewers)

- ECS and S3 are shown as logical runtime containers, not infrastructure
- db-operation is a distinct bounded context, not a shared utility
- Database access is explicitly mediated, not implicit
- This view explains:
    - *how automation is executed*
    - *how data is created, transformed, and persisted*
    - *how external systems are orchestrated*

The db-operation microservice, deployed as a container within an ECS task, was responsible for creating and configuring the hotel site database in the cloud.
This operation was triggered by the SIMAT Customer Account Service as part of a new customer account creation order, ensuring controlled, secure, and automated multi-tenant database provisioning for the Ambiance cloud platform.

This view illustrates **how SIMAT microservices and technical components interact**, including bounded contexts, integration flows, and runtime responsibilities. It shows **data transformations, orchestration, and provisioning flows**, but deliberately **omits cloud deployment details** such as VPCs, subnets, and scaling policies, which are covered in the Physical view.

Cryptographic Trust Chain (Logical Explanation)

1. **LEGIC** issues a **ProjectId**, encrypted into a **Hotel Site Private Key** (unique per hotel site)
2. Site Private Key authenticates communication with LEGIC lock chips
3. **Application Key** (per hotel brand/branch) encrypts the file holding customer door access keys (issued by Ambiance solution)
4. **Customer Door Access Key** is decrypted only after successful site authentication
5. Access is granted via **mobile or plastic credential**

Logical Runtime Decomposition (SIMAT): SIMAT Runtime Components

- **SIMAT Frontend**
  - JavaScript SPA
  - Stored in **Amazon S3**
- **SIMAT Backend APIs**
  - Deployed as **microservices in ECS Tasks**
  - Own orchestration and business logic
- **db-operation Microservice**
  - Dedicated ECS-deployed container
  - Responsible for **controlled database provisioning and operations**
- **Used for:**
  - Cloud tenant creation (Aurora)
  - Schema initialization
  - Secure DB lifecycle actions

Logical Runtime Responsibilities

- **SIMAT Customer Account Service**
  - Owns customer onboarding intent
  - Issues **cloud provisioning orders**
- **db-operation Microservice**
  - Executes **hotel site database creation**
  - Applies **tenant-specific configuration data**
  - Manages schema initialization and secure DB access
- **Aurora Multi-Tenant Database**
  - Hosts per-hotel-site logical databases / schemas

It still **does not answer**:

- which AZs
- how scaling works
- network layout
- IAM policies (could be part of security design section)

Those belong **strictly to the Physical view**

Logical Architecture (How)

SIMAT is implemented as a **cloud-native, microservice-based orchestration layer**, with clear separation between **business intent**, **technical execution**, and **external integrations**.

SIMAT Microservices

- **Account Creation Microservice**
  Orchestrates customer onboarding and site provisioning across ERP, security, access control, and cloud platforms.
  Acts as the **single authority issuing provisioning orders**.

- **Support Microservice**
  Monitors mobile key delivery, manages exception handling, and supports operational follow-up.
- **Finance Microservice**
  Aggregates usage data and triggers billing events through enterprise billing integrations.

Technical Provisioning Service

- **db-operation Microservice**
  A dedicated, containerized service deployed on AWS ECS, responsible for **executing privileged database operations**.
  It creates and configures **hotel site databases in the cloud** (Aurora multi-tenant environment) based on **provisioning orders issued by the Account Creation microservice**.

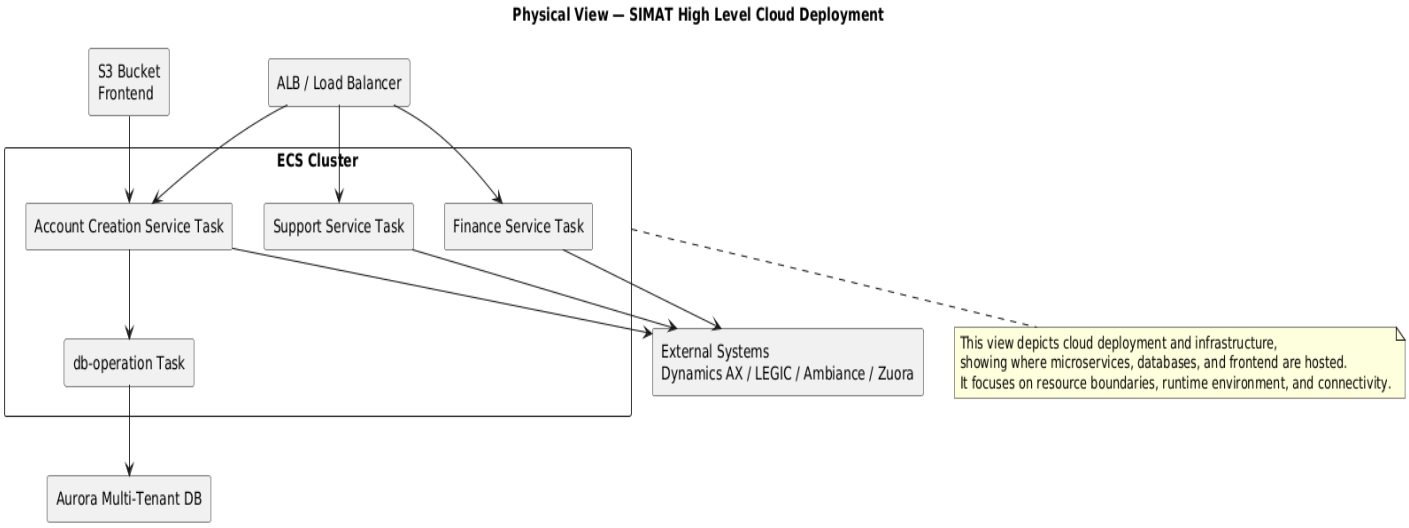
Integration & Flow Summary

1. A new customer account order is created in **Dynamics AX**.
2. The **Account Creation microservice** orchestrates:
   - Hotel site creation and cryptographic key issuance in **LEGIC Connect**
   - Access entitlement setup in **Ambiance**
   - Cloud tenant provisioning via **db-operation**
3. LEGIC delivers mobile keys to **Apple Wallet / Android** when enabled.
4. The **Support microservice** tracks delivery status and exceptions.
5. The **Finance microservice** sends usage and billing data to **Zuora** through **Dormakaba-built REST API connectors**.

IV. Physical View (Where & with what services)

Purpose: Deployment components (example Physical view for cloud)

- **Frontend:** S3 static hosting (JavaScript SPA)
- **Backend Microservices:** ECS tasks in ECS cluster (Account Creation, Support, Finance, db-operation)
- **Database:** Amazon Aurora multi-Tenant per hotel site
- **Integrations:** REST API calls to Zuora, LEGIC SaaS, Ambiance cloud services
- **Mobile Delivery:** Apple Wallet / Android via LEGIC mobile delivery



This view depicts the **actual cloud deployment and infrastructure**, including ECS tasks, Aurora multi-tenant databases, S3-hosted frontend, and integration endpoints. It emphasizes **where services are implemented**, resource boundaries, and runtime environment.

Architecture & Design Description

High level DevOps Platform Architecture

In this section we would only cover AWS infrastructure provisioning, configuration & deployment tasks. Implementing Infrastructure-as-Code using Terraform to provision AWS networking, ECS Fargate clusters, Amazon Aurora databases, and Amazon MQ messaging infrastructure for a multi-tenant hospitality digital access platform. Ansible automated post-provisioning configuration including Docker image preparation, RabbitMQ queue configuration, database schema initialization, and microservice deployment workflows across a highly available multi-AZ architecture. Application Lifecycle is handled by Jenkin Pipeline as CI/CD process.

I. What Terraform Would Provision (AWS)

Mapped directly to your architecture.

A. Networking Foundation

Terraform creates:

- VPC
- Public subnets (ALB)
- Private subnets (Fargate tasks)
- DB subnets
- NAT Gateway
- Route tables
- Security groups & Nacls

B. Load Balancing Layer

Terraform provisions:

- Application Load Balancer
- Target groups
- HTTPS listeners
- ACM certificates

Ensures hotel mobile apps reach APIs securely.

C. ECS Fargate Platform

Terraform creates:

- ECS Cluster
- Task Definitions (baseline)
- ECS Services

Architecture & Design Description

- Autoscaling policies

D. Container Registry

Terraform provisions:

- Amazon ECR repositories per microservice

E. Database Layer

Terraform deploys:

- Amazon Aurora cluster
- Multi-AZ configuration
- Parameter groups
- Backups
- Encryption

F. Messaging Layer

Terraform provisions:

- Amazon MQ (RabbitMQ broker)
- Network access rules
- Credentials (via Secrets Manager)

Critical for microservice communication.

G. **Storage layer**

- S3 bucket (object storage account)

H. Security & Identity

Terraform creates:

- IAM roles for tasks
- Execution roles
- Policies
- Secrets Manager
- KMS keys

I. Observability

Terraform deploys:

- CloudWatch log groups

Architecture & Design Description

- Alarms
- Metrics dashboards

II. What Ansible Would Do (AWS Project)

Now Ansible complements Terraform.

A. Docker Image Preparation

Before Cloud CI/CD maturity, Ansible often handled builds.

Ansible:

- builds Docker images
- injects configs
- tags versions
- pushes to ECR

B. Application Configuration

Ansible configures:

- environment variables
- service endpoints
- RabbitMQ connection settings

C. RabbitMQ Initialization (Amazon MQ)

Terraform creates broker. Ansible configures broker:

- exchanges
- queues
- routing keys
- permissions

D. Database Initialization (Aurora)

Terraform:

- creates database cluster.

Ansible:

- creates schemas
- applies indexes

Architecture & Design Description

- perform configurations

E. ECS Service Final Configuration

Ansible can:

- register updated task definitions
- update container environment variables
- trigger rolling deployments

F. Security Hardening

Ansible applies:

- container runtime configs

III. Deployment Flow

Phase 1 — Terraform

Creates:

- VPC
- ECS cluster
- Aurora
- MQ
- IAM
- ALB

Phase 2 — Ansible

Performs:

1. Build images
2. Push to ECR
3. Configure RabbitMQ
4. Initialize Aurora schema
5. Deploy ECS services

Responsibility Matrix (AWS Version)

Task	Terraform	Ansible
VPC/Subnets	✅	❌
ECS Cluster	✅	❌
Fargate services	✅	⚠️ **update**
ECR repos	✅	❌
Aurora cluster	✅	❌
DB schema	❌	✅
RabbitMQ broker	✅	❌
MQ queues/exchanges	❌	✅
Docker build	❌	✅
Monitoring infra	✅	❌

CI/CD Pipeline process:

Jenkins CI/CD Pipeline (build / test / package)
|
▼
Terraform Apply
|
▼
Provision Infrastructure
|
└─> Calls Ansible (via Terraform provisioners / external scripts)
|
▼
Configure middleware, services

3. Digital Learning Platform

Architecture & Design Description

- Designed a **digital learning platform** for field employees using **VR headsets and tablets**, integrated with **SAP ERP HCM** and **SuccessFactors** systems with potential future expansion to office employees.
- **Data storage & management**:
    - **Data Lake** implemented with **MinIO**, **Elasticsearch**, and **Delta Lake**.
    - Multi-zone storage for different stages: **staging → raw → validated → curated → archive**.
    - **ETL pipelines** orchestrated using **Apache Airflow** and **Apache NiFi**, with processing performed by **Kubernetes worker pods**.
- **Data Warehouse / Analytics**:
    - Star-schema Data Warehouse optimized in **Delta Lake (Parquet on MinIO)**.
    - **Apache Spark** used to transform and aggregate data, enabling **real-time actionable insights** in **Power BI** (both Import and DirectQuery modes).
- **Machine Learning pipelines**:
    - Spark triggers **containerized ML microservices** for:
        - Content classification
        - NLP-based auto-tagging
        - Recommendation logic
    - ML outputs enrich **validated datasets** to power personalized learning content discovery.
- **Key architectural patterns**:
    - Event-driven, multi-zone data processing
    - Containerized ML microservices triggered by Spark jobs
    - Modular ETL orchestration (Airflow + NiFi)
    - Real-time dashboards via Power BI

Architecture Artifacts Summary

1. Executive Architecture (Decision View)

Purpose
Supports *executive and architectural decision-making* by presenting system boundaries, responsibilities, and scalability considerations.

Key Focus

- Clear separation of learning experience, ingestion, analytics, and ML services

Architecture & Design Description

- Explicit support for **bulk + continuous ingestion paths**
- Modular, evolutive ML adoption (classification, NLP tagging, recommendations)
- Analytics as a strategic enabler for workforce readiness and compliance

Audience

- Executives
- Enterprise architects
- Program sponsors
- Business owners

2. Conceptual Architecture (Business View)

Purpose
Defines *what the business needs*, independent of technology choices.

Key Focus

- Digital learning delivery for field employees (VR headsets & mobile devices)
- Integration with enterprise HR systems for user, role, and training management
- Centralized learning data platform supporting both **initial bulk data onboarding** and **continuous ingestion**
- Analytics and machine-learning–driven personalization to improve learning effectiveness

Audience

- Business stakeholders & Architects
- Product owners
- Non-technical leadership

3. Logical Architecture (System Interaction View)

Purpose
Explains *how the system works* from an end-to-end functional perspective.

Key Focus

- Integration of learning devices with HR systems
- Data ingestion orchestration (initial loads and ongoing updates)
- Multi-zone data lake design (staging, raw, validated, curated, archive)
- Spark-based processing and enrichment pipelines

Architecture & Design Description

- ML-enabled content classification and recommendation flows
- BI consumption using both Import and DirectQuery patterns

Audience

- Solution architects
- Senior engineers
- Data engineers

4. Physical Architecture (Deployment & Infrastructure View): The physical architecture is not covered in this document for this project, as it depends on the corporation's existing production environment, which cannot be disclosed in this presentation.

Purpose
Describes *where and on what the solution runs*.

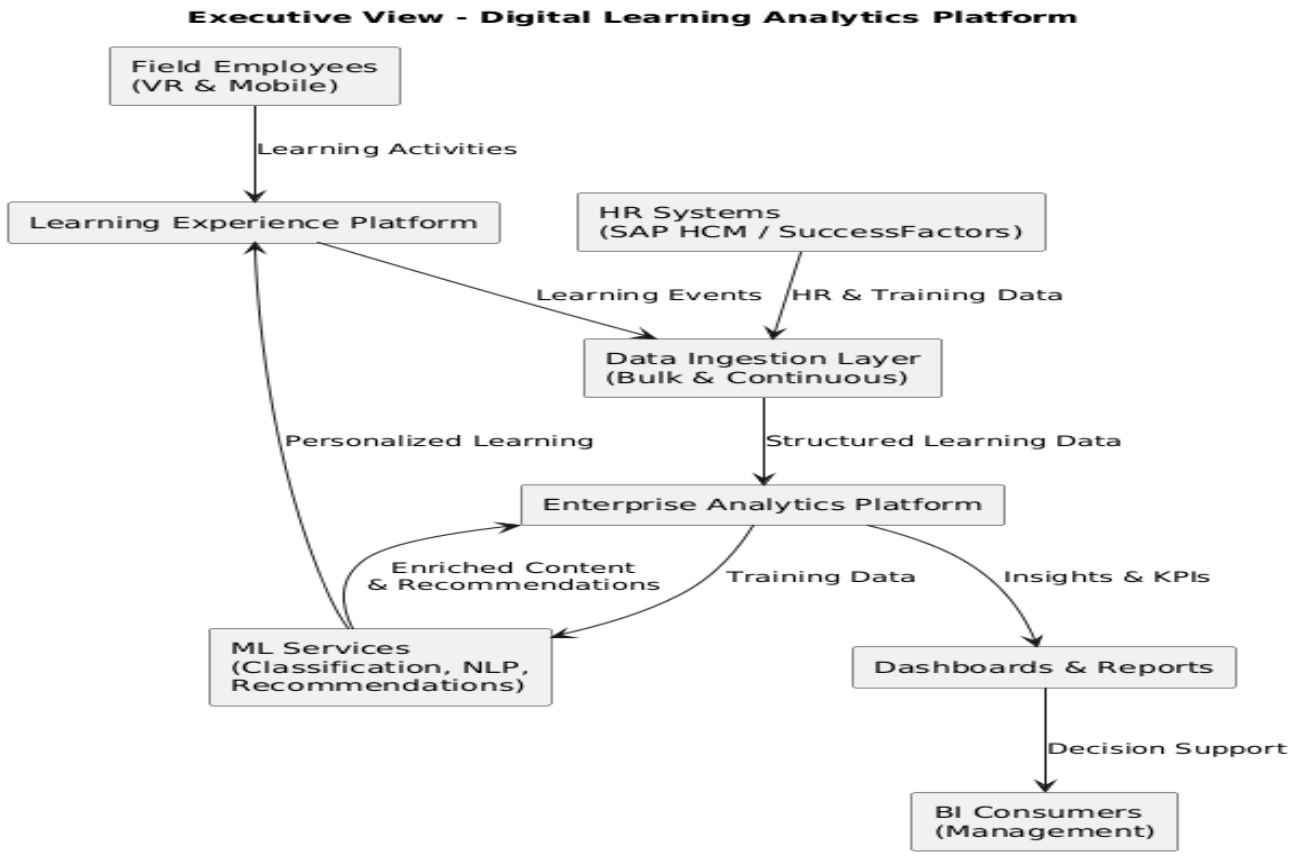
Key Focus

- Containerized deployment on Kubernetes
- Apache NiFi and Airflow for continuous and bulk ingestion orchestration
- Apache Spark for batch and streaming processing
- Delta Lake on MinIO object storage with open formats (Parquet)
- Containerized ML microservices
- Elasticsearch for search and indexing
- Power BI for analytics consumption

Audience

- Platform engineers
- Cloud architects
- Operations and DevOps teams

I. **Executive Architecture – Digital Learning Platform (Diagram 2):** High-level view for **management and sponsors**, focusing on scale, value, and risks.



The proposed architecture establishes a scalable digital learning and analytics platform designed to support field employees across multiple regions. The solution integrates enterprise HR systems with immersive learning devices and consolidates training data into a centralized analytics platform. The architecture supports both initial bulk data onboarding and continuous data ingestion, ensuring a smooth transition from legacy data to real-time operational insights. The platform is designed for horizontal scalability, enabling growth in users, learning content, and analytics workloads without architectural redesign.

Machine learning capabilities are introduced as modular services, allowing progressive adoption of advanced features such as content classification, NLP-based tagging, and recommendation engines. The solution minimizes vendor lock-in through containerized processing and open data formats, while enabling executive visibility through real-time dashboards that support workforce planning, compliance tracking, and training optimization.

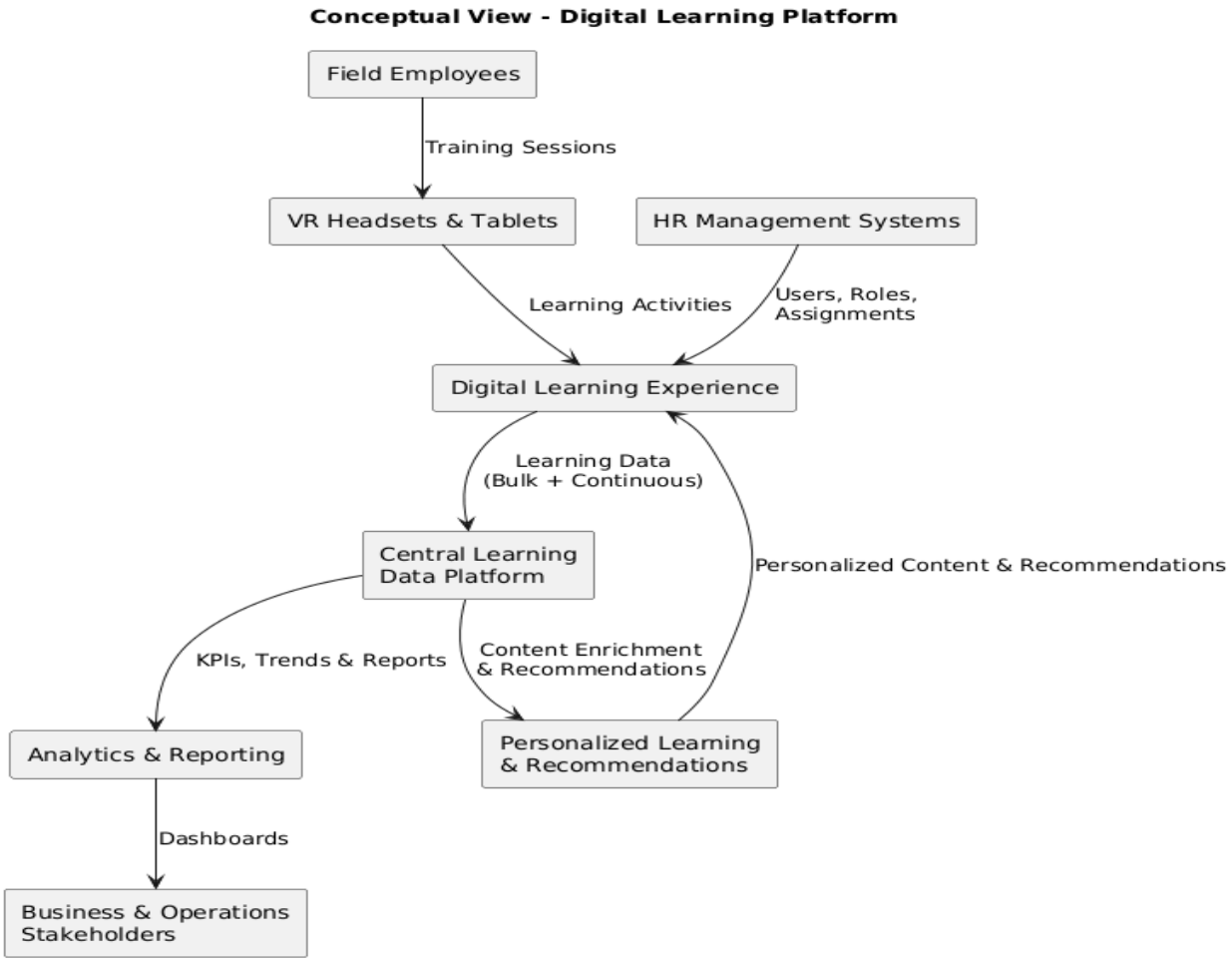
Executive-Level Decisions Highlighted

- Support for bulk + streaming ingestion
- Scalability and future growth
- Modular ML adoption (no big-bang risk)
- Analytics value for leadership
- Technology neutrality / portability

This is the view **executives actually read**.

Architecture & Design Description

II. **Conceptual Architecture – Digital Learning Platform (Diagram 1):** This diagram covers the high-level business specifications required for this project and is used to align stakeholders on scope and objectives. Business-oriented, **technology-agnostic** view describing *what* the platform delivers.



The solution enables a digital learning ecosystem for field employees, supporting immersive training via VR headsets and tablets. The platform integrates enterprise HR systems to manage users, roles, and training assignments, while capturing learning activity and operational telemetry.

Training content and usage data are centralized into an enterprise data platform that supports both historical analysis and near-real-time insights. Advanced analytics and machine learning capabilities enrich learning content through automated classification, tagging, and recommendation, enabling personalized learning journeys.

The platform supports two ingestion modes:

- Bulk ingestion for initial historical data loads
- Continuous ingestion for ongoing learning events and updates

Business stakeholders consume insights through interactive dashboards to measure training effectiveness, compliance, and workforce readiness.

Architecture & Design Description

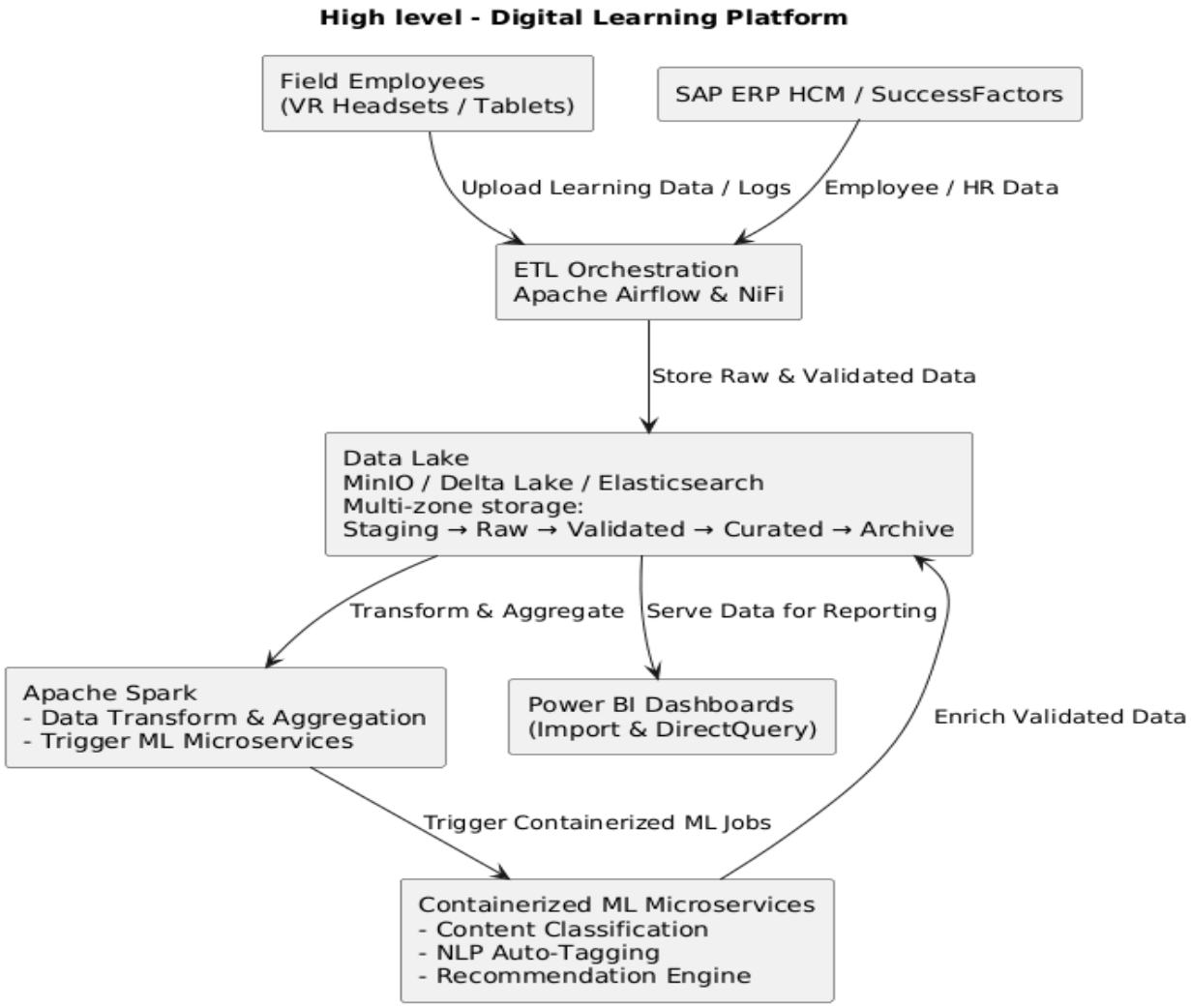
Conceptual View – Key Business Capabilities

- Digital learning delivery (VR, mobile & tablet)
- HR system integration
- Centralized learning data platform
- Historical + continuous data ingestion
- Analytics and reporting
- Personalized learning recommendations

No technology names here — this is intentional

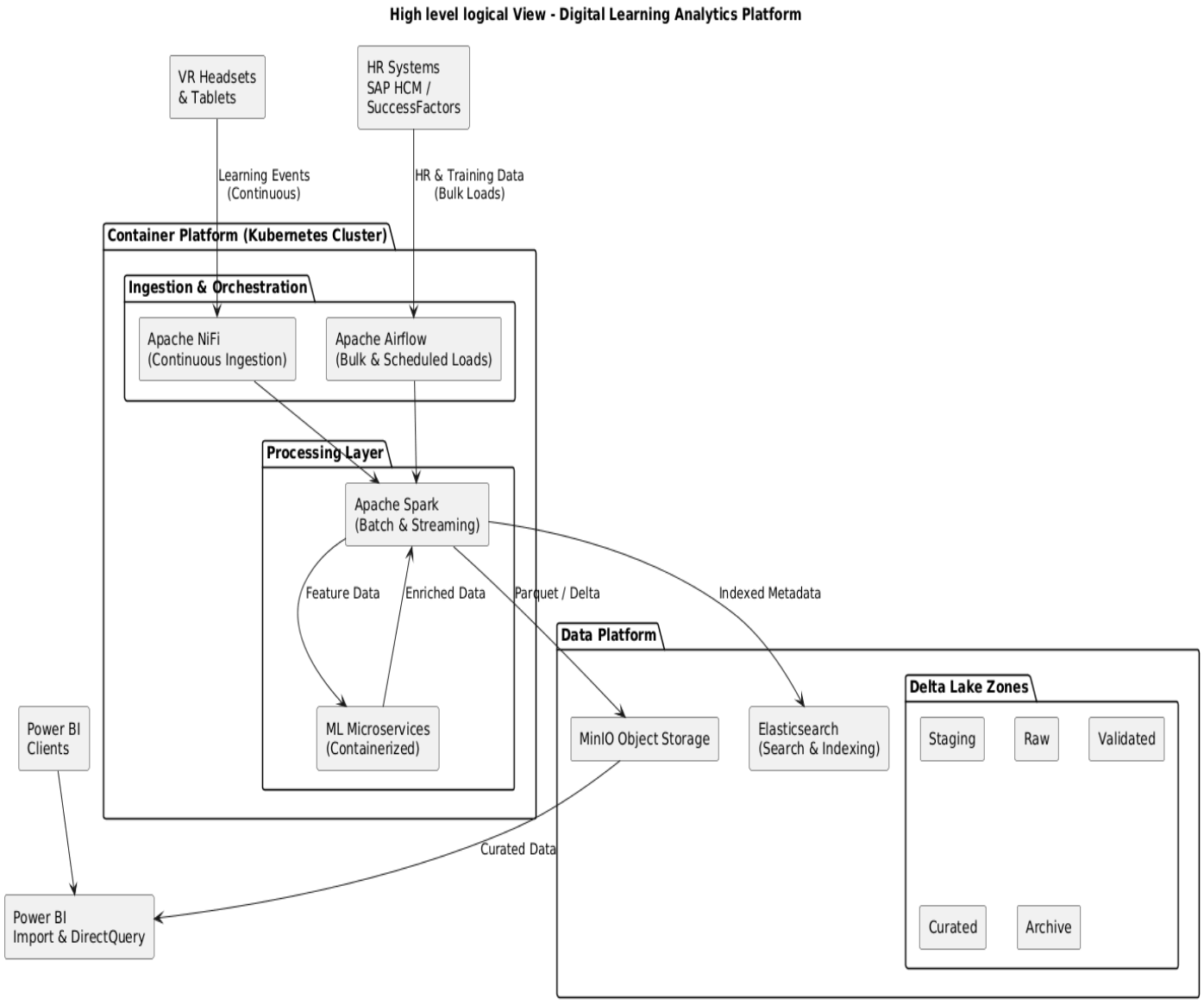
III. **Logical Architecture – Digital Learning Platform (Diagram 3, 4, 5, 6 & 7)**

- **High Level View (Diagram 3)**



Architecture & Design Description

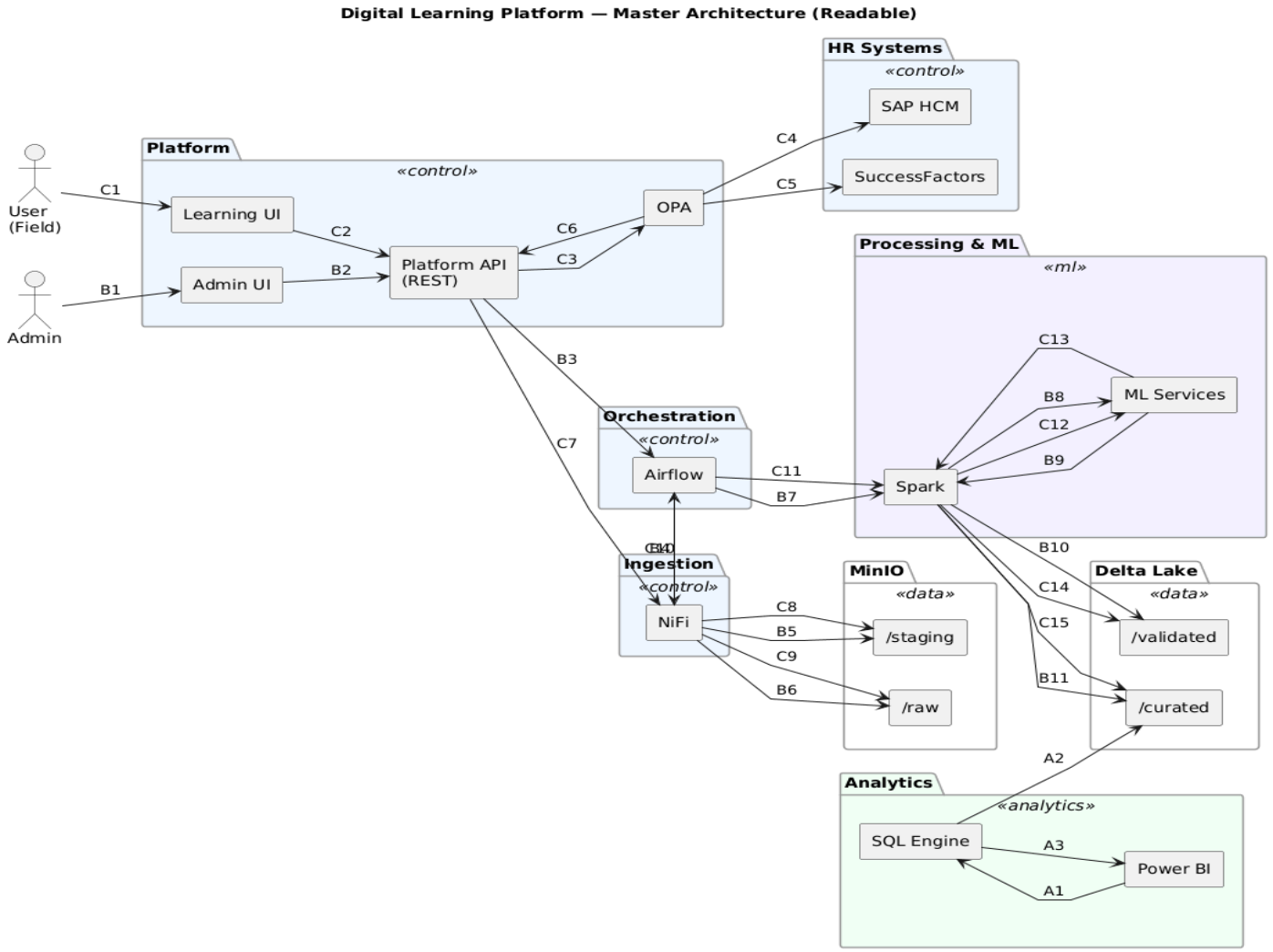
- **High-level Logical View (Diagram 4)**



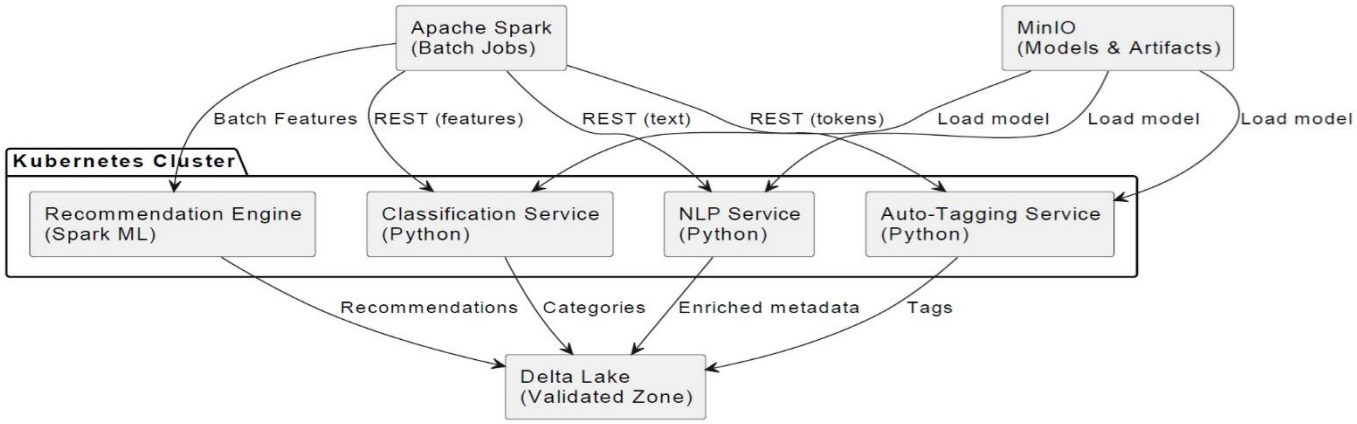
Logical View covers

- Data ingestion pipelines (bulk + continuous)
- Orchestration (Airflow, NiFi)
- Processing (Spark)
- Storage zones (staging → curated)
- ML enrichment pipelines: Restful API microservices (deployed within Kubernetes platform)
- BI consumption (Import + DirectQuery)

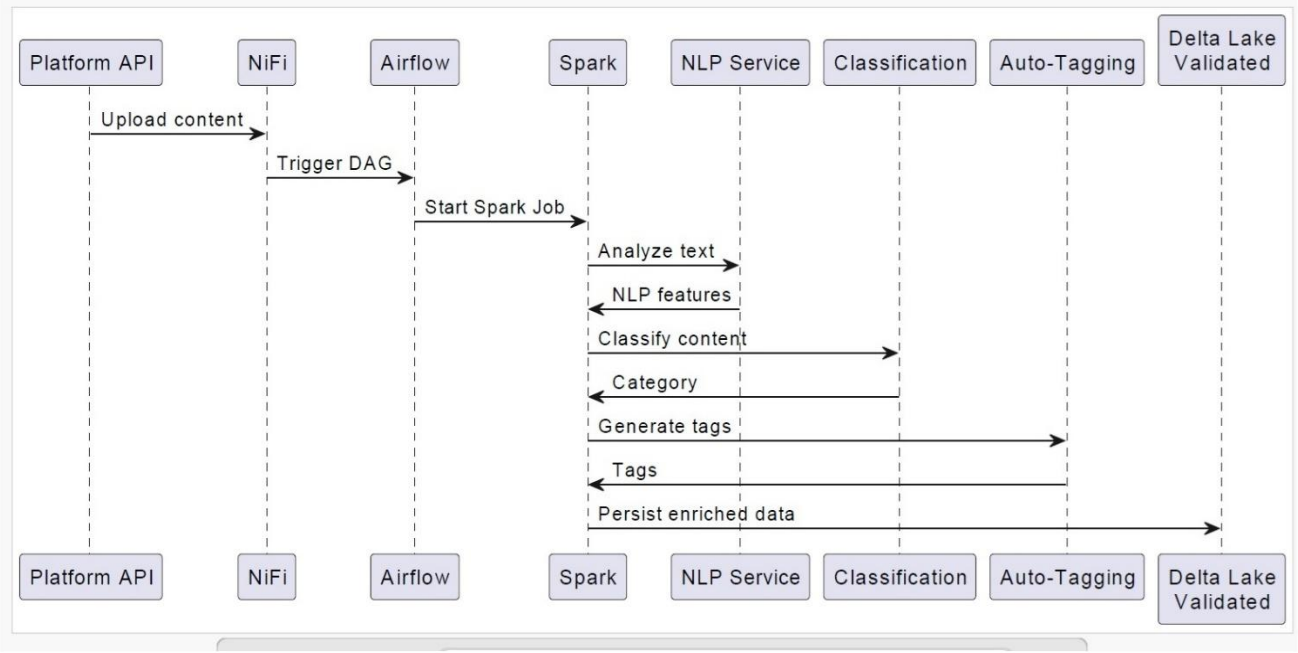
- **Logical View with Data Flow (Diagram 5):** Within this diagram, “C” designates continuous flows, and “B” designates initial bulk flow events.



- **ML Microservices Design Logical View (Diagram 6)**



- **ML Microservices data flow (Diagram 7):** ML microservices NLP pipeline for continuous flow



It shows **systems, components, and interactions**:

- VR devices, tablets
- SAP HCM / SuccessFactors
- Data Lake zones (staging, raw, validated, curated, archive)
- Airflow, NiFi, Spark, Kubernetes, ML microservices
- Power BI consumption modes

It explains **how the system works**, data flows, and responsibilities. The **microservices** ML pipeline flow **remains the same for Bulk ingestion**. The NLP pipeline flows happen before Recommendation engine processing activities.

The logical architecture describes the functional architecture of the digital learning and analytics platform, detailing how learning content, usage telemetry, and operational data are ingested through batch and continuous pipelines, validated and enriched via ETL processes, and processed by analytics and ML services. It illustrates interactions between data ingestion components (NiFi, Airflow), processing engines (Spark), storage layers (Data Lake zones and analytical models), and consumption services (Power BI, ML-driven recommendation services), independently of underlying infrastructure and deployment topology. Metadata Microservice (metadata storage) & Search Index Microservice (document/material indexing with Elasticsearch) are processed after ML microservices execution.

Individual ML Microservices & Recommendation Engine Design

1. NLP Micro Service (Text Processing) / REST (input text) / Python / Outputs: Enriched metadata

Purpose

- Tokenization

- Language detection
- Lemmatization: reduces words to their base or dictionary form (called a lemma / e.g. running --> run, better --> good)
- Keyword extraction
- Embedding (enabling models to understand relationships like "king" related to "queen" as "man" related to "woman")
- NER (Named Entity Recognition / e.g. California: Location, 2003: Date)

Inputs


```
{
 "content_id": "123",
 "text": "Safety procedures for hydraulic systems"
}
```


Outputs: Enriched metadata


```
{
 "content_id": "123",
 "language": "en",
 "keywords": ["safety", "hydraulic", "procedures"],
 "tokens": ["safety", "procedure", "hydraulic"]
}
```


ML techniques / Libraries

- NLTK (Natural Language Toolkit): prepares raw, unstructured text data for use with various ML algorithms. NLTK is an NLP library (NLTK can be used for Lemmatization).
- TF-IDF (Term Frequency-Inverse Document Frequency): measures a word's relevance to a specific document within a collection & identify the most important words in a document (executes tokenization, normalized text by removing stop words, & process lemmatization, then determine the frequency of each term in each document & define how common or rare terms are across all documents then creates a numerical weight for each word). TF-IDF is a statistical measure & a text feature-extraction technique.
- Word2Vec: capture semantic meaning (embedding algorithm / unsupervised ml algorithm)
- BERT (Bidirectional Encoder Representations from Transformers): It focuses on understanding text rather than generating it (leverages deep bidirectional context to understand language & help distinguish the context of each word). BERT is machine learning model architecture & framework developed by Google for NLP. Its pre-training phase uses an unsupervised approach, but its application often involves supervised fine-tuning (used for embedding).
- spaCy: processes raw text through a customizable pipeline of components to produce a structured Doc object with linguistic annotations. spaCy is an NLP library (used for tokenization, lemmatization & NER).

2. Classification Micro Service / REST (inputs features) / Python / Outputs: Categories

Purpose

- Assign category (Safety, Operations, Compliance)
- Predict difficulty level or other classification property

Inputs


```
{
  "content_id": "123",
  "features": {
    "keywords": ["safety", "hydraulic", "procedures"],
    "confidence": 0.94
  }
}
```


Outputs: Categories


```
{
  "content_id": "123",
  "category": "Safety",
   "confidence": 0.94
} // Classification microservice logic e.g.: if confidence > 0.9: category = "Critical Safety Procedure" else …
```


ML techniques (supervised machine learning algorithms)

- Logistic Regression
- SVM (Support Vector Machine)
- Naive Bayes

3. Auto-Tagging Micro Service / REST (inputs: tokens) / Python / Outputs: Tags

Purpose

- Generate tags for search & recommendation

Inputs


```
{
  "content_id": "123",
  "tokens": ["safety", "procedure", "hydraulic"]
}
```


Outputs: Tags


```
{
  "content_id": "123",
  "tags": ["hydraulics", "maintenance", "safety"]
}
```


ML techniques

- TF-IDF ranking
- Keyword matching
- Rule-based enrichment

4. Recommendation Engine (Batch) / Batch (inputs batch features) / Spark ML (PySpark) / Outputs: Recommendations

Purpose

- Recommend learning content

Inputs

- Feature tables from fact & dimension tables (e.g. user_content_affinity or user_engagement_features)

ML techniques

- Spark MLlib ALS (Alternating Least Squares algorithm)
- Content-based similarity: Suggests items to a user based on the features or attributes of items they have been previously liked. It operates by matching an individual user's profile with item characteristics.

Outputs: Recommendations

```
{
 "user_id": "456",
 "recommended_content": [
  {"content_id": "123", "score":0.87}
 ]
}
```

Using Policy Enforcement & Governance (OPA): Rego (declarative policy language used to write rules for the OPA)

Open Policy Agent (OPA) is used as a centralized **policy-as-code engine** to enforce governance, security, and data access rules across the analytics and machine learning platform. OPA policies defined fine-grained authorization rules for **data access, feature consumption, and ML pipeline execution**, ensuring consistent enforcement across **Apache Spark jobs, streaming pipelines, orchestration workflows, and API-based services (including integration with 3rd party system like SAP)**.

By externalizing authorization logic from application code, OPA enabled **declarative policy management, version-controlled governance, and environment-independent enforcement**, supporting regulatory compliance, least-privilege access, and auditable decision-making across both **batch and real-time workloads**. OPA is part of the control plan. In production, it typically operates as a lightweight HTTP server, acting as a policy decision engine that applications query. It can run also as a sidecar container. OPA is commonly run on the same machine (host/ Kubernetes pod) as the service querying it, acting as a local server to reduce latency. It listens for JSON API requests (REST API), evaluates them against Rego policies (policy-as-code), and returns a policy decision.

This is where **OPA is used**:

OPA evaluates

- user attributes (role, job function, skill level)
- content metadata (difficulty, compliance level, language)
- context (device type, location, training phase, prerequisites)

Example OPA decision questions

- can a field technician access advanced VR training?
- can an employee access safety training without prior certification?
- is this content restricted to a specific region or job role?
- is access allowed only after completing prerequisite modules?

Architecture & Design Description

OPA makes the final policy decision. Typical flow: Employee → Learning App → Backend API

→ OPA (policy decision)

→ Content Service (allowed / denied)

OPA is **not replacing IAM.** OPA is **augmenting it** where IAM becomes too rigid or coarse. OPA is used when authorization depends on:

- Multiple attributes (user + content + context)
- Dynamic conditions
- Business rules that change frequently
- Decisions beyond “role = X”

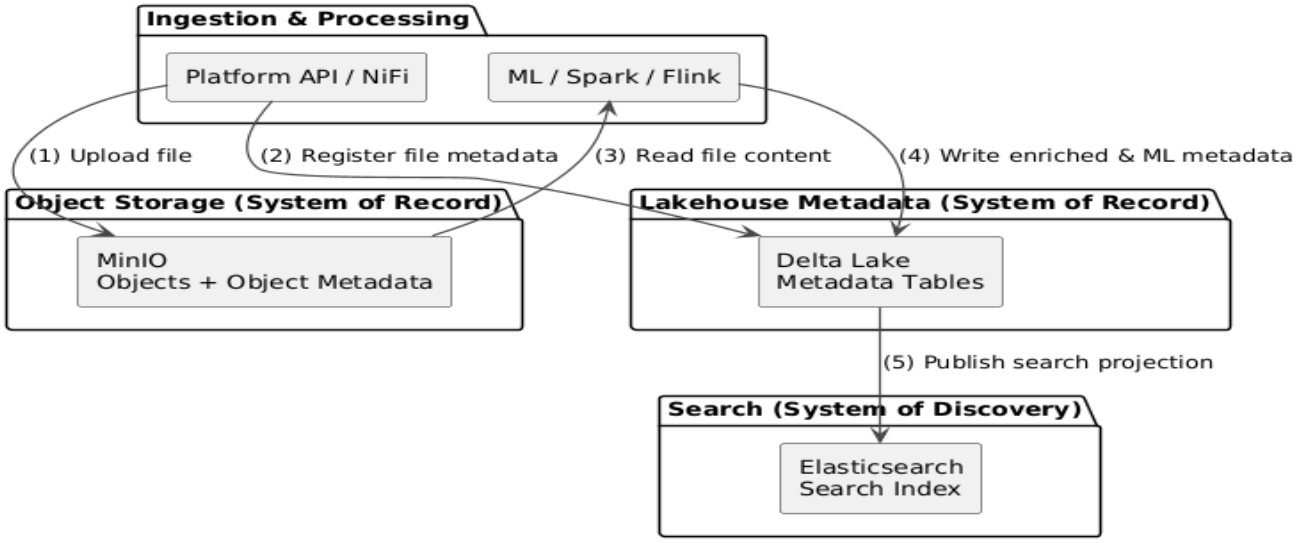
OPA is commonly used in **three additional places** in our platform:

- **Service-to service authorization** like can the Recommendation service access user performance data. The IAM system manages **Identity** (authentication of the service), while **Open Policy Agent (OPA)** manages the **Decision** (contextual authorization).
    - **IAM/Identity Phase:** Using OpenID Connect (OIDC) over OAuth 2.0, the Recommendation Service authenticates with the IAM/Identity Provider (IdP) to obtain an Access Token.
    - **Request Phase:** The Recommendation Service includes this Access Token in the authorization header of its request to the target service (e.g., the User Data Service).
    - **Authorization Phase:** The request is intercepted by **OPA (acting as a policy enforcement proxy)**. This phase validates the token and ensures the Recommendation Service has the "coarse-grained" permission to call the target API.
    - **Decision Phase:** If the service-to-service check passes, the request is forwarded to the target service.
- **Data access governance (analytics & ML)** to enforce data-level policies like data consultation eligibility.
    - **Data Retrieval:** The target service (e.g., Document Consultation Service) receives the request and retrieves the requested records (e.g., from Delta Lake)
    - **Resource Metadata Enrichment:** The service extracts attributes from the retrieved data (e.g., "Document #123 is tagged as a Private Safety Record").
    - **Fine-Grained Policy Enforcement:** The target service makes a call to **OPA**, providing the **Identity Context**, the **Request Metadata**, and the newly retrieved **Resource Metadata**.
    - **Final Decision:** OPA evaluates the full context. If OPA returns "Permit," the service provides the given user with access to the records. If "Deny," the service returns a 403 Forbidden error, ensuring that the Recommendation Service never leaks unauthorized sensitive information.
- **Lifecycle & automation** for enforcing governance in ML workflow like which models ca be promoted to production or which features are allowed for inference.

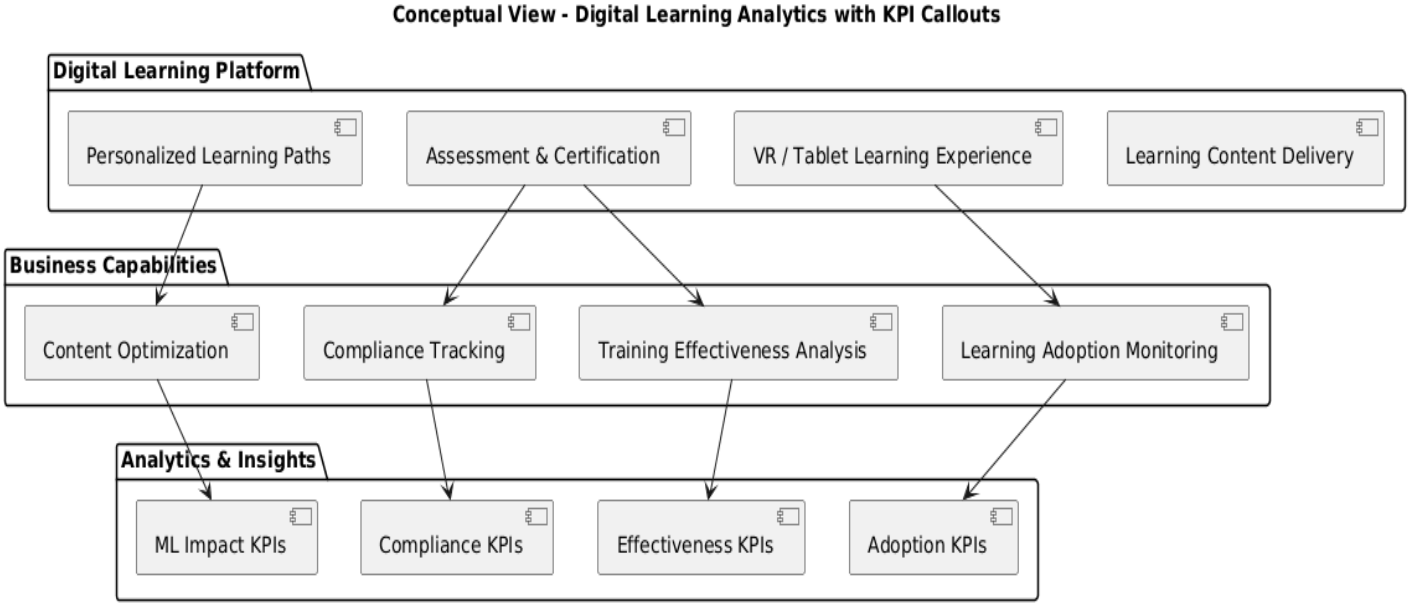
Feature	**IAM (The Token)**	**OPA (Rego Policy)**
Question	Who are you?	Are you allowed to do *this* right *now*?
Output	Access Token	A "True" or "False" decision
Stability	Static (Roles rarely change)	Dynamic (Policies can change instantly)
Granularity	Coarse (e.g., Service:Read)	Fine (e.g., Can read only if Document.Status == 'Public')

IV. Analytic Architecture – Digital Learning Platform (Diagram 8, 9 & 10)

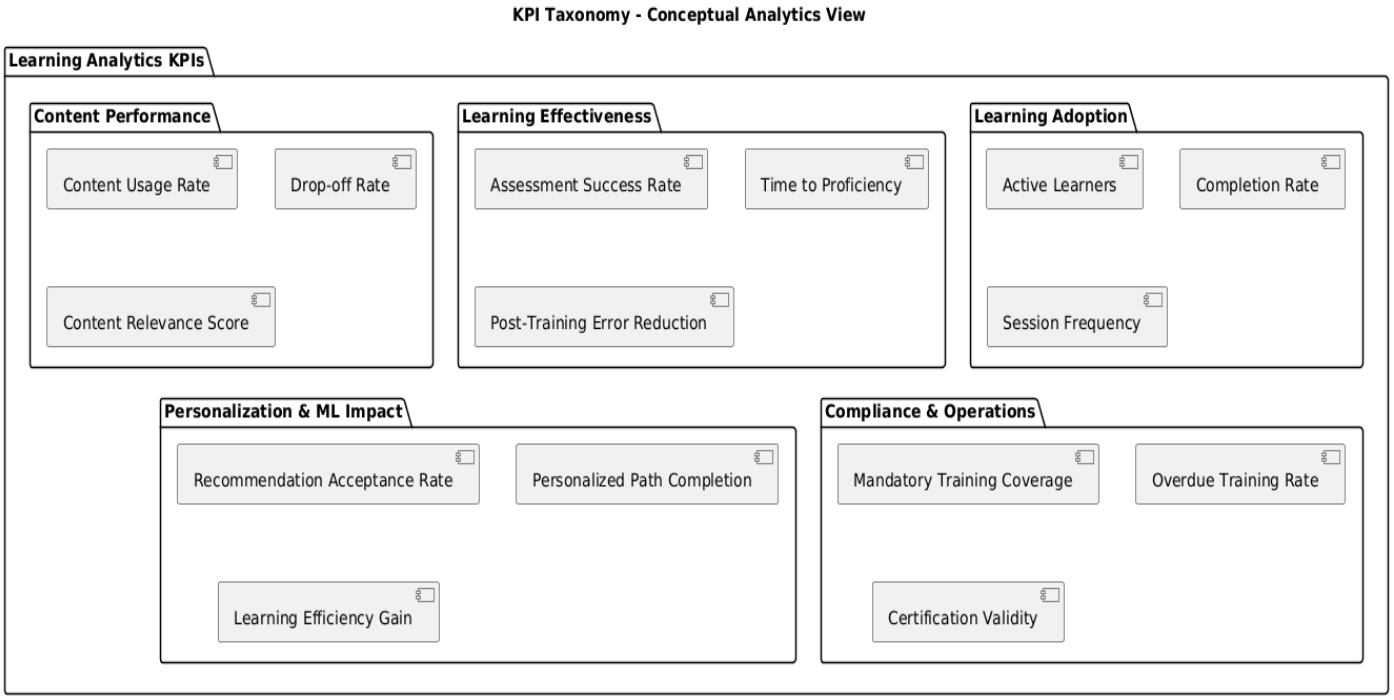
- **Storage & Lakehouse Metadata (Diagram 8):** Metadata supporting storage management, search and indexing activities, machine learning pipelines, and BI & analytics consumption (executive view).



- **Conceptual View - Digital Learning with Analytics with KPI Callouts (Diagram9):** Illustrates business capability map



- **KPI Taxonomy Conceptual View (Diagram 10)**



Overview

The Digital Learning Platform is designed to support **field employees** through **personalized, role-based learning experiences**, while providing **management and business stakeholders** with **actionable insights** on learning effectiveness, engagement, and workforce readiness.

At a conceptual level, the platform integrates **content management**, **learning delivery**, **analytics**, and **intelligence capabilities** into a unified business ecosystem, independent of underlying technologies.

Core Business Capabilities

1. Learning Content Management

This capability enables the organization to centrally manage learning materials (VR modules, videos, documents, interactive content) and maintain content lifecycle governance.

Key business functions

- Content upload and versioning
- Content classification and tagging
- Multilingual and multi-format support
- Content lifecycle management (draft, published, archived)

KPI Callouts

- Number of active learning materials
- Content reuse rate
- Content freshness index (updated vs outdated content)

2. Learning Delivery & User Experience

This capability focuses on delivering learning content to employees through **VR headsets, tablets & computers**, adapted to their role, context, and assigned responsibilities.

Key business functions

- Role-based content access
- Personalized learning paths
- Multi-device learning experience
- Offline and field-friendly access

KPI Callouts

- Active learners per period
- Learning completion rate
- Average learning time per employee
- Device usage distribution (VR vs tablet)

3. User & Workforce Context Management

This capability integrates HR systems (SAP HCM, SuccessFactors) to contextualize learning activities based on organizational structure and employee profiles.

Key business functions

- Employee role and position alignment
- Organizational hierarchy awareness
- Project and assignment-based learning eligibility
- Compliance-driven learning assignment

KPI Callouts

- Learning coverage per role
- Mandatory training compliance rate
- Learning adoption by department

4. Learning Analytics & Performance Monitoring

This capability provides **visibility into learning usage, effectiveness, and outcomes**, transforming raw activity data into business-relevant metrics.

Key business functions

- Learning activity tracking
- Aggregated usage analytics
- Performance trend analysis
- Executive dashboards and reporting

KPI Callouts

- Engagement score (sessions, duration, completion)
- Learning effectiveness indicators
- Skill progression trends
- Correlation between learning and operational KPIs

5. Intelligent Content Enrichment (AI / ML)

This capability enhances learning content using **automated intelligence**, improving discoverability and relevance without manual effort.

Key business functions

- Automated content classification
- Auto-tagging and keyword extraction
- Learning material categorization
- Metadata enrichment for search and analytics

KPI Callouts

- Auto-tagging accuracy rate
- Search success rate
- Reduction in manual content curation effort

6. Personalized Recommendation & Guidance

This capability supports **guided learning** by proactively suggesting relevant materials to employees based on their behavior, role, and learning history.

Key business functions

- Behavior-based recommendations

- Role and project-based suggestions
- Continuous learning guidance
- Learning gap identification

KPI Callouts

- Recommendation adoption rate
- Content relevance score
- Repeat engagement after recommendation

Business Value Summary

From a conceptual standpoint, the platform enables:

- **Improved workforce readiness** through targeted learning
- **Higher engagement** via personalized and immersive experiences
- **Operational insight** through analytics-driven decision making
- **Reduced content management overhead** through automation
- **Continuous learning culture** aligned with business objectives

This conceptual view abstracts implementation details and instead highlights **what the business can do**, **how value is created**, and **which KPIs measure success**, forming the foundation for logical and physical architecture views.

How the Data Is Used End-to-End (Narrative Flow)

1. **User interactions** generate learning and platform events
2. Events are ingested into the **Data Lake (Bronze)**
3. Data is curated into **Silver/Gold Lakehouse layers**
4. KPIs are computed and stored in **Warehouse star schemas**
5. ML pipelines consume features from the **Feature Store**
6. Metadata enables governance, discovery, and trust
7. Power BI consumes **certified KPI datasets**

The platform separates analytical persistence concerns by design: raw and curated data is stored in a Lakehouse for scalability and reprocessing, business KPIs are materialized in a star-schema warehouse for BI consumption, and ML features are managed through a dedicated feature store, all governed by a centralized metadata layer."

7. Analytical Storage Layers

Our analytical architecture uses multiple analytical persistence layers, each optimized for a specific purpose:

Data Type	Primary Storage	Purpose
Raw events & logs	Data Lake (Bronze)	Traceability, replay, reprocessing
Curated analytical data	Lakehouse (Silver / Gold)	BI, KPIs, analytics
KPI aggregates	Star Schema (Data Warehouse)	Power BI, executive dashboards
Feature data (ML)	Feature Store (Lakehouse-backed)	Online & offline ML
Metadata	Metadata & Governance Layer	Discovery, lineage, orchestration

7.1 Data Lake / Lakehouse (Bronze, Silver, Gold)

This is the system of record for analytics.

Stored here:

- Learning events (views, clicks, completions)
- Assessment attempts and results
- Recommendation interactions
- Platform usage telemetry
- Operational business events

Why:

- Schema evolution
- Reprocessing capability
- Supports batch, micro-batch, and streaming
- Shared foundation for BI + ML

Example datasets:

- learning_events_bronze (raw data)
- learning_sessions_silver (3NF data acting as the source for the gold layer)
- content_performance_gold
- learner_outcomes_gold

7.2 Data Warehouse / Star Schema (Gold KPI Layer): This is where **Power BI–ready KPIs** live**.** Used for:

- Executive dashboards

Architecture & Design Description

- Operational reporting
- Cross-domain KPIs

Stored here:

- Fact tables with aggregated metrics
- Dimension tables for slicing and dicing

Example Star Schema

Fact tables

- FactLearningActivity
- FactAssessmentResults
- FactRecommendationUsage

Dimension tables

- DimLearner
- DimContent
- DimTime
- DimPlatform
- DimBusinessUnit

KPIs from our taxonomy (usage rate, drop-off rate, success rate, etc.) are materialized here, not calculated on the fly.

Mapping our KPI Domains to Storage

Examples

- Content usage rate
- Drop-off rate
- Session duration
- Completion rate

Storage

- Raw events → Lakehouse (silver)
- Aggregated KPIs → **FactLearningActivity (Star Schema in gold layer)**

Consumption

- Power BI

- Product analytics dashboards

Business Capabilities KPIs

Examples

- Recommendation acceptance rate
- Personalized path completion
- Learning effectiveness score

Storage

- Feature-level data → Feature Store (Lakehouse-backed)
- Aggregated KPIs → FactPersonalizationImpact

Consumption

- Power BI (aggregated)
- ML monitoring dashboards
- Model evaluation pipelines

8. Metadata: Where It Fits in the Architecture

It is managed by a dedicated metadata layer that spans:

- **Metadata Types**

Metadata Type	**Examples**
Technical	Schemas, partitions, file formats
Operational	Pipeline runs, freshness, SLA
Business	KPI definitions, ownership
ML Metadata	Features, model versions
BI Metadata	Dataset lineage, reports

Metadata supports Search & discovery, Lineage (source → KPI → dashboard), ML pipeline automation, semantic consistency

A recommendation model improves learning outcomes.

1. Feature metadata model improves learning outcomes
2. Pipeline metadata triggers retraining on new data

3. Model metadata tracks performance improvements
4. Prediction results are written back to the Lakehouse
5. KPI metadata defines “Recommendation Acceptance Rate”
6. Power BI consumes certified KPI datasets
7. Lineage metadata links model → KPI → dashboard

Same metadata layer supports ML automation *and* BI consistency. A centralized metadata layer enables end-to-end automation of machine learning pipelines while ensuring semantic consistency across BI and analytics consumption, providing lineage, governance, and trust from raw data ingestion to KPI dashboards.

V. Machine Learning Feature Extraction

1. Data Source

- Bulk data uploaded by Admin (large files, media materials, historical records)

2. Storage

- Stored in Delta Lake, structured in fact & dimension tables:
  - Fact table: precomputed employee-material interactions, recommendation scores, predictions
  - Dimension tables:
    - Employee profile (job type, title, project assignment)
    - Material metadata (documents, media types, field tasks)
    - Manager requests / assignments

3. Feature Extraction

- Spark ML jobs read the Delta Lake tables
- Features derived from fact & dimension tables
    - Employee profile → job type, experience, access level
    - Material metadata → type, field relevance, topic
    - Historical interactions / batch computed recommendations
- These features feed the model training phase (ML NLP, Classification & Auto-tagging microservices)

4. Usage

- Batch predictions are stored back in Delta Lake
- Field employees query these precomputed predictions via offline search

In short

- Source of feature data: Delta Lake fact & dimension tables
- Feature extraction process: done in Spark ML batch jobs
- Predictions: offline, precomputed, served to users from the Delta Lake tables

Enterprise HR & Learning Systems Integration (SAP HCM / LMS) with Policy Governance

The digital learning platform integrates with enterprise HR and Learning Management Systems, including legacy SAP HCM and LMS platforms evolving toward SAP SuccessFactors. The integration ensures secure, auditable, and governed data exchange across hybrid environments, enabling personalized learning, reporting, and analytics capabilities.

To enforce consistent access control and data governance, the architecture leverages a centralized policy enforcement layer using Open Policy Agent (OPA) with Rego rules. This approach guarantees least-privilege access across both on-prem and cloud components, while allowing safe extension of services such as analytics pipelines and machine learning feature stores.

The architecture is designed for hybrid execution: certain workloads remain on-premises due to regulatory or operational constraints, while others are deployed on cloud-native services, providing flexibility, scalability, and resilience.

Context: The platform must integrate with HR and LMS systems (SAP HCM & LMS → SuccessFactors) while maintaining secure, governed access across hybrid cloud/on-prem environments.
Decision: Use OPA (Rego) as a centralized policy enforcement engine for all HR/LMS data flows.
Consequences:

- Ensures least-privilege access and auditable control
- Supports both on-prem and cloud components
- Enables safe integration of analytics and ML pipelines
- Provides a reusable governance model for future enterprise system integrations

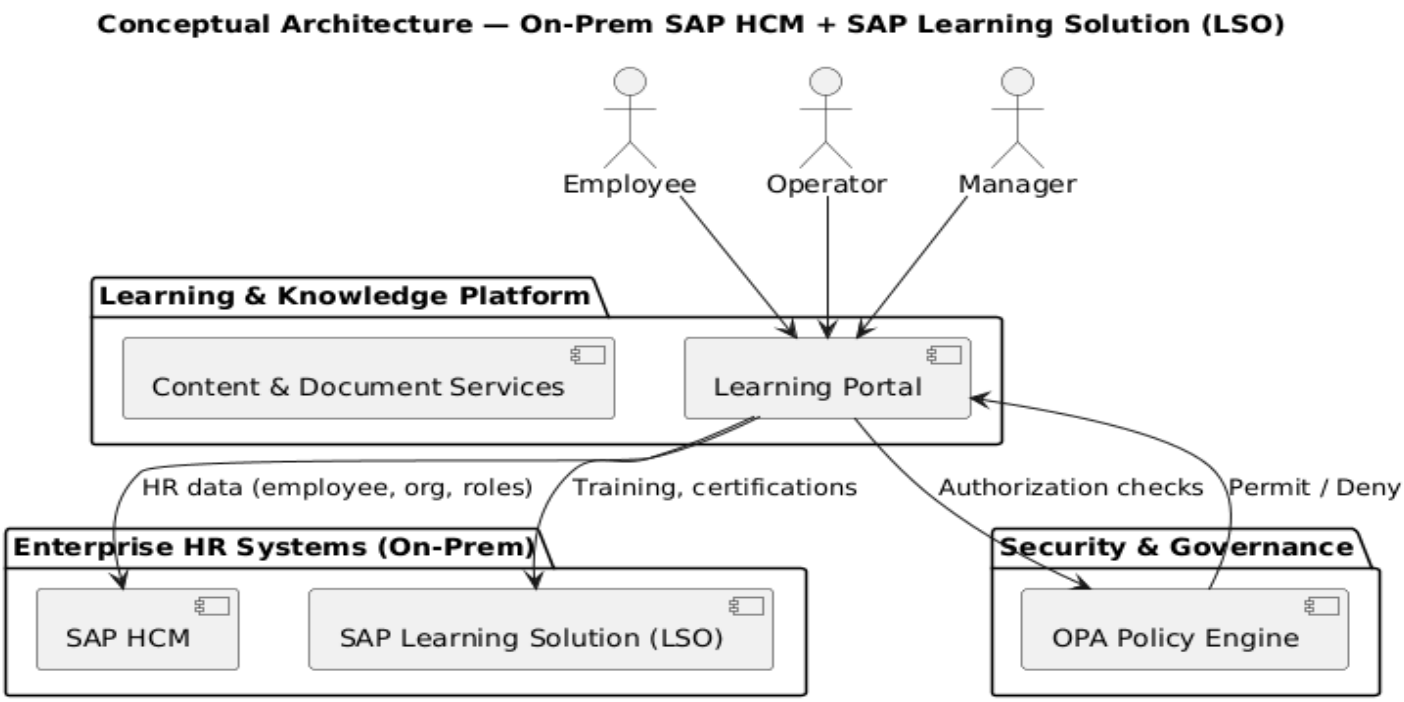
Recommended Diagram Strategy

Scope	Conceptual	Logical
On-prem SAP HCM + SAP LSO	✅ Yes	✅ Yes (detailed flows)
SaaS HR/LMS (SuccessFactors)	✅ Yes	✅ Yes (parallel but evolving)

Conceptual Architecture— On-Prem SAP HCM + SAP LSO (2019)

What this diagram communicates

- Single on-prem HR/LMS backbone
- Learning platform consumes HR + learning data
- Centralized authorization via OPA



Context remind: Before SuccessFactors, SAP Learning Solution (LSO) was part of the SAP HCM suite.

Logical — On-Prem SAP HCM + SAP LSO (with OData + OPA flows)

This is the **most important diagram**.

What must be visible (as you requested)

- OData-based integration
- Access vs download flows
- Examples:
    - Work orders
    - Maintenance reports
    - Operator notes
    - Asset documentation
- **OPA always in the path**

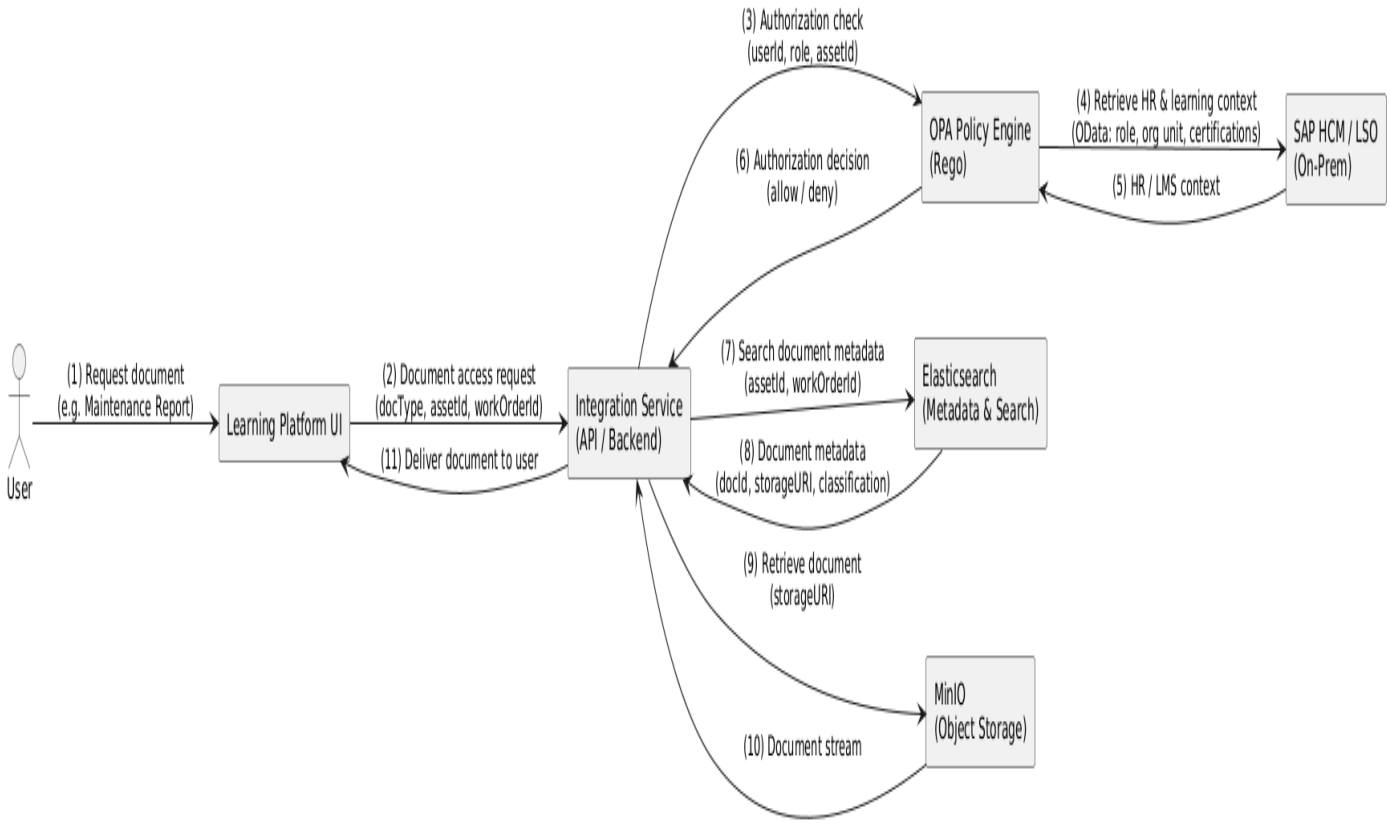
Logical Flow Explained

1. User requests a document (e.g., *Maintenance Report for Asset X*)
2. Platform calls OPA for authorization with:
    - User ID
    - Role

Architecture & Design Description

- Asset / Work Order ID
- Action (view, download)

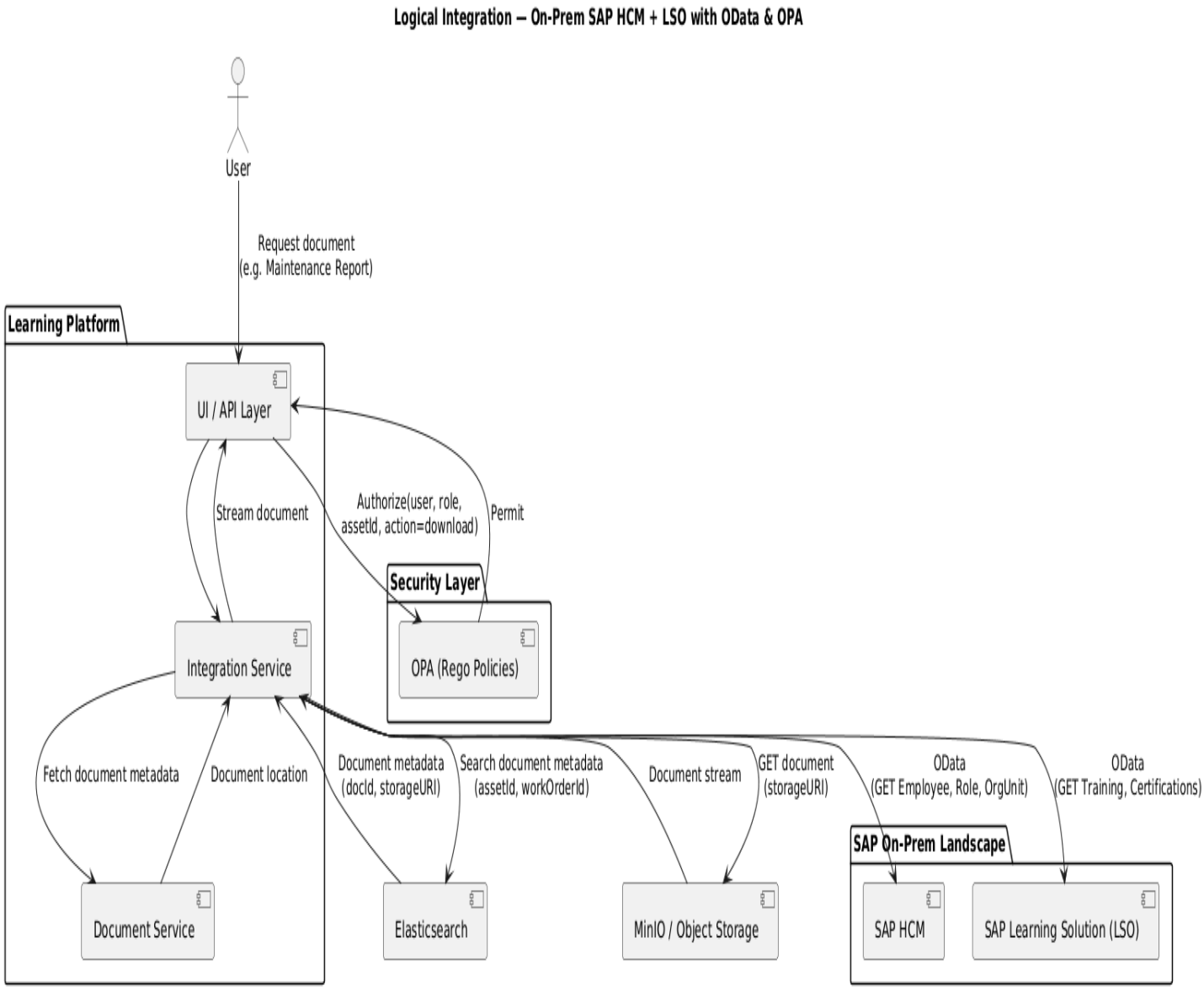
3. If **authorized**:
    - Platform queries **SAP HCM / LSO via OData:** Query SAP HCM / LSO (OData) for context *(role, org unit, certification, training validity)*
4. Query Elasticsearch for document metadata: Document metadata returned
5. Metadata points to object storage location
6. Document is fetched from MinIO/S3 curated zone: Content retrieved:
7. File streamed to user



Note about document metadata:

👉 Document metadata is returned by the platform's metadata/search layer (Elasticsearch)
👉 The actual document/content is retrieved from the storage layer (object storage / data lake — MinIO

Concern	System
Authorization context	SAP HCM / LSO
Search & metadata	Elasticsearch
Content (PDF, reports, docs)	Object storage (MinIO / lake curated zone)



Document metadata includes:

- Document ID
- Asset ID / Work Order ID

Architecture & Design Description

- Version
- Classification
- Storage URI (bucket/path)
- Sensitivity tag
- Retention info

This lives in Elasticsearch, indexed at ingestion time.

Document content:

- PDF
- DOCX
- Images
- Video/audio
- Reports

Lives in object storage (MinIO/S3)
Optionally:

- Raw → curated zones
- Delta Lake if versioned / governed

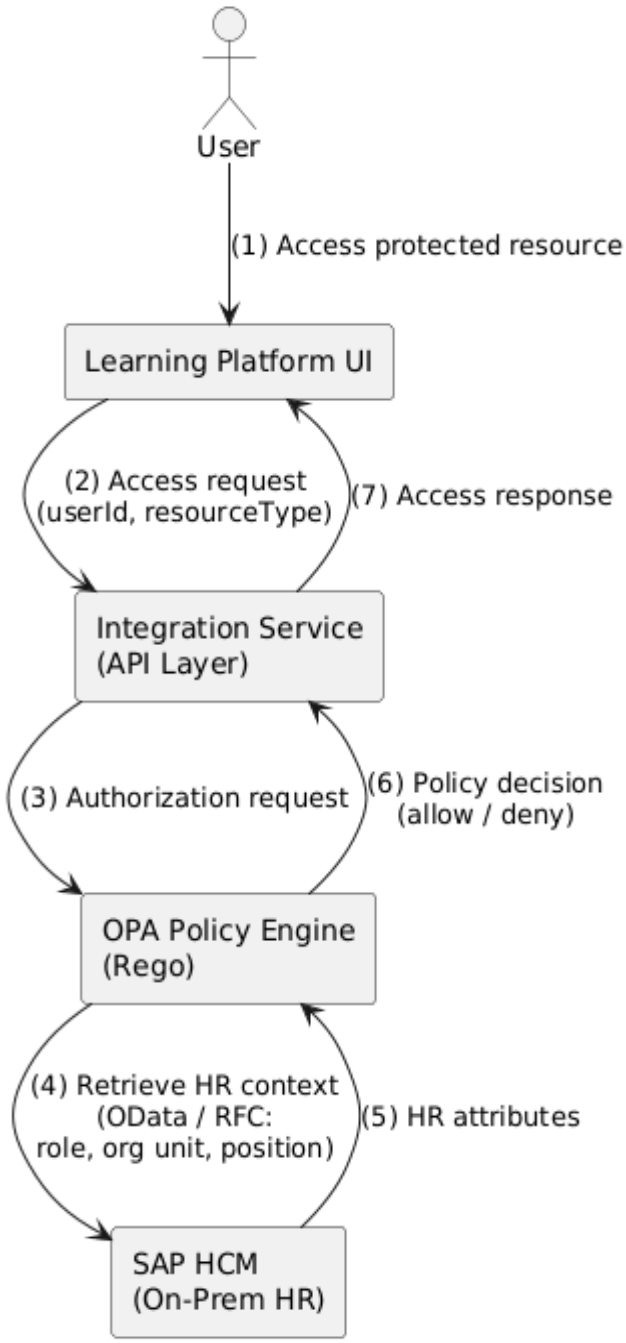
Document authorization is evaluated using HR and learning context retrieved from SAP HCM / LSO via OData. Document metadata and search are handled by a dedicated indexing layer (Elasticsearch), which references content stored in object storage (MinIO/S3), ensuring separation of access control, metadata management, and content storage.

Correct responsibility split

- **SAP HCM / LSO** → *identity & learning context*
- **OPA** → *authorization decision*
- **Elasticsearch** → *metadata & search*
- **MinIO** → *document content*

Diagram 1 — Authorization & HR Context

SAP HCM (On-Prem)

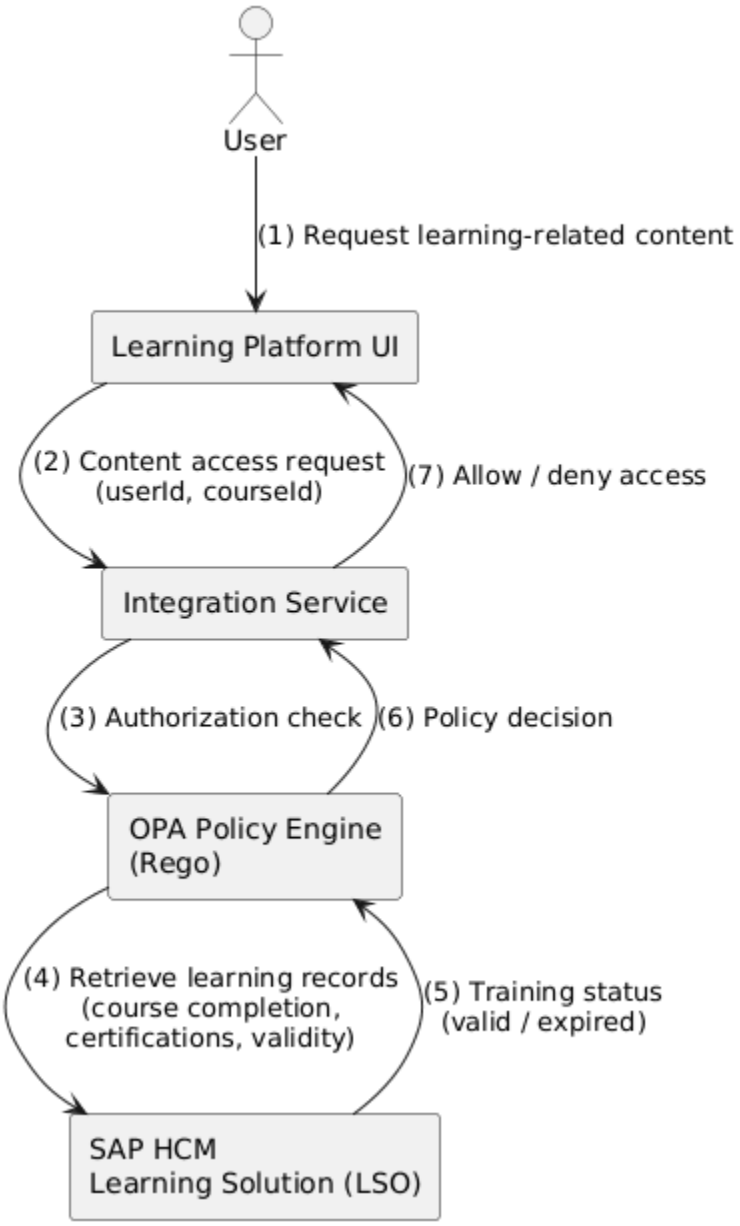


Notes

- SAP HCM is authoritative for identity & org structure
- Integration typically via OData (Gateway) or RFC/BAPI
- OPA already acts as externalized policy decision point

Diagram 2 — Learning & Certification Validation

SAP Learning Solution (LSO inside SAP HCM)

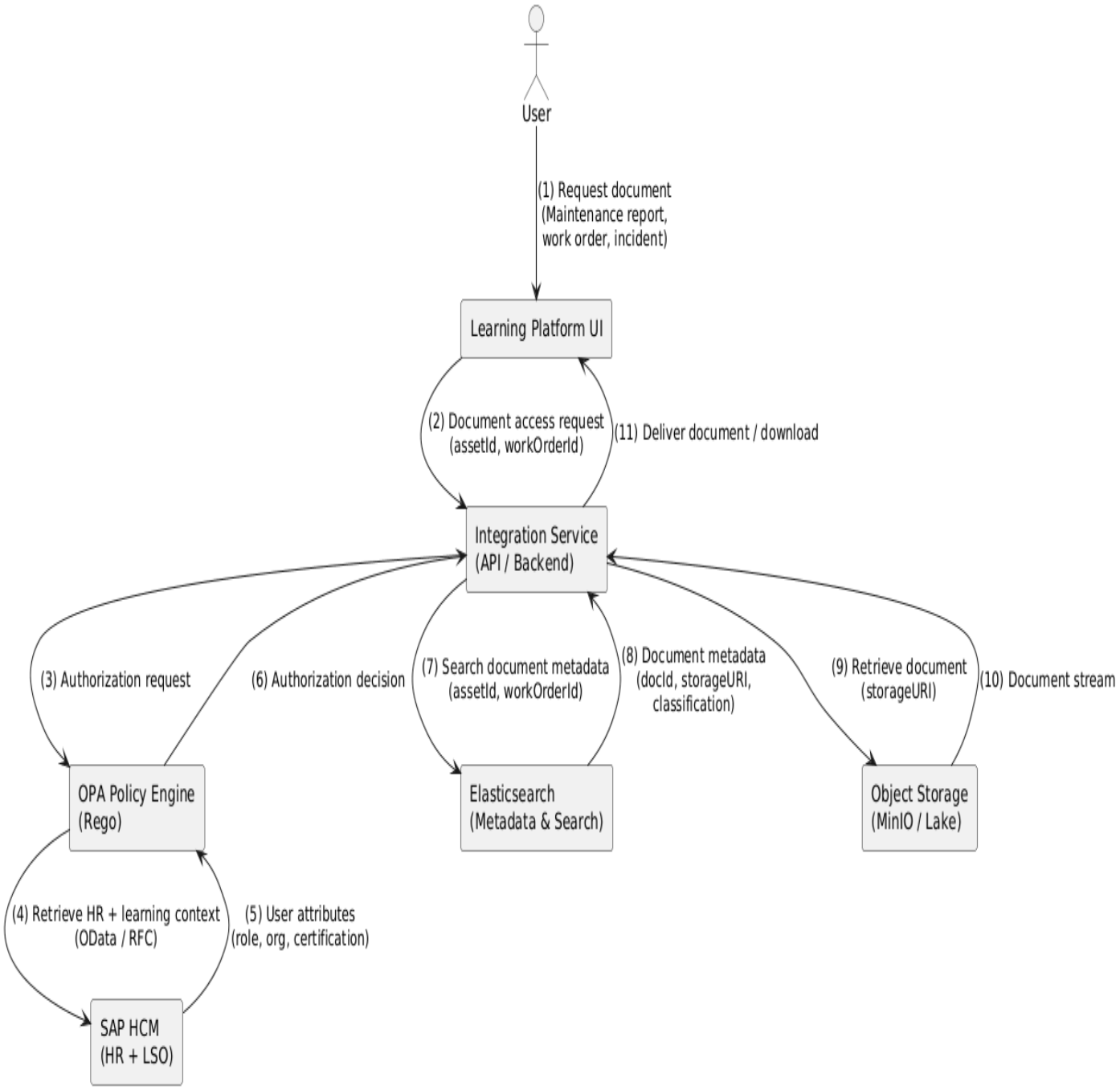


Notes

- LSO is not a separate product — it's a module of SAP HCM
- Training & certification checks are policy inputs, not logic

Diagram 3 — Document Access & Download Flow

SAP HCM / LSO + OPA + Metadata + Object Storage



Digital Learning Platform – Architecture Evolution 2019 → 2025 (supporting Amazon deployment)

1. Overview of 2019 Architecture

In 2019, the digital learning platform for field employees leveraged **batch-oriented ETL pipelines** and micro-batch processes for content ingestion, personalization, and reporting:

- **Data Ingestion:** Apache NiFi handled **bulk (initial loading to staging area) and micro-batch ingestion** from SAP ERP HCM, SuccessFactors, VR headset logs, and tablets.
- **Processing & Orchestration:** Apache Spark (on EMR for Amazon & Kubernetes for on-prem or hybrid deployments scenarios) performed **batch and micro-batch transformations**, and workflows were orchestrated using **Apache Airflow**.
- **Storage Layer:** A **Delta Lake architecture** on MinIO (S3-compatible) organized data across multiple zones: staging, raw, validated, curated, and archive.
- **Analytics:** Star-schema Data Warehouse enabled actionable insights via Power BI (Import and DirectQuery modes).
- **ML Features:** Spark-based containerized ML microservices performed content classification, NLP-based auto-tagging, and recommendation logic on validated datasets.

Limitations in 2019: Micro-batch processing was bound to Spark jobs and NiFi, limiting true real-time streaming and online ML predictions. Airflow, while effective for batch orchestration, lacked native support for continuous streaming workflows.

2. Architecture Evolution — Native 2025 Design

By 2025, the platform evolved to **handle continuous streaming, online ML predictions, and increased flexibility in orchestration**:

- **Streaming & Micro-batch:**
    - **Apache Flink** was introduced to handle **continuous ingestion and real-time processing**, replacing Spark micro-batch for streaming workloads. Apache Flink is considered for real-time personalization, event-driven recommendations, or sub-second analytics (if it is requested as core business requirements).
    - NiFi continued to support **initial bulk loads**, ensuring backward compatibility with historical batch workflows.
    - Kafka / MSK or Kinesis were used to buffer events for Flink pipelines.
- **Processing & ML:**
    - Apache Spark moved to **EKS (Kubernetes)** to perform batch processing and ML model training.
    - Containerized ML microservices could now support **online predictions**, enabling real-time content personalization for employees.

- **Orchestration:**
    - **Dagster** was introduced as an alternative to Amazon MWAA(Managed Workflow for Apache Airflow), offering better integration with streaming pipelines, dependency tracking, and observability.
    - Airflow/MWAA remained optional for batch scheduling, providing flexibility.
- **Storage:**
    - Delta Lake remained the foundation, now complemented by Iceberg tables for **enhanced schema evolution** and **incremental updates**.
    - Data zones (staging, raw, validated, curated, archive) persisted, with **streaming ingestion writing directly to raw and validated layers**.
- **Adding an Explicit Managed Feature Store**
    - Real-time personalization, Streaming ML features, Online inference etc.
- **Analytics & KPIs:**
    - Power BI dashboards now reflect **near real-time KPIs**, including dynamic metrics for VR content engagement, employee progress, and personalized recommendations.
    - Online ML predictions enrich KPIs, allowing adaptive learning experiences.
- **Key Architectural Decisions / Architecture Decision Records (ADRs):**
    - Micro-batch workloads from 2019 migrated to Flink for true streaming.
    - Dagster adopted for superior orchestration of streaming pipelines.
    - Spark retained for batch / ML, ensuring high performance for large dataset computations.
    - Explicit feature management to support real-time inference.

3. Summary of Changes

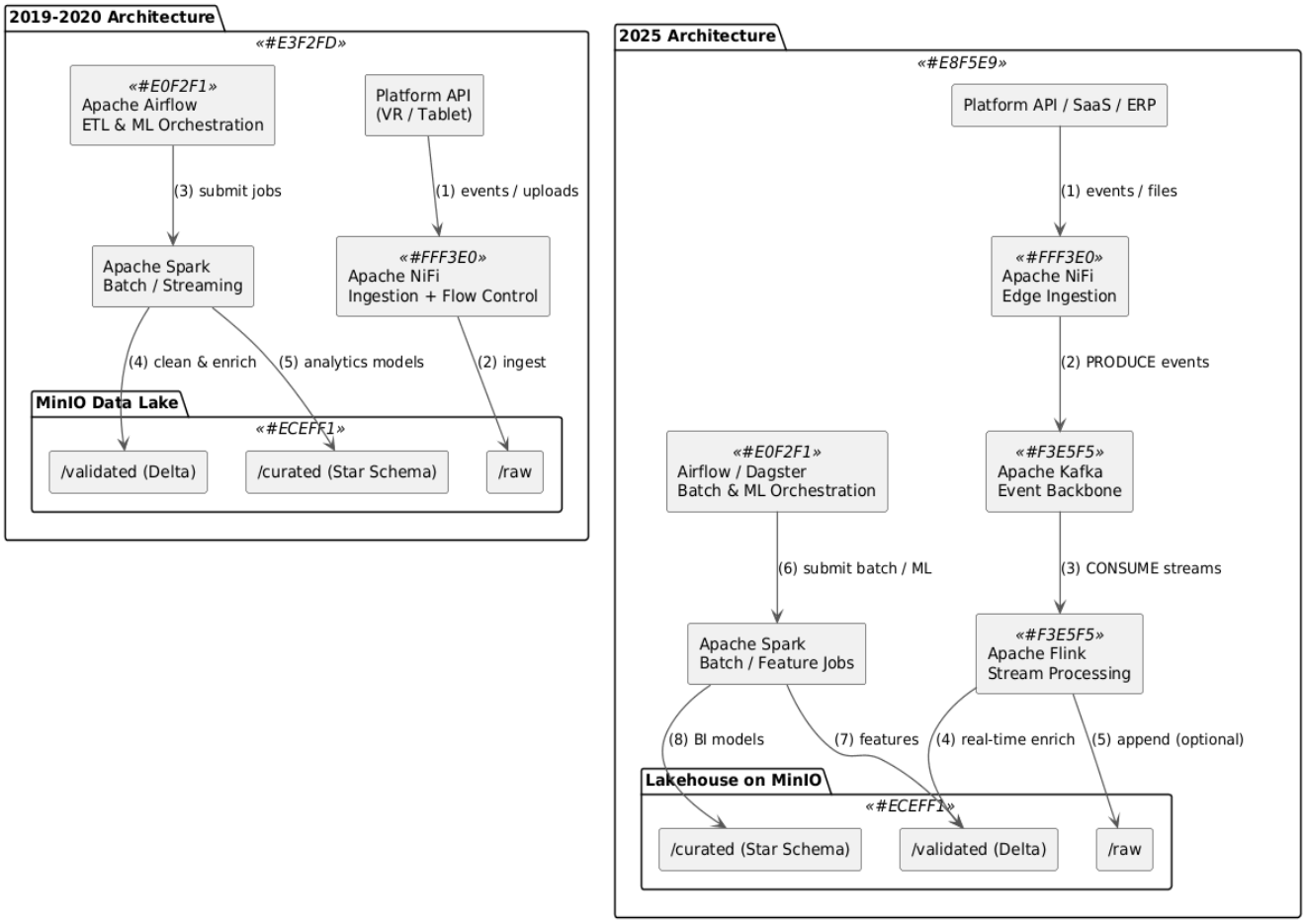
Aspect	**2019**	**2025 Evolution**
Ingestion	NiFi (Bulk + Micro-batch)	NiFi (Bulk) + Flink (Streaming / Micro-batch)
Streaming	N/A	Flink handles real-time event processing
Batch / ML	Spark (Batch + Micro-batch)	Spark on EKS (Batch + ML), Flink handles micro-batch
Orchestration	Airflow	Dagster or MWAA (batch + streaming)
Storage	MinIO + Delta Lake	S3 / MinIO + Delta Lake + Iceberg

Aspect	2019	2025 Evolution
Analytics / BI	Power BI (Import / DirectQuery)	Power BI near real-time dashboards
ML / Personalization	Containerized ML microservices	Online ML predictions, personalized learning

This evolution enabled **real-time analytics, online ML-driven content recommendations, and flexible orchestration**, while maintaining backward compatibility for historical bulk data ingestion.

High level Architecture Diagrams

The following diagram shows involved actors for both 2019 and 2025. What is changing in 2025 is the architecture for continuous ingestion.



ADR (Architecture Decision Record) – Micro-batch → Flink

Title: Migration from Spark micro-batch to Flink for real-time streaming

Context:

- 2019 architecture used Spark micro-batch to handle near-real-time events.

- Spark micro-batch was sufficient at small scale but lacked native streaming semantics, event-time handling, and low-latency processing.

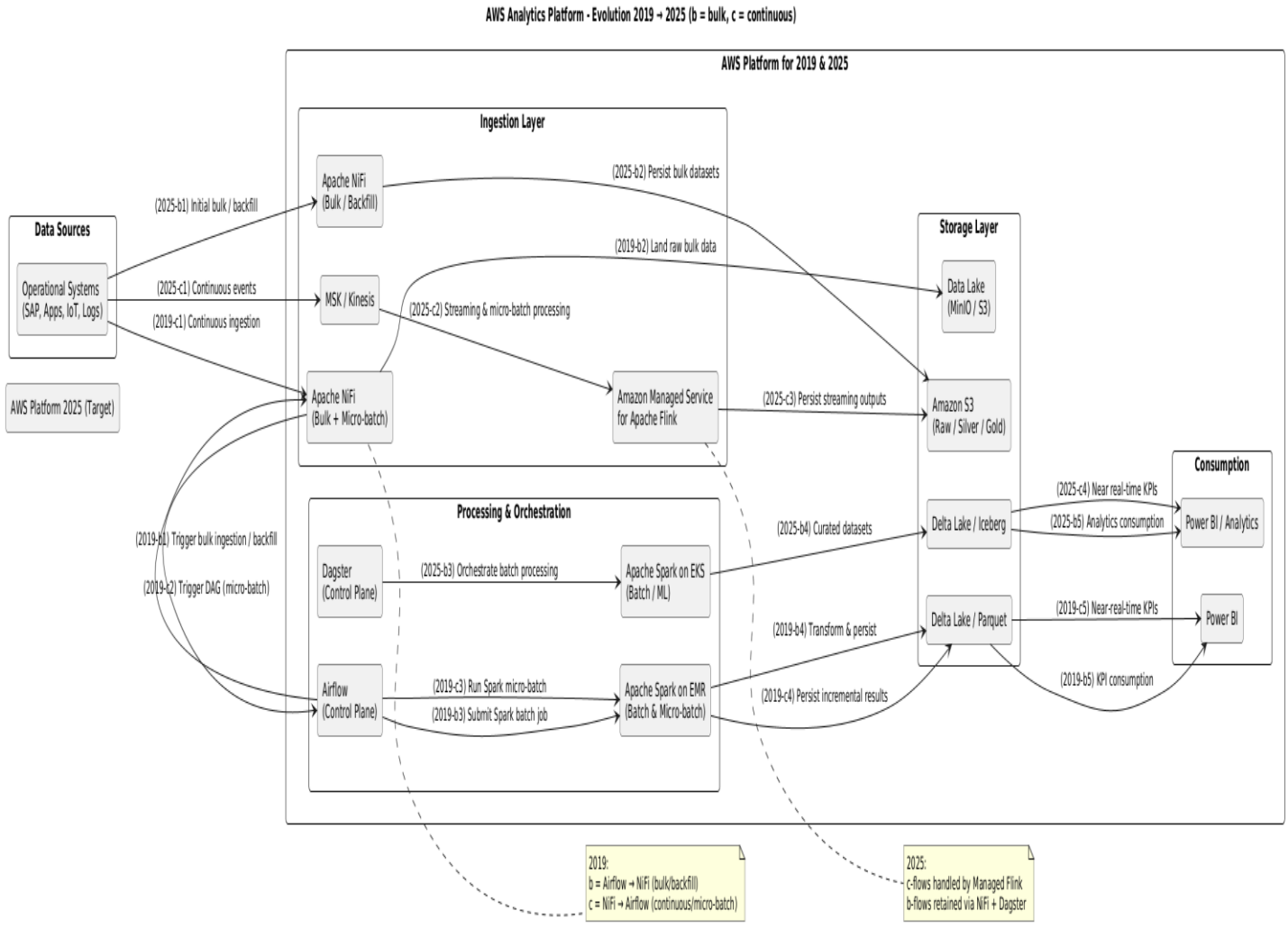
Decision:

- Introduce **Apache Flink** in 2025 for streaming ingestion.
- Retain Spark for **batch & ML**, and NiFi for **bulk ingestion**.

Consequences:

- Micro-batch processing is either migrated to Flink (true streaming) or implemented as high-frequency batch in Spark.
- Better event-time handling, stateful processing, and exactly-once guarantees.
- Clear separation of responsibilities improves scalability, maintainability, and cost efficiency.

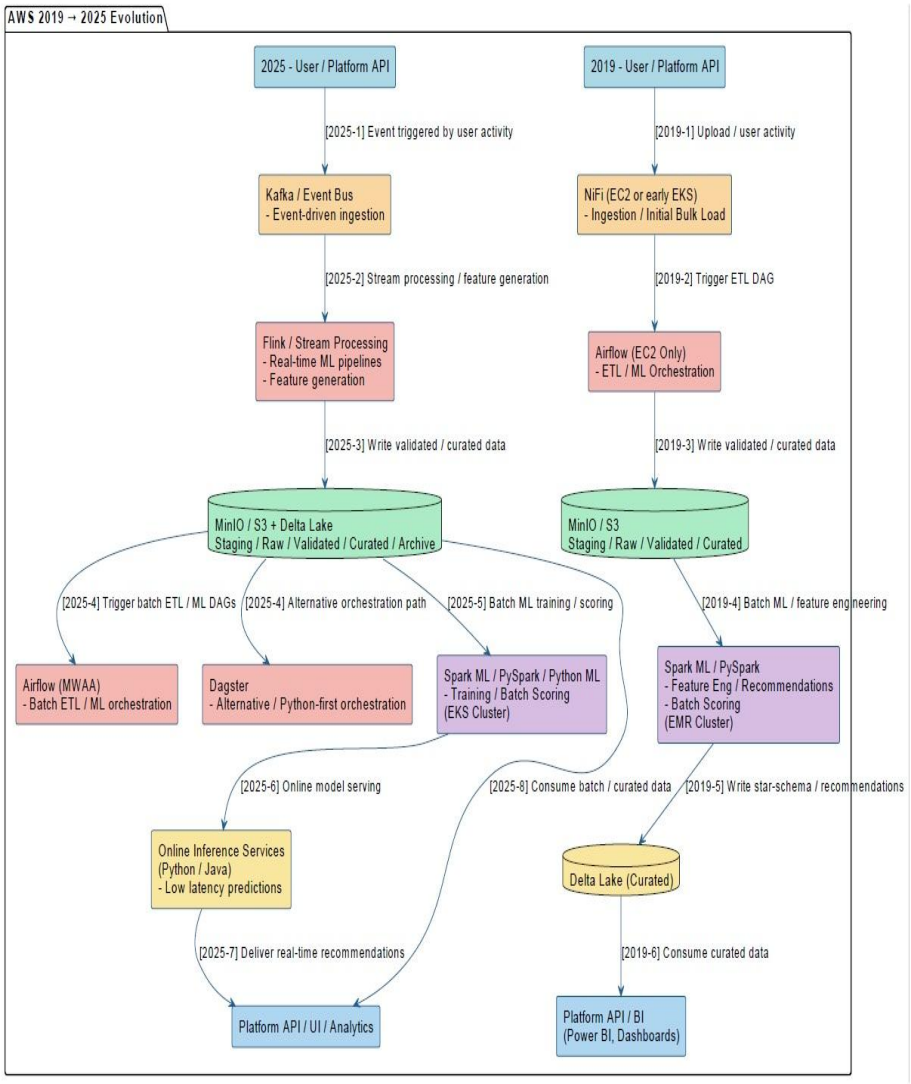
The following 2 diagrams describe the architecture supporting Amazon Cloud deployment. In the first diagram, both 2019 and 2025 approaches are shown for the initial bulk phase and continuous ingestion.



Architecture & Design Description

In 2019, Apache NiFi orchestrated bulk and micro-batch ingestion by triggering Airflow DAGs that executed Spark jobs (on EMR for Amazon cloud & on Kubernets for on-prem or hybrid deployment scenarios) providing near-real-time analytics latency suitable for reporting workloads, while event-driven streaming and message brokers were introduced later as part of the 2025 modernization.

In the second diagram, we illustrate the evolution between 2019 and 2025 approaches showing the changes made for continuous & micro-batch ingestion.

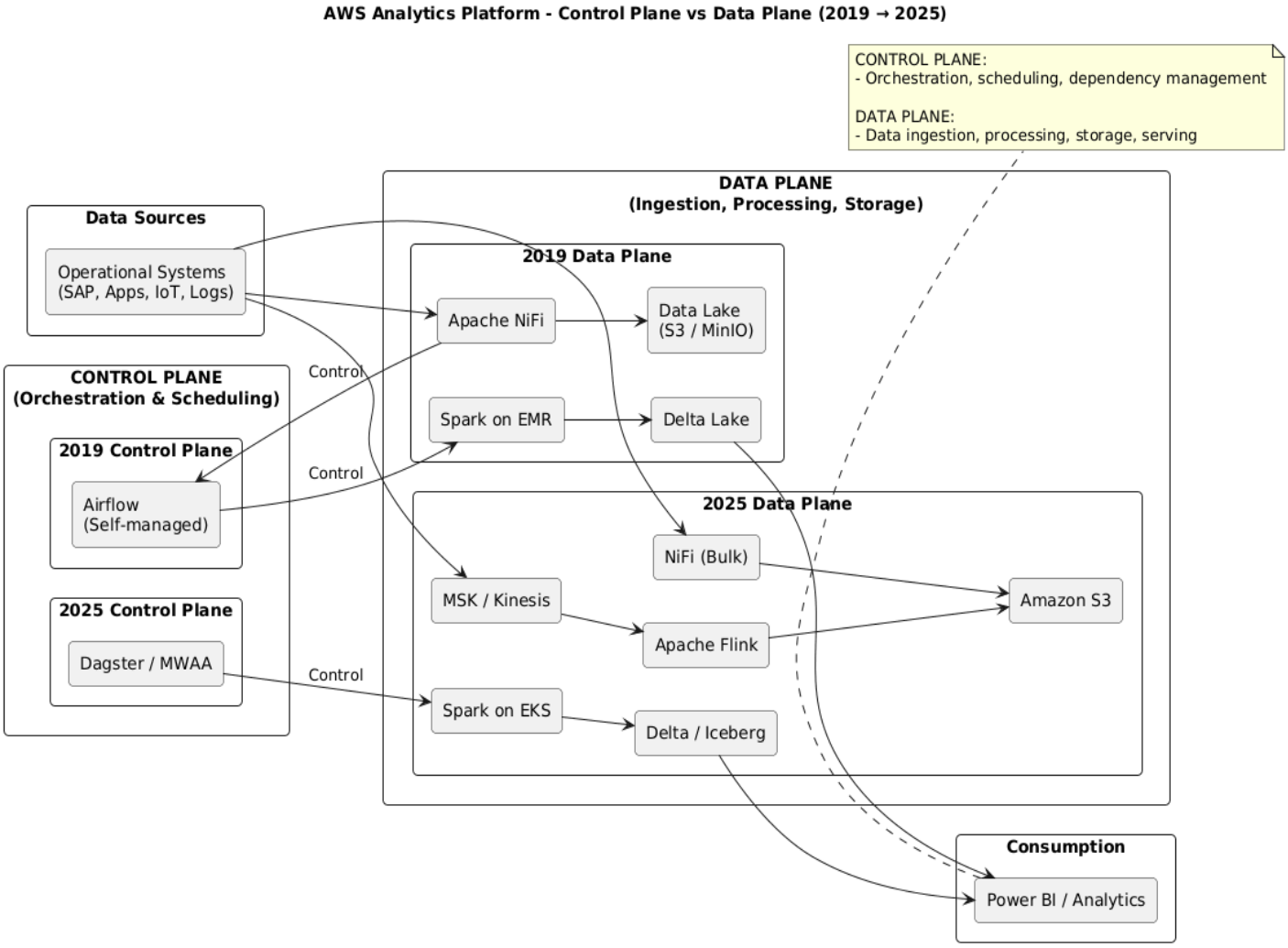


In the 2019 architecture, Apache NiFi handled both bulk and micro-batch ingestion while Spark on EMR performed batch and micro-batch analytics orchestrated via self-managed Airflow. By 2025, the platform evolves to a dual-ingestion approach: NiFi for initial bulk loads and Apache Flink for continuous streaming ingestion. Spark on EKS is retained for batch and ML workloads, with orchestration via MWAA or Dagster. This evolution enables clear

Architecture & Design Description

separation of batch, streaming, and ML processing, optimizes scalability, and supports near real-time KPI generation via Power B.

Control Plan vs Data Plan diagram 2019 vs 2025: This version is architecturally expressive and excellent for senior reviewers, ADRs, and executives.



2019 Architecture – Bulk ingestion

- Airflow is the control plane
- Airflow:

1. Triggers NiFi (via REST)
2. Orchestrates Spark jobs

- NiFi is not autonomous

✔ Airflow → NiFi → Spark

Architecture & Design Description

2019 Architecture – Continuous & micro-batch ingestion

- NiFi:
    - Listens to incoming data
    - Detects new files / events
    - **Triggers Airflow DAGs** when needed
- Airflow still orchestrates Spark

✔ **NiFi → Airflow → Spark**

In the 2019 architecture, Apache NiFi was deployed on EC2 instances (or Kubernetes for on-prem or hybrid scenarios), aligned with the VM-centric operational model of the time.

- NiFi = ingestion (EC2 or Kubernetes for on-prem or hybrid deployment)
- Airflow = orchestration (control plane) / self managed on EC2 for Amazon deployment
- Spark = processing
- ❌ No broker → no event-driven semantics
- ✓ Matches how EMR + Airflow was actually used

2025 Architecture- – Continuous & micro-batch ingestion

In the 2025 target architecture, NiFi is containerized and deployed on Amazon EKS (or Kubernetes for on-prem or hybrid scenarios) alongside Apache Flink and Spark, enabling a fully cloud-native ingestion and processing platform with improved scalability, resiliency, and operational consistency.

- NiFi on Amazon EKS
- Kafka/Amazon MSK introduced for data-plane decoupling
- Flink/Amazon MSAF (Amazon Managed Service for Apache Flink) owns streaming + micro-batch
- Spark focuses on batch & ML
- Dagster or Amazon MWAA (Managed Workflow for Apache Airflow) improves orchestration semantics

The Evolution between 2019-2025 for Continuous & micro-batch ingestion

2019

- NiFi → Airflow → Spark (EMR)

2025

- **NiFi (EKS)** → S3 (bulk)
- **MSK/Kinesis** → **Managed Flink** → S3 (streaming)
- **Dagster or MWAA (Managed Workflow Apache Airflow)** → Spark on EKS (batch & ML)

Architecture & Design Description

In the 2025 target architecture, real-time streaming and micro-batch workloads are handled by Amazon Managed Service for Apache Flink, eliminating the need to operate Flink on Kubernetes. Batch analytics and ML pipelines are executed using Spark on Amazon EKS, while Apache NiFi is containerized and deployed on EKS for bulk ingestion and backfill scenarios.

The evolution from Spark micro-batching (2019) to managed Flink (2025) reflects a shift from time-based batch processing to event-time streaming, enabling near real-time analytics, online feature computation, and improved data freshness guarantees.

ML Features added in 2025

Initial ML implementations relied on batch-derived features embedded in ETL pipelines; later evolved toward explicit feature management to support real-time and scalable ML workloads. In 2019, ML features were produced as part of batch ETL pipelines and stored in curated datasets, without a dedicated feature store. Feature definitions were embedded in Spark jobs and reused by convention. By 2025, the architecture evolved to introduce explicit feature management capabilities to support real-time inference, feature reuse, and ML automation.

Feature Management Evolution (2019 → 2025)

In **2019**, machine learning features were **implicitly managed** as part of batch data processing pipelines. Feature computation was embedded directly within **Spark ETL jobs**, producing engineered feature columns stored in curated datasets (Delta/Parquet). These features were primarily consumed during **offline model training** and analytics workloads. Feature definitions, versioning, and reuse were governed by **convention and documentation**, rather than by a dedicated platform capability. This approach was well suited for batch-oriented ML use cases but introduced limitations around feature reuse, consistency between training and inference, and real-time availability.

In **2019**, the architecture **did not include**:

- A dedicated feature store (Feast Feature Store, SageMaker Feature Store, Databricks FS, etc.)
- Online/offline feature synchronization
- Feature versioning or discovery as first-class concepts

These capabilities were **not mature or widely adopted** at that time, especially in AWS-managed form.

In 2019, **features existed implicitly**, embedded inside:

Spark transformation outputs

- Aggregated datasets in **Delta / Parquet**
- Feature-like columns such as:
    - avg_session_duration_7d
    - content_completion_rate
    - maintenance_report_consulation_rate

Batch-oriented feature preparation

- Features were:
    - Computed **during ETL**
    - Stored in curated (Gold) tables
    - Recomputed for each training job

Training-time only features

- Features used:
    - For **offline model training**
    - Not reused consistently for real-time inference
- Risk of **training–serving skew** existed

Important distinction: Features were data products, not managed assets

What changed by 2025

By 2025, requirements could evolve:

- Real-time personalization
- Streaming ML features
- Online inference
- Multiple ML consumers
- Strong governance

This **forced** the introduction of:

- Explicit feature metadata
- Feature reuse
- Online + offline feature parity
- Automation

Hence the **Feature Store appears in 2025**, not in 2019

By **2025**, the architecture evolved to support **explicit feature management** as a first-class capability. Feature generation was decoupled from core ETL pipelines and aligned with **streaming and near real-time processing** (Apache Flink) as well as batch processing (Spark on EKS). Feature metadata—including ownership, freshness, lineage, and usage—became centrally managed to ensure **semantic consistency** across ML pipelines.

This evolution enabled **online and offline feature parity**, reduced training–serving skew, and supported advanced use cases such as real-time personalization, recommendation systems, and predictive insights. The introduction of

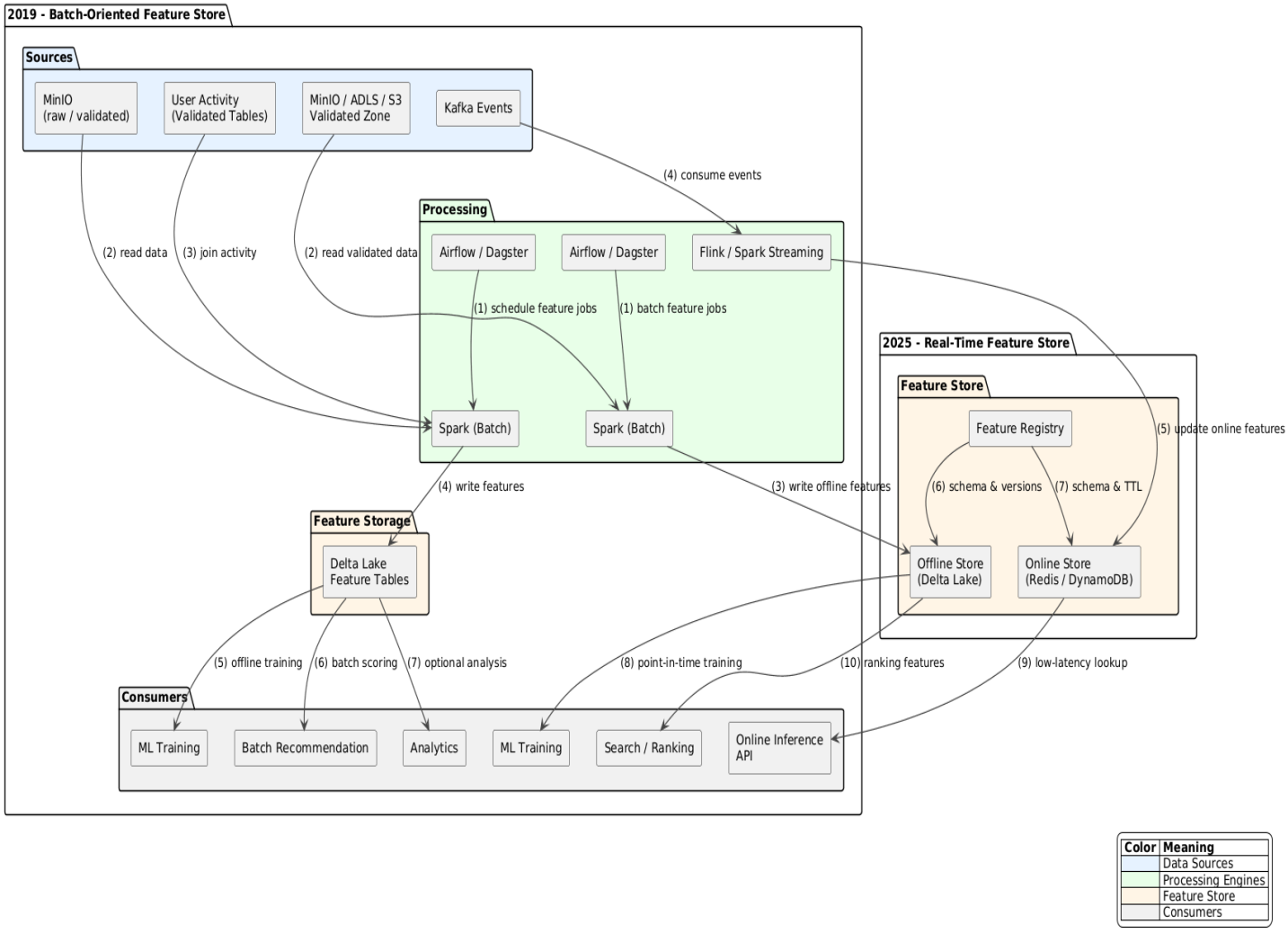
structured feature management significantly improves ML automation, scalability, and governance across the platform.

Feature Store – 2025 Architecture: Addition of Online Feature Store & Feature Registry

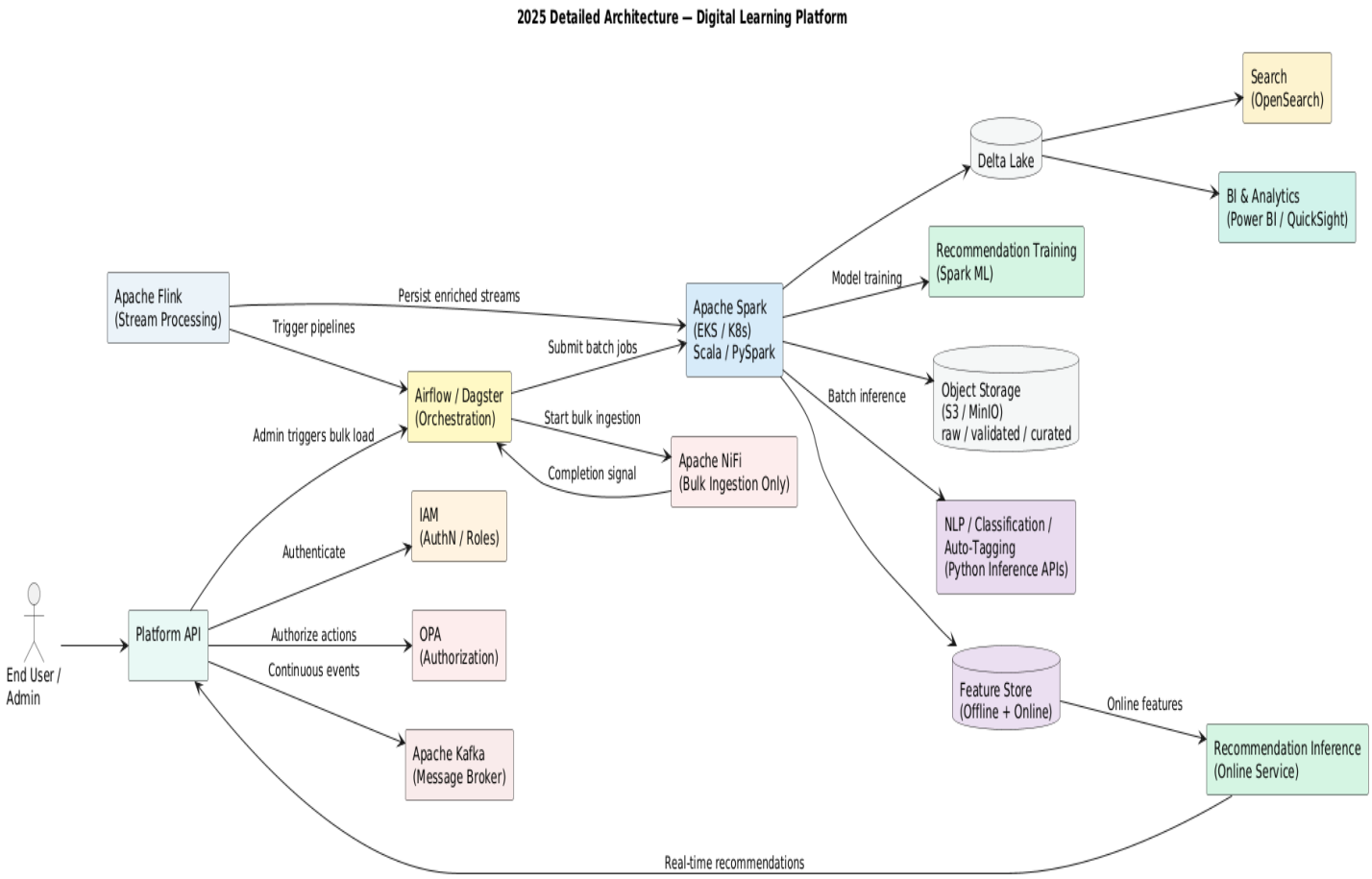
In the target architecture, a Feature Store is introduced as a first-class component to manage machine learning features across batch and streaming pipelines. Engineered features produced by **Apache Flink (streaming)** and **Apache Spark on EKS (batch/ML)** are registered with centralized feature metadata, ensuring **semantic consistency, versioning, lineage, and freshness guarantees**. The Feature Store enables **online feature serving** for real-time inference.

Feature Store – 2025 Architecture Diagram:

The following diagram describes the 2025 feature store architecture within the data flow. It illustrates the 2019 architecture, which relied on a batch-oriented Feature Store, and the addition of a real-time Feature Store in 2025.



2025 Detailed Architecture Diagram



In 2019, ingestion relied on NiFi for both batch and continuous flows, with Spark handling batch analytics and ML. By 2025, continuous ingestion evolved to Kafka and Flink to support high-throughput, low-latency streams, while NiFi was retained for bulk ingestion. Spark remains the core batch and ML engine, but a feature store and real-time inference services were introduced to enable online recommendations.

Capability	**2019**	**2025**
Bulk ingestion	NiFi	NiFi (unchanged)
Continuous ingestion	NiFi	**Kafka + Flink**
Orchestration	Airflow	Airflow / Dagster
Processing	Spark (batch + ML)	Spark (batch + ML)
Feature Store	❌	✓ **Introduced (Feast, Redis / DynamoDB)**
Recommendation	Batch only	**Training + Online inference**

Architecture & Design Description

2019 Architecture (Offline Feature Store)

Users:

- Admin: initiates bulk load of data and scheduled batch predictions.
- Field employee: receives recommendations from offline predictions.

Predictions:

- Offline only: all predictions are precomputed for employees in batch jobs.
- Stored in Delta Lake fact & dimension tables.

Fact & Dimension Tables:

- Fact table could store: predicted recommendations per employee.
- Dimensions could include:
  - Employee profile (job type, title, project assignment)
  - Material metadata (documents, media types, field tasks)
  - Manager requests or assignment context

Example: An electrician sees docs/media/materials relevant to field tasks he's allowed to access.

Search / Access:

- When the employee searches, it queries the precomputed recommendations in the Delta Lake tables.
- Search is limited to available, precomputed references, no real-time adaptation.

2025 Architecture (Offline & Online Feature Store)

Users:

- Admin: still initiates bulk load + manages continuous ingestion.
- Field employee: receives both offline recommendations (from precomputed batch) and online session predictions.

Predictions:

- Batch predictions: still precomputed using Spark ML on large datasets.
- Online / session predictions: computed per user request, using Python ML.

Feature Store / Delta Lake:

- We store user's session-specific features in an online feature store (e.g. user's last 10 searched incident report types)
  - Solution**:** Feast with Redis or DynamoDB for the online feature store.
- We store the relevant material references in Delta Lake tables (start schema):
  - Fact table: session-specific prediction outputs or recommendations (per employee)
  - Dimension tables:
    - Employee profile,
    - Job type,
    - Manager requests,
    - Project assignments or assignment context
    - Material metadata

Online predictions query the feature store + Delta Lake to generate session-specific recommendations.

Architecture & Design Description

The online search uses these metadata + features for real-time inference.

Key point:

- The type of material a user should see is stored in Delta Lake (e.g. fact_user_material_preference), while the data related to the user's current session behavior (such as latest search queries) is stored in an online feature store (e.g. most_viewed_material_type in the existing session).
- Online session prediction does not compute all materials globally, only uses features relevant to the user's profile and current session context. Online features allow to understand the user's recent focus.

This enables personalized, low-latency recommendations for field employees. While the "last 10 searches" would live in the online store for immediate use, the entire history of all user searches would be stored as an offline feature (within Delta Lake) for analyzing long-term patterns.

Aspects	**2019**	**2025**
Predictions	Offline batch only	Batch + Online session predictions
Storage	Delta Lake: fact/dim tables for precomputed recommendations	Delta Lake + Feature Store: facts/dims + session-specific features
Search	Query precomputed recommendations	Query feature store + Delta Lake + session context
Personalization	Limited to batch results	Real-time per-user/session predictions
Data Used	Employee profile, job type, material, manager requests	Employee profile, job type, material, manager requests, session context

Continuous Ingestion Justification (Flink) — Next-Generation Platform

For the next-generation of the learning platform, the scope extends from field employees to office employees, covering a wide spectrum of learning materials: safety procedures, engineering reports, legal documents, multimedia content, and curated external resources.

Continuous ingestion must support:

1. Daily additions from internal departments and curated external content.
2. Sporadic bursts of large batches of material (e.g., engineering reports or multimedia files).
3. Real-time updates of features used by ML microservices for classification, auto-tagging, and recommendation engines.

The **need for near-real-time feature generation, exact processing semantics, and support for peak ingestion events** makes **Apache Flink the ideal choice**. Flink seamlessly integrates with Kafka to consume events, enrich them, and feed both the **Feature Store** and **ML inference services**, enabling actionable, low-latency recommendations for all users. By using Flink, the platform is **future-proof**: as more content and users are added, the system can scale without redesign, ensuring continuous ingestion remains reliable, consistent, and low-latency. NiFi remains in place for bulk ingestion during initial platform launches or major departmental content uploads.

How this process works

1. **Repository / Drop Folder**:
    - A shared location (S3 bucket, MinIO folder, network drive) where content is uploaded.

- Could be structured by department, category, or content type.

2. **Admin triggers continuous ingestion**:
    - Instead of manually uploading each file or scheduling fixed daily batches, the admin **selects a folder** (or multiple folders) for ingestion.
    - The ingestion engine monitors the repository for **new files or subfolders**.
3. **Flink handles ingestion automatically**:
    - Flink continuously watches the repository (or consumes events via Kafka that report new files).
    - Each new file triggers:
        - Validation / preprocessing
        - Feature extraction
        - ML enrichment (classification, auto-tagging)
        - Writing to **validated / curated zones** and **Feature Store**
4. **Advantages**:
    - No need to guess or schedule daily batches
    - Can handle **sporadic uploads of unknown volume**
    - Supports **incremental processing**: only new files are processed
    - Scales naturally with future growth

Option	**How it works**	**Notes**
Flink + File/Event Watch	Flink job monitors repository or consumes Kafka events that indicate new files	Fully real-time, can handle spikes, incremental processing
Flink + Periodic Scan	Flink triggers a scan every N minutes for new files in the drop folder	Slightly higher latency (minutes), simpler to implement
Airflow / Dagster Trigger	Admin triggers DAG that scans folder and sends files to Flink	Useful if some validation or approval is needed before ingestion

Business workflow example

1. Morning: Admin sets /new_uploads/Engineering as continuous ingestion folder.
2. Flink starts monitoring folder or consumes Kafka events from the repository.
3. Afternoon: Admin adds /new_uploads/Legal and /new_uploads/HR.
4. Flink **automatically processes the new files** and updates Feature Store / ML pipelines.

5. Users querying the platform see **updated recommendations and searchable materials** without waiting for daily batch jobs.

This is effectively a **drop-folder-based continuous ingestion**, which is much more flexible than fixed batch scheduling.

Platform Event Flow for 2025 Architecture

Case A — Bulk ingestion

1. **Admin** triggers bulk load → **Airflow/Dagster** starts **NiFi**
2. **NiFi** loads data to staging/raw zones
3. **Airflow/Dagster** triggers **Spark** batch jobs to:
    - Transform data
    - Populate curated / validated zones
    - Run ML microservices for batch inference
    - Update Feature Store

Case B — Continuous ingestion

1. **Platform API / repository** publishes new file/event → **Kafka**
2. **Flink** consumes the **Kafka** events or monitors the repository.
    - Performs **stream enrichment**, basic validation, incremental feature computation.
    - Can **directly write enriched events / features to the Feature Store** for online usage.
    - Can optionally send **batches or mini-batches to Spark** if heavier transformations are needed.
3. **Airflow/Dagster** can be triggered periodically or on-demand to run **Spark** DAGs that perform:
    - ML microservice batch inference
    - Feature store batch update
    - Recommendation engine model retraining
4. **ML microservices** (NLP, classification, auto-tagging) can be invoked either:
    - Directly by Spark (batch)
    - Or by Flink streaming (real-time inference for online features)

Key difference for 2025:

- **Continuous ingestion:** Flink can **directly feed Spark or FS**, bypassing Airflow/Dagster for low-latency updates.
- **Bulk ingestion:** Always goes through Airflow/Dagster → NiFi →Spark → ML → Feature Store.

2025 Architecture Evolution: ML Platform, Fined Grain Security & Governance

Governance and Security Evolution (2019 → 2025)

2019 Architecture — Implicit and Tool-Centric Governance

In the 2019 architecture, data governance and security were primarily enforced at the processing and infrastructure levels. Access control was largely managed through IAM permissions, Spark job configurations, and application-level logic. Metadata existed in multiple places, including application schemas, star-schema definitions, and embedded pipeline logic, without a centralized metadata authority.

Key characteristics of the 2019 governance model included:

- Coarse-grained access control (bucket-level or job-level)
- Governance embedded in ETL and application code
- Limited cross-engine consistency
- No unified dataset catalog
- No formal lineage or auditability across analytics and ML workloads

This approach was sufficient for a limited number of consumers and processing engines but did not scale to increased data sharing, regulatory requirements, or machine learning workloads.

2025 Architecture — Centralized, Fine-Grained Governance

The 2025 architecture introduces governance as a first-class, cross-cutting capability, decoupled from processing engines and application logic.

AWS Glue Data Catalog is introduced as a centralized metadata registry describing all curated and analytical datasets stored in the data lake, including Delta Lake tables used for analytics, feature engineering, and machine learning training. Glue serves as the authoritative reference for dataset definitions across all consuming engines.

AWS Lake Formation builds on top of the Glue Data Catalog to enforce fine-grained access control policies, including table-, column-, and row-level permissions, as well as dynamic data masking. These policies are applied consistently across Spark, Flink, Athena, and SageMaker, ensuring uniform security enforcement regardless of the execution engine.

Key improvements introduced in 2025 include:

- Centralized dataset discovery and metadata management
- Fine-grained, role-based access control across all engines
- Separation of governance concerns from ETL and application code
- Improved auditability and regulatory compliance
- Consistent governance across analytics and machine learning workloads

This evolution enables secure data sharing, supports multiple personas (data engineers, ML engineers, legal teams, field operations), and provides the foundation required for scalable machine learning and advanced analytics.

Architecture & Design Description

Aspect	2019	2025
Metadata	Distributed, implicit	Centralized (Glue Data Catalog)
Access Control	Coarse-grained	Fine-grained (Lake Formation)
Governance Location	Embedded in jobs	Centralized policy layer
Cross-engine Consistency	Limited	Full
ML Readiness	Low	High
Auditability	Minimal	Strong

Key architectural points (important)

- **Glue Data Catalog**
    - Registers *what datasets exist* (Delta tables, curated datasets, training datasets)
    - Is the single metadata reference for all engines
- **Lake Formation**
    - Enforces *who can access what*
    - Applies fine-grained security consistently across:
        - Spark
        - Flink
        - Athena
        - SageMaker
- **Nothing is duplicated**
    - Data stays in Delta Lake
    - Glue = metadata only
    - Lake Formation = policy only

In the 2025 AWS-based architecture, AWS Glue Data Catalog is introduced as a centralized metadata registry for curated and analytical datasets stored in the data lake, including Delta Lake tables used for analytics and machine learning. AWS Lake Formation builds on top of the Glue Data Catalog to enforce fine-grained access control and governance policies across all consuming engines (Spark, Flink, Athena, and SageMaker), without replacing existing ETL pipelines, feature stores, or domain-specific metadata.

Architecture & Design Description

Data Access conceptual governance flow

S3 / Delta Lake

↓

Glue Data Catalog (WHAT exists)

↓

Lake Formation (WHO can access)

↓

Spark / Flink / Athena / SageMaker

↓

Feature Store / BI / ML APIs

Why Glue Data Catalog & Lake Formation should exist in our 2025 AWS architecture

- Our platform **already works without Glue and Lake Formation**
- They are introduced to add:
    - Centralized metadata governance
    - Fine-grained security
    - Cross-engine consistency (Spark, Flink, Athena, SageMaker)
    - Auditability

Roles:

- **Glue Data Catalog** = centralized technical metadata registry
- **Lake Formation** = centralized access control & governance layer

In our architecture, what gets registered in Glue: Tables that SHOULD be in Glue Data Catalog

These are **data lake–level datasets**, shared across engines:

- **Curated Delta Lake tables**
    - Example: s3://hq-data/curated/documents_enriched/
    - Registered as
        - database: curated
        - table: documents_enriched
        - format: delta

  Glue stores: Schema, Location, Format, Partitions

- **Analytical / star-schema tables**
    - Examples: fact_documents, dim_assets, dim_sites

Registered so that: Athena, BI tools, ML pipelines & Auditors all see the **same definition**.

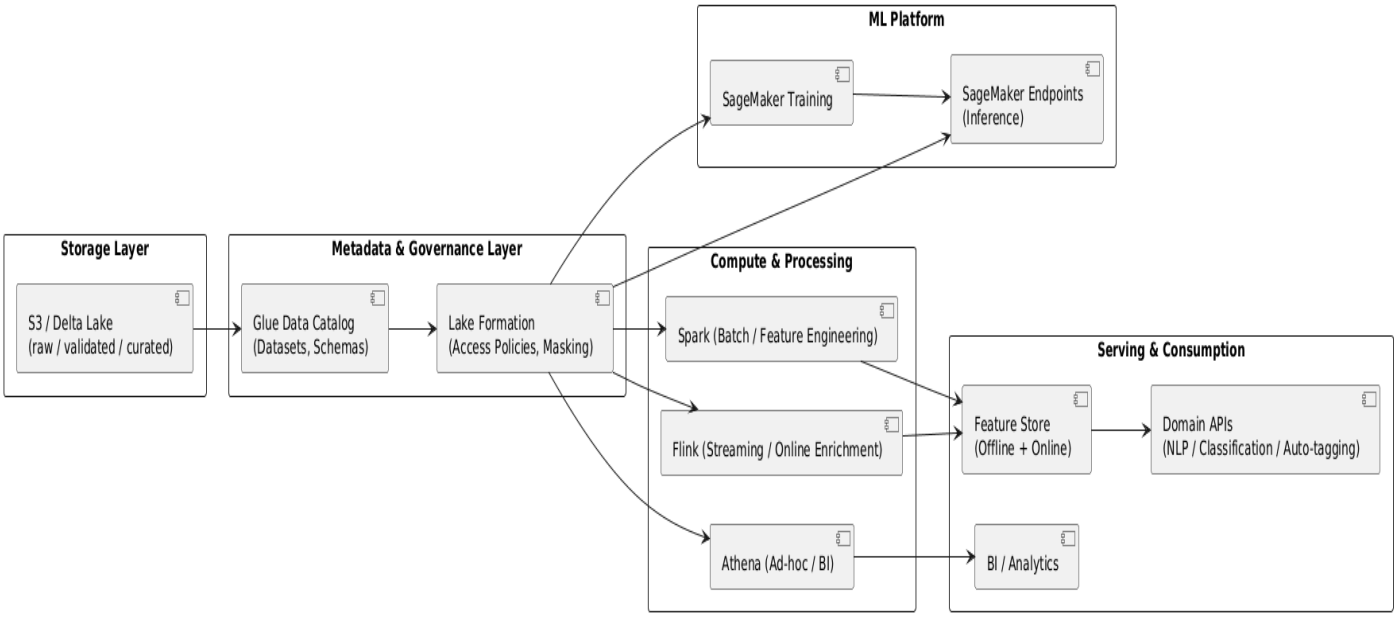
- **Training datasets (important for ML Platform like Amazon SageMaker)**
    - Example: curated. training_nlp_v3

This allows: SageMaker training jobs Governance, Reproducibility & Lineage

Data Relationship Summary

Component	**Role**
Delta Lake	Stores data
Glue Data Catalog	Describes data
Lake Formation	Secures data
Feature Store	Serves ML features
MinIO	Object storage
Star Schema	Analytics model
SageMaker	ML execution

Below architecture diagram's describes the addition of governance layer through Amazons Glue Data Catalog & Lake Formation.



Next four sections cover briefly security, governance, ADR (architecture decision record) & details of integration of ML Platform (Amazon SageMaker) to our architecture.

1. Add Security / IAM layers (Amazon): Key security principles

Across 2019 → 2025, what changed is not security existence, but security granularity.

Core principles

- Identity-first access (IAM over network trust)
- Least privilege
- Separation of duties
- Data + ML artifacts secured independently

Security & IAM (2025 focus) – Amazon deployment scenario

Security layers

- IAM Roles
    - Spark jobs (IRSA)
        - IRSA: In Amazon EKS for Spark jobs, IRSA (IAM Roles for Service Accounts) is a security feature that lets Kubernetes Service Accounts (SAs) directly assume specific AWS IAM roles, granting fine-grained access to AWS services (like S3, DynamoDB) without embedding AWS credentials in pods or using node IAM roles. It provides secure, temporary credentials, following the principle of least privilege by linking a pod's SA to an IAM role via OIDC, making Spark apps more secure & manageable on EKS
    - SageMaker training
    - SageMaker endpoints
- Lake access
    - S3 bucket policies
    - Table / column permissions via Glue + Lake Formation
- Online Feature Store
    - DynamoDB IAM policies
    - Redis (ElastiCache supporting IAM authentication)
- Network
    - Private VPC endpoints
    - No public S3 / SageMaker access
- Secrets
    - AWS Secrets Manager
- Audit
    - CloudTrail
    - Model access logs

2. Governance & Lineage — what changed from 2019 to 2025

2019 reality

- Metadata existed
- Governance was table-centric
- No ML lineage
- No feature visibility

2025 reality

- Governance expands to four artifact types: Amazon deployments

Artifact	**Governed by**
Data tables	Glue
Features	Feature Store + metadata
Models	Amazon SageMaker
Endpoints	ML platform

Key additions

  - Column-level security
  - Feature lineage (source → feature → model → endpoint)
  - Impact analysis (drop a feature → which models break?)

Conceptual Governance Flow

Raw Data

↓

Curated Tables

↓

Feature Definitions

↓

Feature Store

↓

Model Training

↓

Model Registry

↓

Online Endpoint

Governance follows the ML lifecycle, not just data ingestion.

3. Final Architecture Decision Record (ADR)

Below is a clean, reusable ADR we can paste directly into our architecture repository.

ADR-: Adoption of Feature Store–Centric ML Platform based Architecture (2019 → 2025)

Context: The 2019 analytics and ML platform relied on:

- Curated Delta Lake fact tables
- Spark batch jobs for feature engineering (feature extraction / feature concept is implicit) & recommendations
- Custom REST-based ML microservices (NLP, classification, auto-tagging)
- Limited ML governance and no online inference parity

As ML usage evolved toward:

- Real-time recommendations
- Online inference with SLAs
- Feature reuse across models
- Increased regulatory & audit requirements

The existing architecture no longer provided sufficient correctness, reproducibility, or operational guarantees.

Decision: We adopt a Feature Store–centric ML architecture with:

- Offline Feature Store implemented on Delta Lake
- Online Feature Store implemented on low-latency key-value storage
    - AWS: DynamoDB / Redis
- Managed ML platforms as control planes
    - AWS SageMaker
- Identity-first security
    - AWS IAM + IRSA
- Centralized governance & lineage
    - AWS Glue + Lake Formation

Facts

- Delta Lake already provided strong offline guarantees (ACID, time travel) but could not meet online inference latency requirements.
- Feature Stores formalize features as versioned, governed ML contracts, not BI artifacts.
- Managed ML platforms provide standardized training, deployment, & monitoring pipelines.
- Identity-centric security aligns with least-privilege & zero-trust principles.
- Governance tools extend lineage beyond data tables to features, models, and endpoints.

Consequences:

Positive

- Training/serving parity enforced
- Reduced feature duplication
- Improved auditability & compliance
- Scalable real-time inference
- Clear separation of responsibilities

Architecture & Design Description

Trade-offs

- Increased platform complexity
- Additional operational components (online store, registries)
- Stronger dependency on cloud-native services

Alternative Considered

Option	**Reason Rejected**
Delta-only Feature Store	Cannot meet online latency SLAs
Cache on top of Delta	Complex invalidation, no guarantees
Pure custom ML services	Poor governance & scalability
Hadoop/YARN-centric ML	Misaligned with cloud-native ML

Decision Outcome

This architecture supports both:

- 2019 batch-oriented workloads
- 2025 real-time, feature-driven ML use cases

& provides a sustainable foundation for future ML expansion.

The evolution from 2019 to 2025 is not a storage upgrade — it is a shift from data-centric analytics to feature-centric, governed, identity-secured ML platforms.

Executive Summary — ML Platform Evolution (2019 → 2025)

Why the Architecture Evolved

2019 reality

- Data platforms optimized for analytics & batch ML
- Features embedded implicitly in Delta fact tables
- ML models deployed as custom REST services
- Limited governance, no online inference parity

2025 reality

- ML systems are production systems with SLAs
- Real-time inference & personalization are mainstream
- Features must be reusable, versioned, auditable contracts (the adoption on of ML platform such as Amazon SageMaker, Azure ML as managed cloud services or Apache ML flow for cloud & hybrid deployment scenarios)
- Strong regulatory, security, & operational requirements

Architecture & Design Description

What Changed Architecturally

From:

- Data-centric pipelines
- Batch-first Spark ML
- Cluster-based security
- Table-level governance

To:

- Feature-centric ML platforms
- Offline + Online Feature Store
- Managed ML control planes (Amazon SageMaker)
- Identity-first security & end-to-end lineage

4. ML Platform Architecture Handout — 2019 → 2025 Evolution

Purpose: Provide a clear, business-aligned overview of how the ML platform evolved from batch, data-centric analytics (2019) to a feature-centric, governed, real-time ML platform (2025) on AWS.

2019 Baseline Architecture (Data-Centric)

Core characteristics

- Lakehouse built on Delta Lake
- Spark batch jobs for feature engineering and recommendations
- Custom REST ML microservices:
    - NLP
    - Classification
    - Auto-tagging
- ML primarily offline (training & batch scoring)
- Governance limited to tables, not ML artifacts

Limitations

- Features implicit (columns in fact tables)
- No training/serving parity
- Limited reuse across models
- No low-latency inference guarantees

2025 Target Architecture (Feature-Centric)

Core characteristics

- Features treated as first-class ML contracts

Architecture & Design Description

- Offline + Online Feature Store split
- Managed ML platforms act as ML control planes
- End-to-end lineage model lineage
- Supports real-time inference with SLA

Cloud Deployment Mapping (considering Amazon & Azure choices)

Capability	**Amazon**	**Azure**
Lakehouse	S3 + Delta	ADLS + Delta
Feature Engineering	Spark on EKS	Databricks / Spark on AKS
Offline Feature Store	Delta Lake	Delta Lake
Online Feature Store	DynamoDB / Redis	Redis / Cosmos DB
ML Control Plane	SageMaker	Azure ML
Governance	Glue + Lake Formation	Microsoft Purview
Identity & Security	IAM + IRSA	Azure AD + Managed Identity

Security & Governance Model

Security

- Identity-based access (IAM)
- Least privilege for:
  - Data
  - Features
  - Models
  - Endpoints
- Private networking & secret management

Governance

- Table, column, feature, and model lineage
- Feature → Model → Endpoint traceability
- Audit-ready architecture

Why Feature Store Was Not Present in 2019: Storage and compute capabilities already existed. Missing elements were

- ML operational maturity
- Online inference requirements
- Feature ownership & governance

Architecture & Design Description

Feature Store became necessary only when

- Models moved to production
- Predictions required SLAs
- Feature reuse became critical

Business Outcomes

- Consistent training & inference behavior
- Faster delivery of ML use cases
- Reduced feature duplication and errors
- Improved compliance and auditability
- Scalable real-time personalization & recommendations

This evolution is not a tooling upgrade, it is a shift from data-centric analytics to feature-centric, governed, production ML platforms. The 2025 architecture supports both legacy batch ML & modern real-time AI, providing a sustainable foundation for future growth.

2019 recap (what we had): In 2019, our architecture was Spark-centric:

- Spark batch jobs did everything
  - Feature engineering
  - NLP (NLP pipeline)
  - Classification
  - Auto-tagging
  - Recommendation engine
- Outputs were written to Delta fact tables
- REST APIs (used by our Microservices serving NLP pipeline) were often:
  - Thin wrappers
  - Or even absent (batch consumption)

This worked because

- Inference was mostly offline
- Latency was not a business constraint
- Models were not continuously retrained

2025 core principle (this is the key shift)

- Spark is no longer the inference runtime.
- Spark remains the feature & training engine.

In 2025

- Spark = data processing & model training
- AWS SageMaker / Azure ML / Apache MLflow = model lifecycle & inference

This is the most important mental model change.

What SageMaker as an ML Platform could actually replace (& what it does NOT)

An ML Platform (such as SageMaker) replaces:

Capability	**Before (2019)**	**After (2025)**
Model training orchestration	Spark jobs	SageMaker training jobs
Model registry	Ad-hoc / none	SageMaker Model Registry
Online inference runtime	Spark batch	Managed endpoints
Scaling inference	Manual	Auto-scaling endpoints
A/B testing, versions	Manual	Native

SageMaker does not replace:

Item	**Why**
Feature engineering	Still Spark's job
Business logic	Still needed
Application APIs	Still needed
Event-driven logic	Still needed
Some batch ML jobs	Still needed

What happens to our 3 ML REST microservices? Important distinction

We had two things mixed in 2019:

1. Model inference
2. Service orchestration / business logic

In 2025, these are explicitly separated.

2025 pattern for NLP / Classification / Auto-Tagging REST Microservices

Before (2019)

Spark Job

├ Feature extraction

├ Model inference

├ Business rules

└ Write results

Architecture & Design Description

After (2025)

API / Service

├ Fetch features (online store)

├ Call SageMaker endpoint: ml computing happens at this level

├ Apply business rules

└ Return response

The REST services (NLP / Classification / Auto-Tagging) still exist, but:

- They are lighter
- They do not run Spark
- They do not host models
- They orchestrate inference, not compute it.

What about the Recommendation Engine? This one is nuanced.

Option A — Offline recommendations (still Spark)

If recommendations are:

- Nightly / hourly
- Large-scale batch
- Written to tables

Spark batch jobs remain valid

Option B — Online / real-time recommendations (2025 target)

If recommendations are:

- User-driven
- Context-aware
- Low-latency

Flow becomes

Client

→ Recommendation API

→ Online Feature Store

→ SageMaker endpoint

→ Ranked recommendations

Spark is no longer in the request path.

So, do we still need the recommendation engine? Yes, but its **form changes**.

2019	**2025**
Spark job	Trained model
Embedded logic	Encapsulated model
Batch output	Endpoint response
Table-based	Feature-based

We no longer maintain:

- Spark inference logic
- Spark serving clusters

We still maintain:

- Recommendation models
- Recommendation APIs
- Recommendation features

And SageMaker becomes our ML execution plane.

We still need:

- APIs (for NLP, classification, tagging, recommendations)
- Event pipelines
- Feature pipelines
- Governance & security layers

What disappears is:

- Spark as a serving engine
- Custom inference infrastructure

SageMaker does not eliminate our microservices or recommendation engine. It eliminates Spark-based inference and replaces it with managed model endpoints.

What stays on Spark vs moves to ML endpoints (decision table)

Keep on Spark (even on 2025)

Workload	**Why**
Feature engineering	Heavy data transforms
Offline training	Large datasets
Batch scoring	Cost-efficient

Architecture & Design Description

Workload	Why
Historical backfills	Deterministic replay
Recommendation pre-compute	When latency is not required

Move to ML endpoints (SageMaker)

Workload	Why
NLP inference	Tokenization + low latency
Classification	Stateless, request-based
Auto-tagging	Online usage
Real-time recommendations	SLA-driven
Model experimentation	Versioned endpoints

Spark is not suitable for online inference

Spark

- Feature engineering
- Offline training

Kafka + Flink

- Feature freshness
- Stream aggregation
- Feature materialization

SageMaker

- Model training orchestration
- Model registry
- Online inference endpoints

Flink feeds features, not predictions.

2025 maturity means Spark for data, Flink for streams, and SageMaker for inference. And NLP / Classification / Auto-Tagging REST Microservices are not model training they are model inference services.

Clear separation: Training vs Inference

Model training: What it is

- Learning model parameters (weights)
- Computationally heavy

Architecture & Design Description

- Runs on historical datasets
- Not latency-sensitive

Typical characteristics

- Batch
- Hours or minutes
- Produces a model artifact

Runs on

- Spark (distributed training / feature prep)
- SageMaker Training Jobs (for Amazon cloud deployment) or Azure ML training pipelines (for Azure cloud deployment)

Model inference: What it is

- Applying an already-trained model
- Stateless or lightly stateful
- Latency-sensitive (often)

Typical characteristics

- Online (ms–100 ms)
- Or batch scoring
- Produces predictions

Runs on

- REST services
- SageMaker endpoints (for Amazon cloud deployment) ore Azure ML endpoints (for Azure cloud deployment)

(Historically) Spark jobs — but this is no longer recommended

Where NLP / Classification / Auto-Tagging fit. What these services actually are

Service	**What it does**	**Category**
NLP service	Text → embeddings / labels	**Inference**
Classification service	Feature vector → class	**Inference**
Auto-tagging	Content → tags	**Inference**

These services:

- Consume a trained model
- Do not update weights
- Do not learn

Architecture & Design Description

- Return predictions

They are inference services, not training.

In 2019, training & inference were collapsed into Spark jobs:

Spark Job

├ Feature extraction

├ Model training (sometimes)

├ Model inference

└ Write results

So conceptually:

- We ran a Spark job
- We got predictions

It felt like “the service is training”

But architecturally, these were different phases happening in the same runtime.

How this maps in 2025

Training Pipeline

Delta Lake

→ Spark (feature engineering)

→ SageMaker Training Job

→ Model Artifact

→ Model Registry

Inference Pipeline

Client / Event

→ NLP / Classification / Auto-Tagging API

→ Online Feature Store

→ SageMaker Endpoint

→ Prediction

The service:

- Does not train
- Does not store models
- Does not scale ML infrastructure

Architecture & Design Description

- Only orchestrates inference

Even if the same team owns both:

- Training runs in pipelines
- Inference runs in services

They are never the same thing in a mature architecture.

In 2019 we trained models — but training was embedded inside Spark batch jobs, not treated as a first-class lifecycle. There was no "training platform", no registry, and no separation between:

- feature engineering
- training
- inference

They all lived in the same execution context.

What "training" looked like in your 2019 architecture:

Typical 2019 Spark job pattern

Spark Job (NLP / Classification / Auto-Tagging / Recommendation)

├ Read historical data (Delta)

├ Feature extraction

├ Fit model (Spark ML / MLlib / custom Python)

├ Apply model to same data

└ Write predictions back to Delta

Key observations:

- fit() was called → that is training

Models were often: transient, overwritten & not versioned

Inference happened immediately after training. Outputs were stored, not models

Which of our 2019 workloads involved training

Based on our description:

Recommendation engine (Spark ML)

- Definitely trained
- Collaborative filtering / ranking models
- Trained on historical interactions
- Batch-scored

NLP service

- If using:
  - TF-IDF
  - Word2Vec
  - Topic models

  → Training occurred

Classification

- Logistic regression / tree / SVM (Support Vector Machine)

→ Training occurred

Auto-tagging

- If rule-based → ❌ no training

- If ML-based → ✓ yes training

Rule-based auto-tagging applies predefined logic and does not require training, whereas ML-based auto-tagging learns tagging patterns from historical data and therefore requires an explicit model training and lifecycle pipeline.

In most enterprise setups, at least 3 of the 4 involved training. Why it didn't feel like "model training"

Because in 2019:

Missing Element	**Impact**
Model registry	Models invisible
Training pipelines	No explicit lifecycle
Versioning	Overwrites
Validation gates	Ad-hoc
Deployment step	None

So:

- We ran a Spark job
- We got results
- We moved on

Training was just "part of the job".

The critical 2019 limitation (architectural): Training was a side-effect, not an intentional artifact. This caused:

- No reproducibility
- No rollback
- No auditability

Architecture & Design Description

- No training/serving parity

Which directly led to:

- Feature Stores in 2025
- Managed ML platforms
- Explicit separation of phases

How the same workload looks in 2025 (contrast)

2019

Spark job → predictions

2025

Spark → features

SageMaker → train model (happens as scheduled jobs periodically)

Registry → version

Endpoint → inference

We trained models in 2019, but we did not manage training — Spark did it implicitly for us. In our 2025 architecture without SageMaker, model training was still part of Spark jobs used by our microservices & recommendation engine. That architecture functionally worked, but it was architecturally transitional, not fully mature.

What actually happened in Our 2025 (Flink + Spark, without SageMaker)

Our architecture likely looked like this without the addition of an ML platform such as Amazon SageMaker for the model training lifecycle:

Offline / Batch side

- Spark jobs did:
  - Feature engineering
  - Model training (fit())
  - Batch scoring
- Models were:
  - Generated inside Spark
  - Stored in object storage
  - Overwritten or manually versioned

Online / Near-real-time side

- Kafka + Flink
  - Streaming feature aggregation
  - Online feature materialization

Microservices

- Called Spark outputs or Flink results
- Did not manage model lifecycle

Training still lived inside Spark jobs, exactly like 2019. In our 2025 architecture *without* SageMaker, model training is still part of Spark jobs used by our microservices and recommendation engine.

The difference vs 2019 was streaming features, not ML lifecycle maturity.

Aspect	**Status**
Training location	Spark jobs
Inference location	Spark / Flink
Model registry	❌
Version promotion	❌
Training–serving parity	Weak
Rollback	Manual

This means:

- Models were side-effects
- Features were tables, not contracts
- ML governance remained implicit

What SageMaker (as an ML Platform) changes

SageMaker removes training from Spark's responsibility. With SageMaker (or Azure ML)

Spark

→ Feature Engineering ONLY

SageMaker

→ Training

→ Model Registry

→ Versioning

→ Deployment

Endpoints

→ Inference

Spark no longer:

- Trains models
- Owns model artifacts
- Serves predictions

This is the architectural maturity leap.

Our 2025 Flink-based architecture without SageMaker still trained models inside Spark jobs — just like 2019 — and SageMaker's role is to extract training and inference from Spark into a managed ML lifecycle. SageMaker does not replace our NLP, Classification, Auto-Tagging REST microservices, nor our recommendation engine.

What SageMaker replaces is where models are trained & executed, not the services themselves.

In 2019 and in our 2025 (no-SageMaker) architecture:

Spark jobs contained everything:

- Feature logic
- Training logic
- Inference logic
- Business orchestration

So, it felt like:

- "The microservice is the ML system"

In 2025 with SageMaker:

- ML is extracted from those services
- Services become orchestrators, not ML runtimes

What SageMaker actually replaces? It replaces Spark-based ML execution

Before	**After**
Spark job trains model	SageMaker training job
Spark job runs inference	SageMaker endpoint
Spark cluster scales ML	Managed scaling
Manual versioning	Model Registry

Spark is no longer an ML runtime.

2025 “Before vs After” — exact comparison

Before (Spark / Flink inference)

NLP Service

└─ Spark job

  ├─ Feature extraction

  ├─ Model inference

  └─ Response

After (SageMaker inference)

NLP Service

 ├─ Fetch features (online store)

 ├─ Call SageMaker endpoint

 ├─ Apply business rules

 └─ Response

SageMaker becomes our ML execution & lifecycle layer.

Our services:

- Own API contracts
- Own orchestration
- Own business logic

SageMaker:

- Owns models
- Owns training
- Owns inference execution

SageMaker replaces Spark-based training and inference inside our ML services.

What we used to do (before SageMaker)

Inside your NLP / Classification / Recommendation code (Spark or service):

- Load model artifacts from storage
- Manage model version paths
- Keep model in memory
- Handle concurrency
- Scale workers
- Restart on failure
- Redeploy on new model
- Roll back manually

Architecture & Design Description

All of that logic was implicitly our responsibility. What SageMaker endpoint is doing instead

A SageMaker endpoint:

- Hosts one specific model version
- Exposes a stable HTTPS endpoint
- Scales automatically
- Handles concurrency & retries

Supports:

- Canary / A-B testing
- Rollback
- Monitoring

We did not lose logic — we externalized infrastructure & introduce the model training lifecycle pattern (initiating model training sessions periodically within Amazon SageMaker).

Before SageMaker, our stack looked like:

NLP Service

├Load model from S3

├ Deserialize model

├ Run inference

├ Manage memory & threads

└ Return result

SageMaker makes the model processing explicit and standardized.

Moves to SageMaker

- Model loading
- Model execution
- Runtime scaling
- Model version binding
- Inference infrastructure

Does NOT move

- Feature logic
- Business rules
- API contracts
- Domain logic

We do “Apply business rules: same code as before” but the ML part of it would be executed inside SageMaker as a packaged docker container exposing an endpoint interface through http or gRPC protocols to be called by our Microservices for ML computing jobs. SageMaker sits between feature fetch and business rules

Architecture & Design Description

SageMaker replaces the model execution & lifecycle we previously embedded inside our services, while our business rules & APIs remain exactly the same. “In 2025, our NLP or other microservices no longer runs ML code — it orchestrates features, inference, and business rules.” They orchestrate requests, features, & decisions; SageMaker owns training & prediction execution.

Microservice handles:

- Input validation
- Feature orchestration
- Business rules
- -Response formatting

SageMaker handles:

- Loading model into memory (coded & packaged as a docker container by us but managed by SageMaker at runtime)
- Executing ML math / inference (executing our own code which now leaves outside of our microservice & packaged as container exposing an endpoint interface)
- Auto-scaling & runtime management
- Model versioning / lifecycle

This makes responsibilities mutually exclusive and clear.

Observations: Without SageMaker, our service did everything ML-related, which:

- Coupled business logic & ML runtime
- Made scaling, rollback, & versioning harder

With SageMaker, our service:

- Only orchestrates
- Leaves ML math, scaling, & lifecycle management to a managed service

Can focus on business rules and orchestration

Example: What happened to our 2019 NLP pipeline with SageMaker?

Our 2019 NLP pipeline:

- Tokenization (spaCy)
- Language detection
- Lemmatization (NLTK / spaCy)
- Keyword extraction (TF-IDF- scikit-learn)
- Embeddings (Word2Vec / BERT)
- NER (spaCy / Flair)
- Classification
- Auto-tagging

These are ML logic, not business logic.

They are:

- Feature engineering
- Representation learning
- Model inference logic

They never disappear in 2025. They move location. Where do those steps live in 2025 with SageMaker:

Before (2019)

NLP pipeline (stared by NLP microservice execution)

├Tokenization

├ Language detection

├ Lemmatization

├Keyword extraction

├ Embeddings

├ NER

├ Classification, Auto-Tagging (Classification & Auto-tagging microservices)

└ Business rules

After (2025 with Amazon SageMaker)

NLP Microservice

├–Fetch features

├ Call SageMaker endpoint (executing a SageMaker nlp-model version)

└ Business rules

SageMaker NLP Model (nlp-model-v3)

├ Tokenization

├ Language detection

├ Lemmatization

├Keyword extraction (Classical ML with TF-IDF)

├ Embeddings (modern ML with BERT for semantic understanding)

├ NER

├ Classifier

└ Prediction

Architecture & Design Description

nlp-model-v3 contains our own machine learning code (packaged as a container) & being executed by SageMaker managed service runtime. All NLP pipeline steps now live inside the model artifact deployed in SageMaker.

So, what exactly is inside nlp-model-specified version? nlp-model typically contains our preprocessing code

- Tokenizer
- Lemmatizer
- Keyword extraction (not mandatory when embedding logic is included)

our embedding logic

- Word2Vec model or
- BERT encoder

our trained model

- NER model
- Classification head
- Tagging model

Inference wrapper

- Code that accepts JSON
- Runs preprocessing + model
- Returns predictions

Model version

- v1, v2, v3 = different training data, embeddings algorithms, or architectures

Who decides Word2Vec vs BERT? We do — at training time. Not at inference time.

Important distinction. SageMaker does NOT decide:

- Which embedding algorithm to use
- Which NLP pipeline to apply (do we use keyword extraction with TF-IDF when BERT embedding is used)

We decide:

- During model development & training
- When building the model artifact

Example decisions we make before deployment

- NLP approach:
  - TF-IDF (scikit-learn) + Logistic Regression for key word extraction & classification (classic ML)
    OR / AND
  - Word2Vec + CNN or BERT fine-tuned for embedding, NER & classification (modern ML)
- We train the model accordingly
- We package it
- We deploy it as nlp-model-v3

At inference time, SageMaker just executes what we trained. SageMaker is responsible for:

Architecture & Design Description

- Running our container
- Scaling it
- Exposing it as an endpoint
- Versioning & monitoring

The model container (designed & coded by us) is responsible for:

- Tokenization strategy
- Lemmatization & keyword extraction logic
- Embedding process
- NER architecture

SageMaker is a managed runtime for our ML pipelines.

In the 2025 architecture, ML NLP pipelines activities (tokenization, keyword extraction, lemmatization, embeddings, NER, and prediction) are encapsulated within versioned SageMaker model artifact. Application microservices no longer execute ML logic directly; instead, they orchestrate feature retrieval, invoke SageMaker inference endpoint, and apply domain-specific business rules.

nlp-model-v3 (contain our own code written in python within several services)

Client (NLP microservice)
|
▼
SageMaker Endpoint
|
▼
Container runtime
|
▼
Entrypoint handler
|
▼
inference.py
|
▼
Tokenizer.py → lemmatizer.py → extract-keyword.py → embeddings.py → NER.py → classifier.py
|
▼
Prediction JSON

Packaged as:

- Docker container (most common)
- Or SageMaker prebuilt container + our code (python services)
- Model artifacts stored in S3
- Deployed as a scalable endpoint

SageMaker 2025 mindset

NLP Microservice

├ build feature payload

├ call SageMaker endpoint (HTTP)

└ apply business rules

SageMaker Container

├ load NLP model

├ run NLP pipeline

└ return prediction

The nlp-model-v3 artifact encapsulates our custom NLP pipeline implementation (tokenization, language detection, lemmatization, keyword extraction, embeddings, NER, and inference logic) and is packaged as a SageMaker model packaged as a container. This container is deployed & executed in a managed SageMaker runtime environment, independent from application microservices, which invoke it remotely for inference. What belongs to **SageMaker** (ML responsibility).

When **NLP microservice call SageMaker endpoint for nlp-model-v** for a document, it receives some thing like this:

```
{
  "content_id": "123",
  "classification": "Safety",
  "confidence": 0.94,
  "tags": ["hydraulics", "maintenance", "safety"],
  "entities": [
    {"type": "EQUIPMENT", "value": "hydraulic pump"}
  ]
}
```

The classification value is transmitted to classification microservice.

Classification Microservice Input example

```
{
  "content_id": "123",
  "ml_prediction": {
    "classification": "Safety",
    "confidence": 0.94
  }
}
```

Classification Microservice apply only classification business logic.

Example:

```
if classification == "Safety" and confidence > 0.9:
    category = "Critical Safety Procedure"
elseif classification == "Safety":
    category = "General Safety Document"
else category = "Other"
```

Architecture & Design Description

Classification Microservice output would be something like

{
"documentCategory": "Critical Safety Procedure"
}

Then Auto-tagging Microservice is called with the tag values defined by **SageMaker nlp-model-v** where the received values are mapped to predefined taxonomy results.

Input tag values for Auto-tagging microservice (as Json): "tags": ["hydraulics", "maintenance", "safety"]

Example of taxonomy mapping business logic:

- Safety → Critical Safety Procedure
- Maintenance → Equipment Maintenance

After Auto-tagging microservice, Metadata microservice (for storing document related metadata data) & Search Index microservice (for indexing the document within Elasticsearch) would be executed.

The complete flow

Document ingestion

```
      |
      ▼
NLP Microservice
 (REST API)
      |
      ▼
Amazon SageMaker Endpoint
 (nlp-model-v3 container)
      |
      ▼
Predictions
  ├ entities
  ├ classification
  ├ tags
      |
      ▼
Business Microservices (REST APIs)
  ├ Classification Service (business logic)
  ├ Auto-Tagging Service (business logic)
  ├ Metadata Service (Metadata storage)
  └ Search Index Service (document indexing)
```

SageMaker is responsible for **everything related to the ML model itself**.

1. Model training

- Consume historical, labeled data
- Learn model parameters

- Produce a trained model artifact
- Register and version the model

2. Model hosting

- Load the trained model into memory
- Bind a specific model version to an endpoint
- Expose a stable inference endpoint
- Handle concurrency, retries, scaling, and failures

3. Model inference

- Accept feature inputs
- Execute the mathematical prediction
- Return raw prediction results (scores, probabilities, embeddings)

Important: SageMaker has no knowledge of business rules, users, tenants, policies, or workflows.

What belongs to our microservices (business responsibility): NLP, Classification, Auto-tagging

The NLP service is an **orchestration and decision layer**, not an ML runtime.

Step 1 — Request handling

- Receive (e.g. an NLP) request from an application or upstream system
- Validate input format and required fields
- Apply basic request-level controls (size, language, tenant)

Step 2 — Feature retrieval

- Identify which features are required for this model
- Fetch online features (for real-time requests) from the feature store
- Optionally fetch offline features (for batch or asynchronous workflows)

Step 3 — Invoke the SageMaker endpoint

- Send the prepared feature payload to the correct model endpoint
- Do not load or execute the model
- Treat the endpoint as an external ML dependency
- Receive raw prediction outputs

This is the **exact point where ML computation happens** — outside our service.

Step 4 — Apply business rules

- Interpret prediction results
- Apply confidence thresholds
- Enforce domain policies (e.g., tenant rules, compliance rules)
- Combine predictions with non-ML logic
- Decide final outcomes

This logic is **unchanged** compared to your 2019 or 2025 (no SageMaker) architecture.

Step 5 — Response construction

- Build a domain-friendly response
- Mask or enrich results as needed
- Return the response to the caller
- Optionally emit events or logs

The Microservices orchestrates requests, features, and decisions; SageMaker owns training and prediction execution.

Offline vs Online clarification

Online inference

- NLP service fetches online features
- Calls SageMaker endpoint
- Applies business rules
- Returns response immediately

Offline / batch inference

- Spark prepares feature datasets
- SageMaker runs batch inference **or**
- Spark performs batch inference using a registered model
- Business rules are applied downstream

SageMaker is **optional for offline inference**, but **preferred for online inference**.

Role of SageMaker in 2025 Architecture

- SageMaker takes over all ML responsibilities:
    - Model training

- Model versioning & registry
- Model hosting & scaling
- Online inference execution

Microservice Responsibility Changes

Microservice	**Pre-2025 (Spark embedded)**	**2025 (with SageMaker) / ML Responsibilities**
NLP Service	Load model, run inference, manage memory/scaling, apply business rules	Only orchestrate: fetch features, call SageMaker endpoint, apply business rules
Classification Service	Same	Same pattern as NLP Service
Auto-Tagging Service	Same	Same pattern
Recommendation Engine	Spark batch: train + inference	Microservice orchestrates features, calls SageMaker endpoint for inference

Clear boundaries between ML responsibility and business responsibility

Key Benefits of SageMaker Introduction

- Scalable inference without increasing microservice complexity
- Model versioning and rollback for safety
- Decouples ML computation from business logic
- Improves operational reliability and observability

NLP, Classification, and Auto-Tagging services are inference pipelines responsible for applying trained models to incoming data streams (online or batch) in order to enrich datasets and persist predictions to Delta Lake. Model training and evolution are decoupled and executed independently via SageMaker training workflows.

Training pipelines (SageMaker ML Platform Responsibility):

- Use large historical datasets
- Run periodically
- Produce **new model versions**
- Write:
  - Model artifacts
  - Metrics
  - Metadata

Architecture & Design Description

Training flow:

Delta Lake (historical data)

↓

SageMaker Training Job

↓

Model artifacts (S3)

↓

Model Registry

Inference pipelines:

- Use trained models
- Process new data
- Write:
    - Predictions
    - Enriched data
    - Features

These are **different lifecycles**, **different SLAs**, and **different failure modes**.

Mental model:

Ingestion pipeline: “Apply knowledge”

Training pipeline: “Create knowledge”

They may use the **same infrastructure**, but they are **not the same pipeline**. Training pipelines are often:

- Periodic
- Background
- Triggered by humans or governance

They’re usually drawn as **side pipeline.**

Document ingestion and enrichment pipelines apply trained models to data, while model training is a separate, intentional process that periodically consumes historical curated data to produce new model versions

Typical 2019 pattern model training:

Developer trains model locally / in a notebook

↓

Exports artifacts (vectors, weights, vocabularies)

↓

Packages them with application code

↓

Deploys Spark job / microservice

From an architecture point of view:

- Training was **outside the system**
- Not visible
- Not repeatable
- Not governed

So architecturally speaking: **Training did not exist as a pipeline.**

Why this was acceptable in 2019

- ML was “embedded intelligence”
- Scale was smaller
- Online learning was rare
- Model refresh cadence was slow
- Governance requirements were minimal

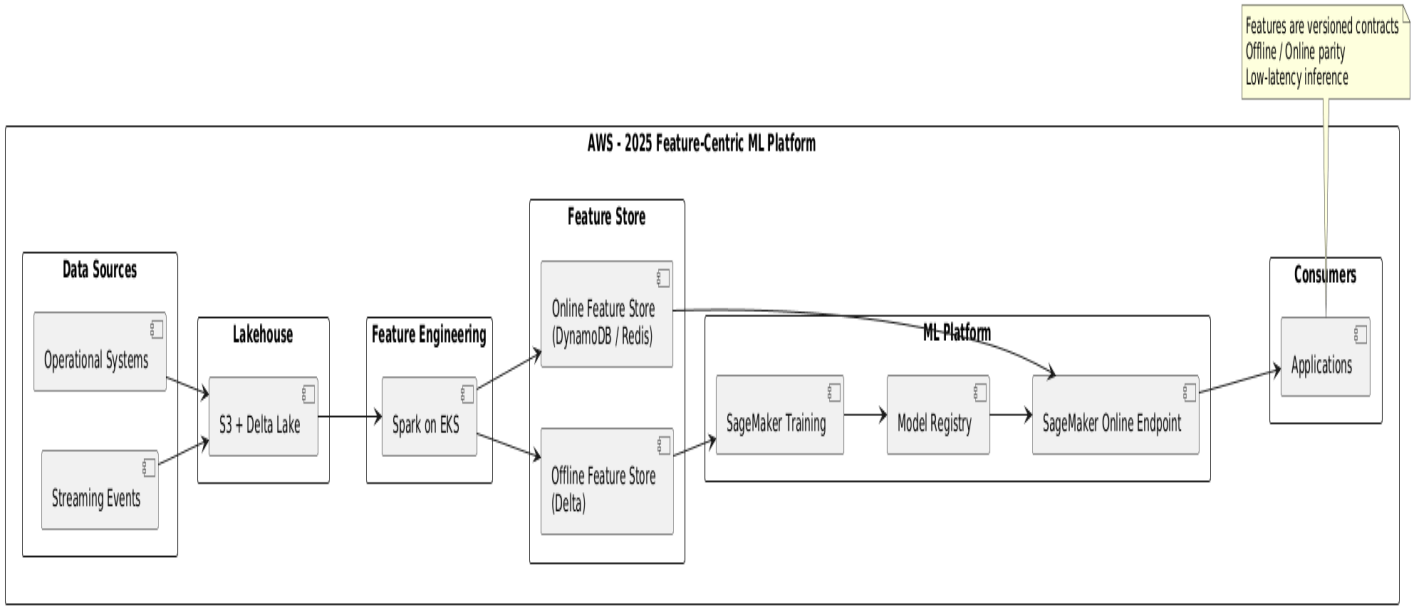
This was **normal** for that time. **What changed by 2025**

By 2025, training becomes:

✓ Explicit
✓ Automated
✓ Repeatable
✓ Auditable
✓ Governed

and therefore, **must appear in architecture**. In the 2019 architecture, model training was not implemented as an explicit pipeline. Machine learning logic was embedded within Spark jobs and microservices, with model artifacts packaged and deployed alongside application code. The 2025 architecture introduces model training as a first-class, decoupled capability.”

Architecture Diagram for the integration of Amazon SageMaker:



Example of business scenario: Field Operations — Better incident & maintenance recommendations

Scenario

Field technicians upload:

- Inspection reports
- Photos
- Maintenance logs
- Free-text comments

2019 (no training pipeline)

- NLP service extracts keywords
- Classification uses static TF-IDF / rules
- Recommendation engine uses generic similarity

Limitations:

- Cannot learn from actual outcomes
- Same issue described differently → poor matching
- Seasonal patterns not captured
- Recommendations remain generic

This looks similar to previous cases — but not smarter over time.

Architecture & Design Description

2025 (with training)

Training data:

- Past incident reports
- Actions taken
- Success / failure outcomes
- Time to resolution

Model learns:

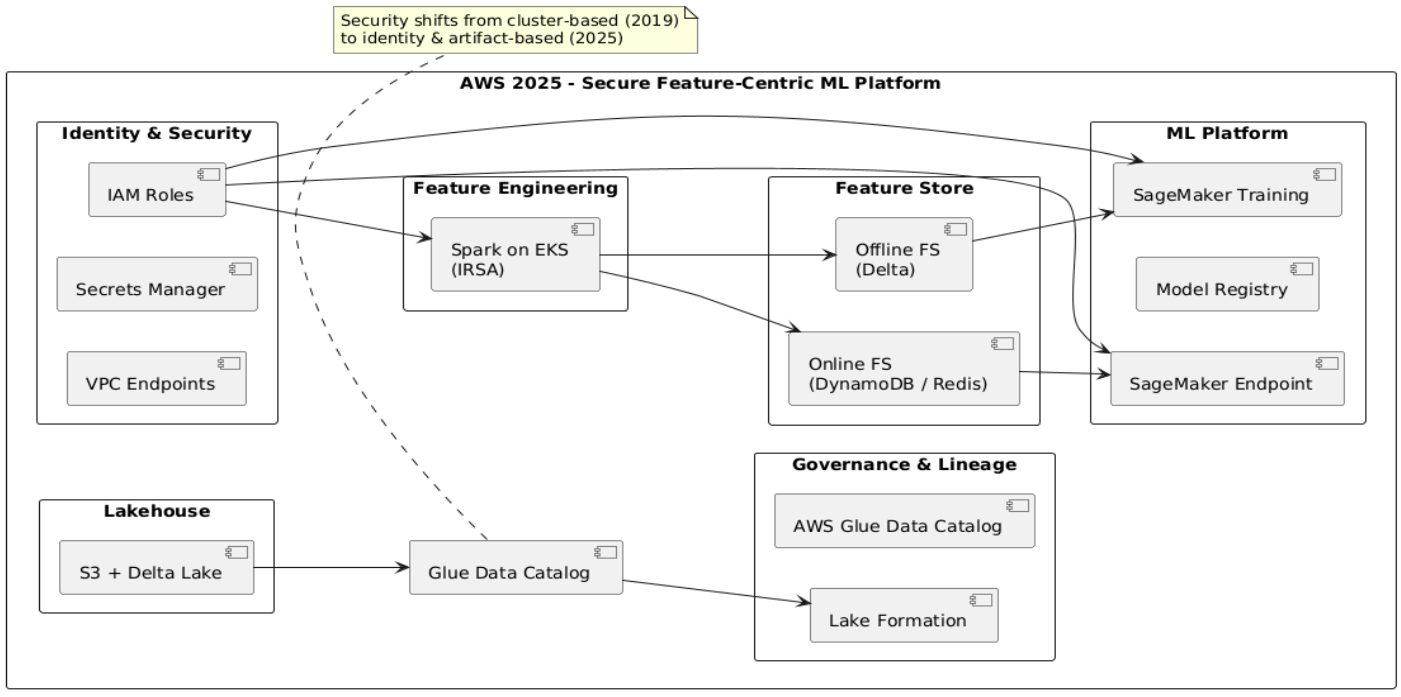
- Which textual patterns lead to failures
- Which fixes worked in which contexts
- Regional / seasonal variations

Benefits:

- **Field worker gets:**
    - More accurate root-cause suggestions
    - Context-aware recommendations
    - Reduced mean time to repair (MTTR)

Training allows the system to learn from operational history, not just apply generic NLP.

Architecture Diagram Covering Security, Governance & ML Platform (machine learning model training pipeline):



Architecture & Design Description

2025 Storage layer Architecture Evolution

Context

In 2019, Delta Lake was used as a unified storage format for batch analytics, curated datasets, and BI consumption. While Delta provided ACID guarantees and limited schema evolution, increasing data variety, streaming ingestion, and ML feature development introduced frequent schema changes that risked impacting downstream BI consumers.

By 2025, the platform required:

- Safer schema evolution
- Decoupling operational analytics from BI contracts
- Support for streaming, ML, and experimentation without destabilizing KPIs

Decision

Adopt a **dual table format strategy**:

- **Apache Iceberg** for operational, streaming, and ML-facing analytical tables
- **Delta Lake** for curated, governed fact and dimension tables powering BI

Iceberg absorbs schema volatility; Delta enforces analytics contracts.

Schema evolution occurs in Iceberg tables, where new attributes, event structures, and ML features can be introduced without impacting downstream consumers.

Schema governance occurs in Delta Lake tables, which represent stable analytical contracts for facts, dimensions, and KPIs consumed by Power BI.

This separation ensures that innovation and experimentation do not compromise BI reliability.

In the 2025 target architecture, the solution analytics platform introduces a dual table format strategy to decouple schema evolution from BI consumption. Apache Iceberg tables are used for operational analytics, streaming ingestion, and machine learning pipelines, where schemas evolve frequently and require flexibility. Curated analytical datasets, including fact and dimension tables supporting KPIs, remain stored in Delta Lake to preserve stable analytics contracts for Power BI. A dedicated Metadata & Control Plane governs schema versions, ingestion status, lineage, and promotion rules between Iceberg and Delta, enabling controlled evolution while maintaining analytical reliability.

Layer	**Purpose**	**Table format**
Analytics contract	Stable facts & dimensions	**Delta**
Operational analytics / streaming	Fast-changing schemas	**Iceberg**

Iceberg does not replace Delta for star schemas; it absorbs upstream schema volatility so that Delta remains a stable analytics contract for BI and KPIs.

Architecture & Design Description

Raw / Streaming

↓

Iceberg (operational analytical tables)

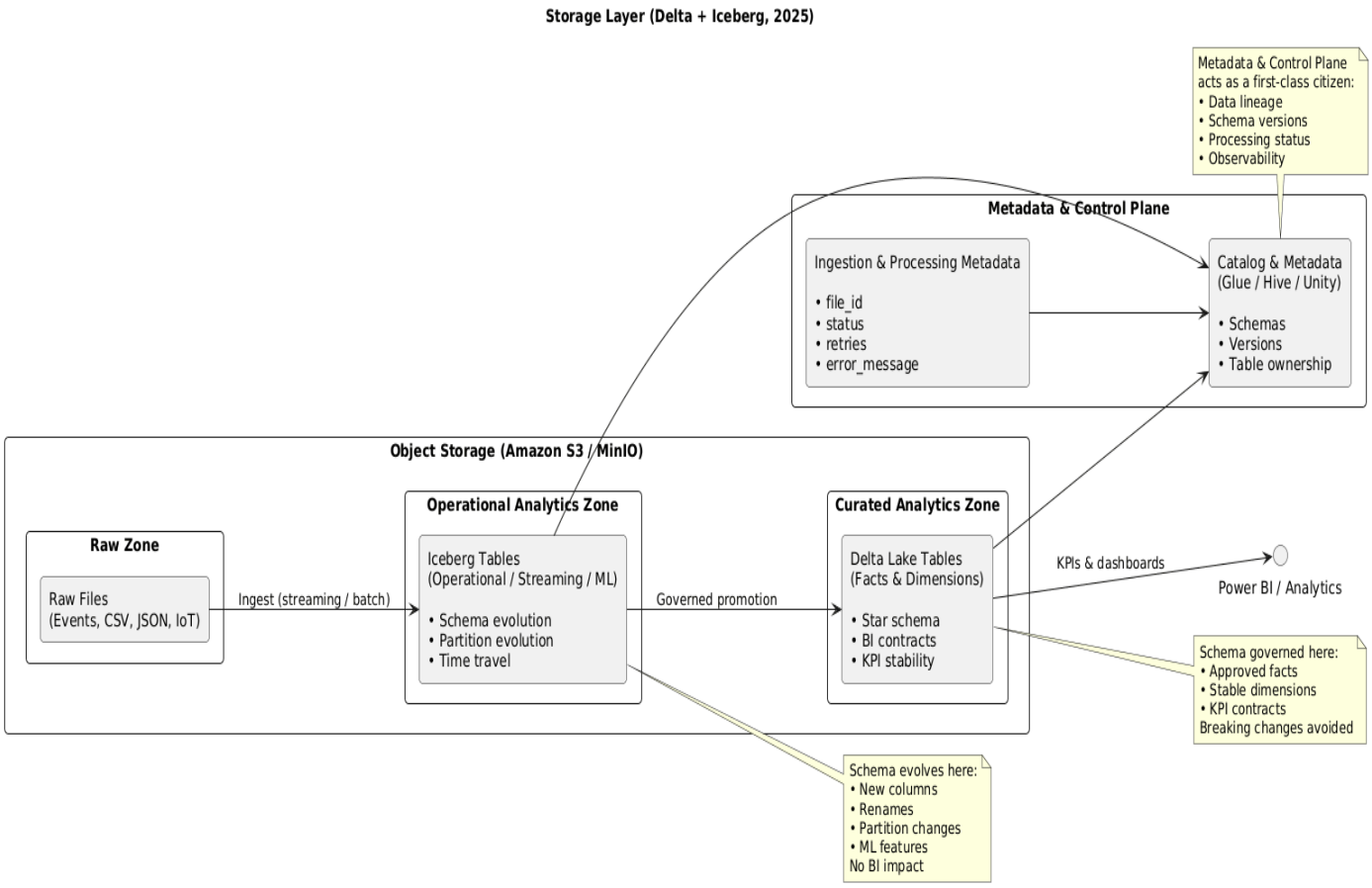
↓

Spark promotion

↓

Delta (facts & dimensions)

Storage Layer Architecture Diagram: Delta + Iceberg - 2025



The **Metadata & Control Plane** is **orthogonal to storage formats** and becomes first-class in 2025.

It manages:

- **Technical metadata**
    - Table schemas

Architecture & Design Description

    - Column versions
    - Snapshot lineage (Delta & Iceberg)
- **Operational metadata**
    - File-level ingestion status
    - Retry counters
    - Error diagnostics
    - Reprocessing eligibility
- **Governance metadata**
    - Which Iceberg tables are allowed to promote into Delta
    - Ownership & stewardship
    - BI exposure rules

Storage layer structure:

Object Storage (S3 / MinIO)

|

├ **Iceberg Tables**

| ├ Streaming outputs

| ├ Incremental aggregates

| ├ Feature computation inputs

|

├ **Delta Lake Tables**

| ├ Star schema (facts & dimensions)

| ├ KPI aggregates

| ├ BI-ready datasets

|

└ **Metadata & Control Plane**

  ├ Table schemas

  ├ Snapshots / versions

  ├ Quality & promotion rules

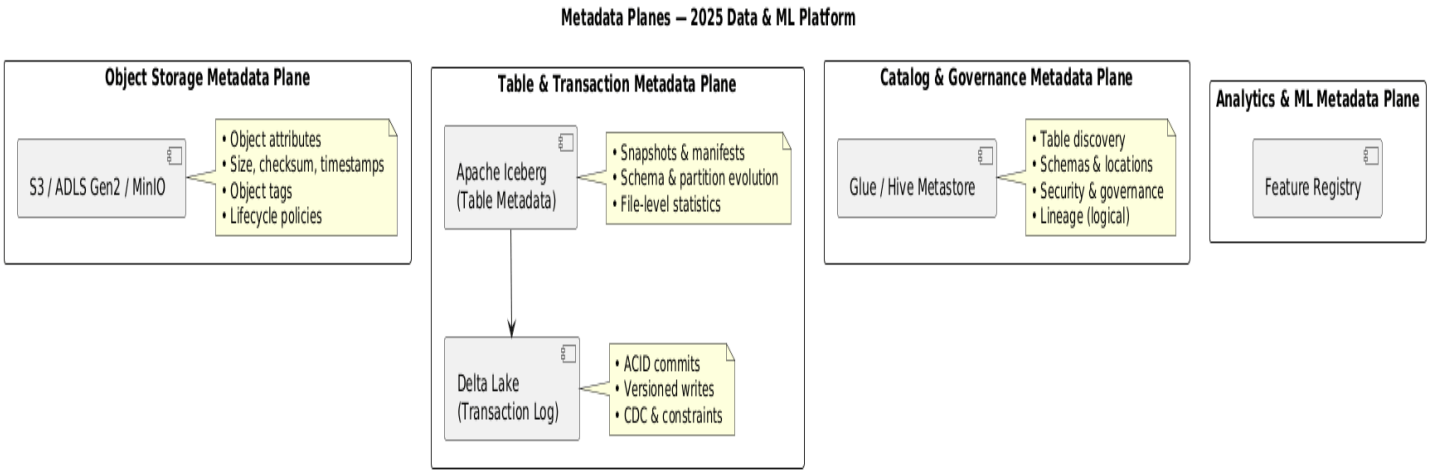
Where do frequent schema changes happen? In Iceberg tables, not in Delta. Iceberg is not introduced for BI facts. Iceberg is introduced for everything upstream of BI facts.

Metadata Planes for Storage Layer Architecture: Defining distinct logical planes, each responsible for a specific category of metadata.

- **Object Storage Metadata Plane (MinIO/ S3)**
  - Physical object attributes (size, checksum, timestamp, tags)
- **Table & Transaction Metadata Plane (Apache Iceberg, Delta Lake)**
  - Logical table state
  - File versions
  - ACID guarantees
  - Schema & partition evolution
- **Catalog & Governance Metadata Plane (AWS Glue, Hive Meta store)**
  - Dataset discovery
  - Schema & locations
  - Access control & governance
- **Analytics & ML Metadata Plane (Governed Iceberg/Delta Lake table, Feature Registry tables)**
  - Feature definitions & versions
  - Aggregation logic & encoding
  - Data quality metrics
  - Model-feature relationship
  - Analytical success indicators

Any metadata required to interpret, trust, reproduce, or govern analytical or ML results must live in the Analytics & ML Metadata Plane (logical entity) as data.

Metadata Planes Architecture Diagram



Feature registry tables would be within Iceberg or Delta Lake tables (recommended) in the curated zone (cataloged in Glue) or within Feast open-source feature store. We could also use Rego OPA engine (executing policies as code) as decision engine for feature promotion evaluating eligibility, quality, & compliance.

Architecture & Design Description

Implementing the Feature Registry as governed Delta/Iceberg tables in the curated zone (cataloged in Glue), rather than using Feast, avoids tight coupling to a specific ML serving framework and ensures that feature metadata remains auditable, queryable, and governed alongside analytical data.

Feature Registry (Iceberg / Delta tables)

↓

Feature Engineering Pipelines

↓

Offline Feature Store (Delta / Iceberg)

↓

Online Feature Store (Redis / DynamoDB)

KPI → Feature → Delta Promotion Logic:

In the 2025 architecture, KPIs are modeled as analytical features derived from Iceberg tables and registered in the Feature Store. Once validated, these KPI-grade features are promoted into governed Delta Lake fact tables, ensuring consistency between machine learning models and BI dashboards. This approach eliminates metric duplication and establishes a single semantic definition for both predictive and descriptive analytics.

How "KPIs as Features" Works (Conceptually)

2019 (implicit, limited)

- KPIs existed only in BI layer
- Defined via:
    - SQL views
    - Power BI DAX
- Not reusable for ML
- No lineage between KPIs and data pipelines

2025 (explicit, intentional)

KPIs are treated as **first-class analytical features**.

Flow:

1. **Raw + streaming data** lands in Iceberg
2. **Features are derived** (aggregations, rates, scores)
3. **Feature Store registers them**
4. **Two consumption paths**:
    - ML models (online/offline)

Architecture & Design Description

- Analytics (via Delta)

KPIs are **not re-computed separately** for BI
KPIs are **promoted features**

Mapping Examples

KPI	Iceberg Source	Feature Store	Delta Table
Content usage rate	learning_events_iceberg	content_usage_rate	fact_learning_engagement
Drop-off rate	session_events_iceberg	dropoff_rate	fact_learning_engagement
Recommendation acceptance	recommendation_events	rec_acceptance_rate	fact_ml_effectiveness
Error reduction post-training	operational_errors	error_reduction_score	fact_training_effectiveness

Storage layer for 2025

Iceberg: Where signals evolve and experimentation happens.

Feature Store: Where analytics and ML meet.

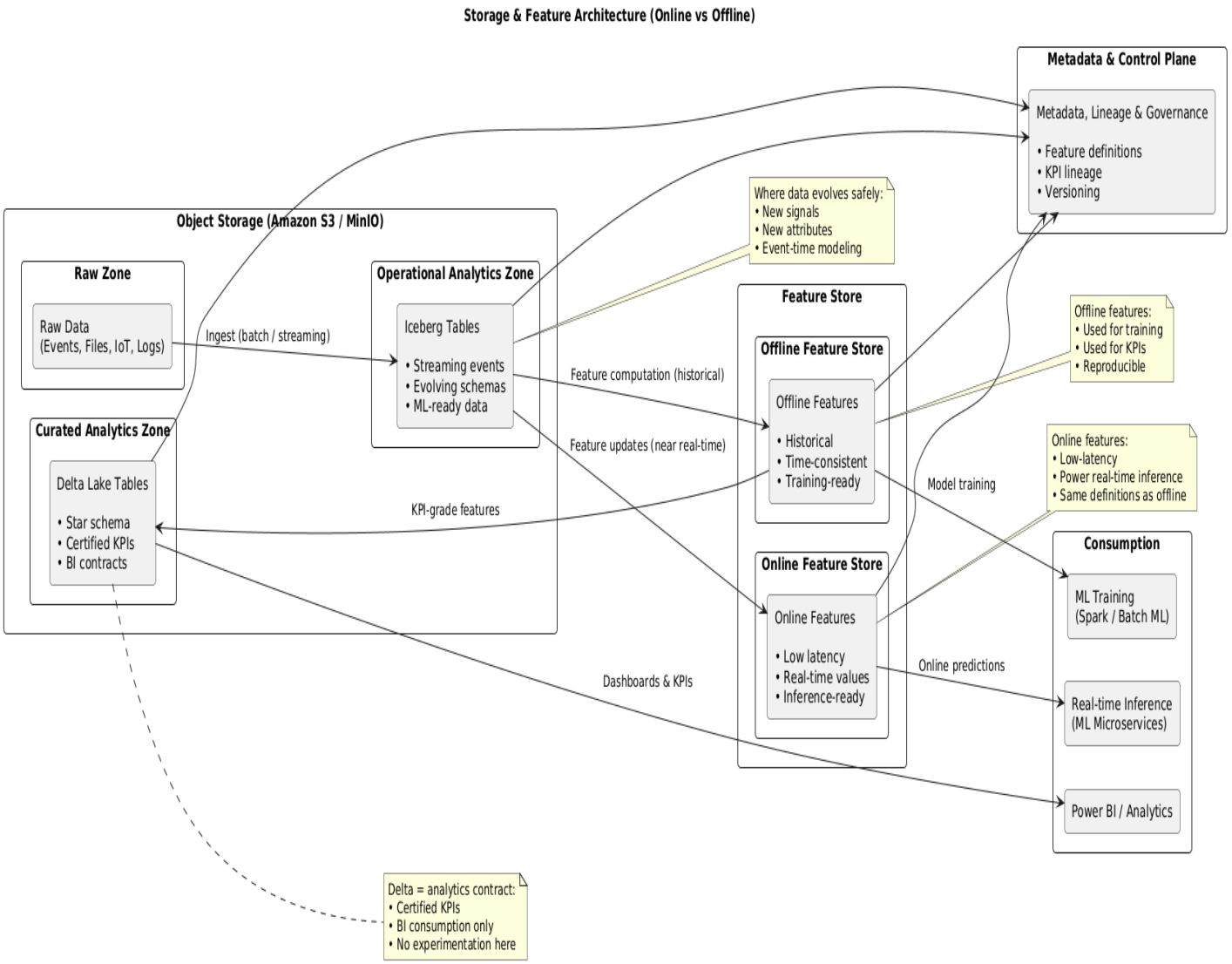
- **Offline features**
  - Training
  - KPI computation
  - Historical analysis
- **Online features**
  - Real-time inference
  - Operational decisions

Delta Lake: Where KPIs are governed and exposed.

Layer	Purpose
Iceberg	Source of truth for evolving signals
Offline Feature Store	KPI calculations & ML training
Delta Lake	Certified KPIs for BI
Power BI	Semantic layer & visualization

Architecture & Design Description

Storage & Feature Architecture Diagram (Online vs Offline) - 2025



Architecture & Design Description

Enterprise HR & Learning Systems Integration (SuccessFactors) with Policy Governance

The integration patterns established in 2019 with SAP HCM and LSO were preserved during the transition to SAP SuccessFactors, with minimal changes limited to connectivity and system boundaries, while authorization, metadata management, and content delivery remained platform-controlled.

Diagrams — SAP SuccessFactors Integration (SaaS)

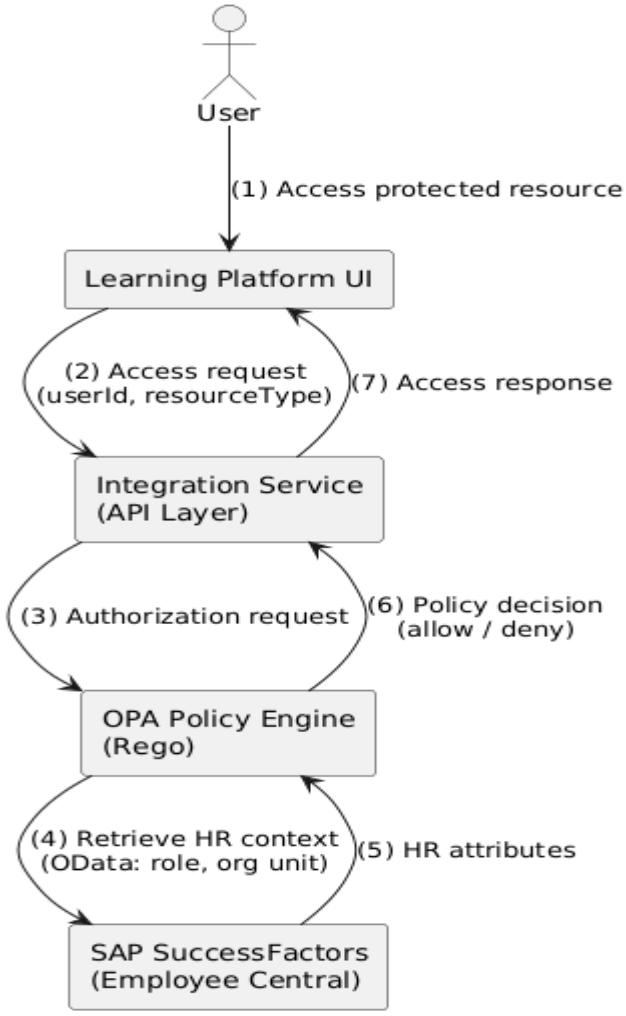
Diagram 1 — Authorization & HR Context Resolution (SuccessFactors)

(Who is the user, what are they allowed to see?)

Scope

- Identity & role resolution
- HR / organization context
- Centralized authorization via OPA

Logical Flow



Architecture & Design Description

Key architectural point

- **SuccessFactors = system of record for HR attributes**
- **OPA = single decision point**
- No business logic leaks into SAP

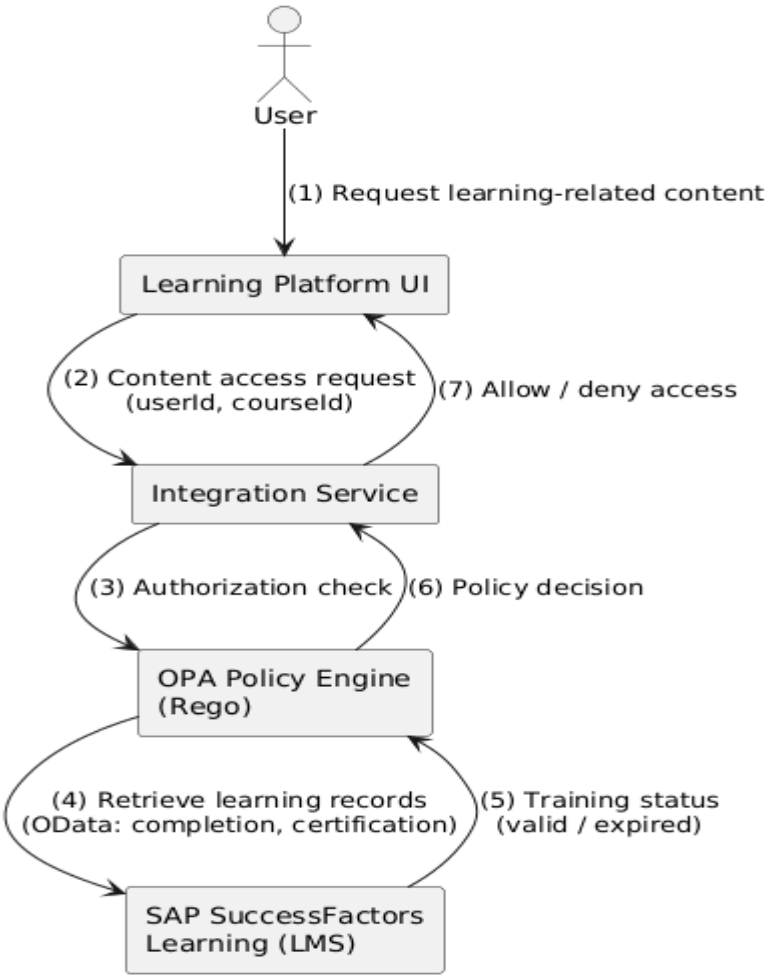
Diagram 2 — Learning & Certification Validation (SuccessFactors LMS)

(Is the user trained / certified to access the content?)

Scope

- LMS validation
- Training validity
- Certification expiry

Logical Flow



Architecture & Design Description

Key architectural point

- LMS checks are **policy inputs**, not business logic
- Clean separation between **training systems** and **content delivery**

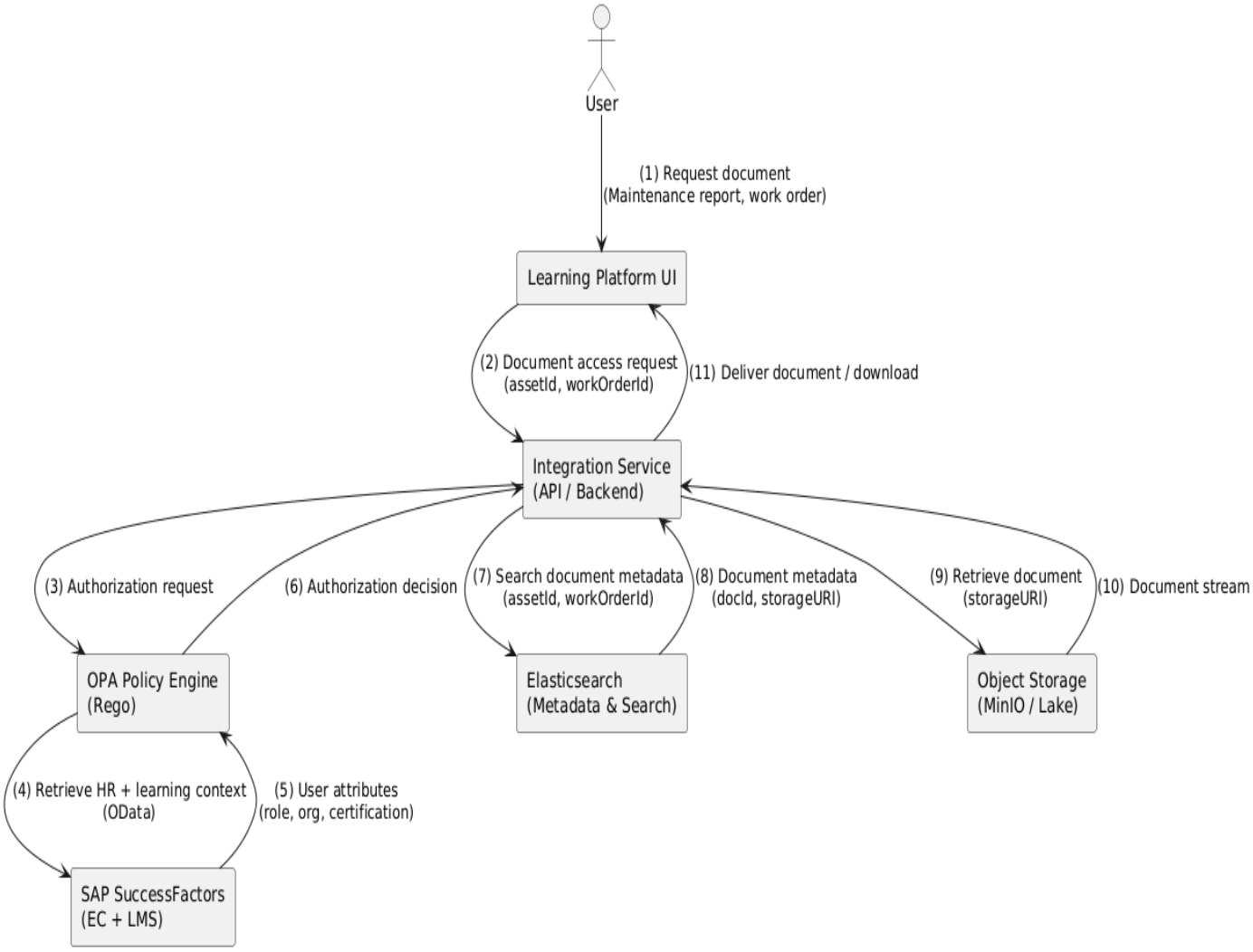
Diagram 3 — Document Access & Download Flow (SuccessFactors + OPA)

(Work orders, maintenance reports, operator notes, asset documents)

Scope

- Full end-to-end access flow
- Metadata vs content separation
- Explicit download path

Logical Flow



Architecture & Design Description

Key architectural clarification (important)

- SuccessFactors never stores documents
- Elasticsearch returns metadata only
- Object storage returns content
- OPA governs *every* sensitive access